HelixTerminator — API & Database Specification

Document 07: API and Database

Project: HelixTerminator
Backend: Go 1.25, Gin Gonic framework
Module: `helixterminator.io/core`
Database Stack: PostgreSQL 17.2 (primary), SQLite (dev/embedded), Redis 8 (cache/sessions)
Architecture: Database-per-service microservices (25 services)
API Style: REST (external clients) + gRPC (internal service-to-service)
Version: 1.0.0
Last Updated: 2026-06-28

Table of Contents

---

1. API Design Principles

1.1 Overview

HelixTerminator exposes a versioned HTTP/REST API for all external clients (web application, desktop client, CLI, third-party integrations). Internal microservice communication uses gRPC for low-latency, strongly-typed contracts. This document is the authoritative specification for both surfaces.

The REST API adheres to RFC 7231 (HTTP Semantics), RFC 7807 (Problem Details for HTTP APIs), RFC 8288 (Web Linking), and the OpenAPI 3.1 specification. All timestamps are RFC 3339 / ISO 8601 in UTC. All identifiers are UUID v4 (RFC 4122).

1.2 REST Conventions

Resource Naming

Resources are named as **plural nouns** in lowercase, with words separated by hyphens when needed. The URL structure follows a hierarchy that mirrors the domain model.

Pattern	Example	Meaning
`/api/v1/{resource}`	`/api/v1/hosts`	Collection
`/api/v1/{resource}/{id}`	`/api/v1/hosts/abc-123`	Single resource
`/api/v1/{resource}/{id}/{sub}`	`/api/v1/hosts/abc-123/connections`	Sub-collection
`/api/v1/{resource}/{id}/{action}`	`/api/v1/sessions/abc-123/resize`	Action on resource

Rules: - **Never** use verbs in resource URLs (wrong: `/getHosts`, `/createHost`). Use HTTP methods for actions. - **Always** use lowercase. Never camelCase or PascalCase in URLs. - Use hyphens (`-`) not underscores (_) in URL path segments. - Trailing slashes are **not**

allowed. `/api/v1/hosts/` returns `301 Moved Permanently` to `/api/v1/hosts`. - UUIDs in path parameters use lowercase hex with hyphens: `550e8400-e29b-41d4-a716-446655440000`.

HTTP Methods

Method	Semantics	Idempotent	Safe	Body
`GET`	Retrieve resource or collection	Yes	Yes	No
`POST`	Create resource or invoke action	No	No	Yes
`PUT`	Replace resource entirely	Yes	No	Yes
`PATCH`	Partial update (JSON Merge Patch, RFC 7396)	No	No	Yes
`DELETE`	Remove resource	Yes	No	Optional
`HEAD`	Same as GET but no body (used for existence checks)	Yes	Yes	No
`OPTIONS`	CORS preflight; also lists allowed methods	Yes	Yes	No

`PATCH` uses JSON Merge Patch semantics: send only the fields you want to change; omitted fields remain unchanged; set a field to `null` to clear it. Full replacement operations use `PUT`.

HTTP Status Codes

2xx Success

Code	Name	Usage
`200 OK`	Standard success	GET, PUT, PATCH with body in response
`201 Created`	Resource created	POST that creates a resource; `Location` header present
`202 Accepted`	Async operation started	Long-running jobs queued
`204 No Content`	Success, no body	DELETE, POST actions with no response body
`206 Partial Content`	Range response	Binary downloads with `Range` header

3xx Redirection

Code	Name	Usage
`301 Moved Permanently`	URL changed	Trailing slash removal
`302 Found`	Temporary redirect	OAuth callback redirects
`304 Not Modified`	Conditional GET hit	ETag / If-None-Match

4xx Client Errors

Code	Name	Usage
`400 Bad Request`	Malformed request	Syntax error, missing required field, type mismatch
`401 Unauthorized`	Not authenticated	Missing or invalid token; include `WWW-Authenticate` header
`403 Forbidden`	Authenticated but not authorized	Insufficient permissions for the resource/action

Code	Name	Usage
`404 Not Found`	Resource does not exist	Also returned deliberately for privacy (e.g., vault of another user)
`405 Method Not Allowed`	Wrong HTTP method	Include `Allow` header listing valid methods
`406 Not Acceptable`	Cannot satisfy `Accept` header	Server cannot produce the requested content type
`409 Conflict`	State conflict	Duplicate unique constraint, optimistic lock failure
`410 Gone`	Resource permanently deleted	Soft-deleted resource accessed after retention period
`413 Content Too Large`	Body exceeds limit	File upload too large
`415 Unsupported Media Type`	Wrong `Content-Type`	Send JSON without `Content-Type: application/json`
`422 Unprocessable Entity`	Semantic validation failure	Well-formed JSON but business rule violation
`429 Too Many Requests`	Rate limit exceeded	Include `Retry-After` header

5xx Server Errors

Code	Name	Usage
`500 Internal Server Error`	Unhandled exception	Should never be exposed; always log with trace ID
`502 Bad Gateway`	Upstream service failure	Microservice unreachable
`503 Service Unavailable`	Maintenance / overload	Include `Retry-After`
`504 Gateway Timeout`	Upstream timeout	gRPC or DB timeout

1.3 Versioning Strategy

HelixTerminator uses **URI path versioning**. The version segment is the second path component after the API root:

```
https://api.helixterminator.io/api/v1/hosts
https://api.helixterminator.io/api/v2/hosts
```

Version lifecycle:

Phase	Duration	Behavior
Current	Indefinite	Fully supported, receives bug fixes and features
Deprecated	12 months	Supported, deprecation notice in response headers, no new features
Sunset	3 months	`410 Gone` with migration guide URL

Deprecated endpoints include:

```
Deprecation: Sat, 01 Jan 2028 00:00:00 GMT
Sunset: Sat, 01 Apr 2028 00:00:00 GMT
Link: <https://docs.helixterminator.io/migration/v1-to-v2>;
rel="deprecation"
```

Breaking vs. non-breaking changes:

Non-breaking (no version bump required): - Adding new optional fields to response bodies - Adding new optional query parameters - Adding new endpoints - Adding new enum values to non-exhaustive enumerations

Breaking (require new version): - Removing fields from responses - Changing field types - Changing authentication requirements - Removing endpoints - Changing pagination semantics

Version routing in Go/Gin:

```
v1 := router.Group("/api/v1")
{
    v1.Use(middleware.AuthRequired())
    hosts := v1.Group("/hosts")
```

```
    {
        hosts.GET("", hostHandler.List)
        hosts.POST("", hostHandler.Create)
        hosts.GET("/:hostId", hostHandler.Get)
        hosts.PUT("/:hostId", hostHandler.Update)
        hosts.DELETE("/:hostId", hostHandler.Delete)
    }
}

v2 := router.Group("/api/v2")
{
    v2.Use(middleware.AuthRequired())
    // v2-specific routing
}
```

1.4 Pagination

All collection endpoints that may return more than 20 items support cursor-based pagination. Cursor-based pagination is preferred over offset-based pagination because:

1. It is stable: inserting or deleting records between pages does not cause items to be skipped or duplicated.
2. It scales: no `COUNT(*)` or `OFFSET` scans required; just `WHERE id > $cursor LIMIT $pageSize`.
3. It is efficient for large datasets (e.g., audit logs with millions of rows).

Cursor Pagination Request Parameters

Parameter	Type	Default	Description
`cursor`	string	(none)	Opaque cursor returned by previous page
`limit`	integer	`25`	Number of items per page (1–100)
`sort`	string	`created_at:desc`	Sort field and direction, colon-separated
`direction`	string	`next`	`next` or `prev` — direction of

Parameter	Type	Default	Description
			cursor traversal

The `cursor` value is a **base64url-encoded JSON object** containing enough state to reconstruct the query position. It is opaque to clients.

Example cursor payload (before encoding):

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "created_at": "2026-01-15T10:30:00Z",
  "sort_field": "created_at",
  "direction": "next"
}
```

Encoded:
`eyJpZCI6IjU1MGU4NDAwLWUyOWItNDFkNC1hNzE2LTQ0NjY1NTQ0MDAwMCIsImNyZWF0ZWRfYXQiOiIyMDI2LTAxLTE1VDEwOjMwOjAwWiIsInNvcnRfZmllbGQiOiJjcmVhdGVkX2F0IiwiZGlyZWN0aW9uIjoibmV4dCJ9`

Cursor Pagination Response Envelope

All collection responses use a standard envelope:

```
{
  "data": [...],
  "pagination": {
    "cursor_next": "eyJpZCI6...",
    "cursor_prev": "eyJpZCI6...",
    "has_next": true,
    "has_prev": false,
    "limit": 25,
    "total_count": 1847
  }
}
```

`total_count` is only included when the query cost is acceptable (small collections). For large tables it is omitted or approximated.

Offset Pagination (Legacy / Admin)

Certain admin endpoints that require random access (e.g., audit log export with page jumps) support offset pagination:

Parameter	Type	Default
`page`	integer	`1`
`per_page`	integer	`25` (max `100`)

Response:

```
{
  "data": [...],
  "pagination": {
    "page": 3,
    "per_page": 25,
    "total_pages": 74,
    "total_count": 1847
  }
}
```

1.5 Filtering, Sorting, and Searching

Filtering

Filters are passed as query parameters. Multi-value filters use repeated parameters or comma-separated values.

```
GET /api/v1/hosts?status=active&os=linux
GET /api/v1/hosts?tag=production,staging
GET /api/v1/audit/events?
event_type=login_success&event_type=login_failure
```

Range filters use `_gte`, `_lte`, `_gt`, `_lt` suffixes:

```
GET /api/v1/audit/events?
created_at_gte=2026-01-01T00:00:00Z&created_at_lte=2026-02-01T00:00
```

Boolean filters accept `true`/`false`:

```
GET /api/v1/hosts?jump_enabled=true
```

Sorting

The `sort` parameter accepts a comma-separated list of `field:direction` pairs:

```
GET /api/v1/hosts?sort=name:asc
GET /api/v1/hosts?sort=created_at:desc,name:asc
```

Allowed sort directions: `asc`, `desc`. Invalid field names return `400 Bad Request`.

Full-Text Search

The `q` parameter activates full-text search (backed by PostgreSQL `pg_trgm` trigram indexes or `tsvector` full-text search):

```
GET /api/v1/hosts?q=prod-web
GET /api/v1/snippets/search?q=git+rebase
```

Search scoring is returned in `_score` when `q` is present.

Field Selection (Sparse Fieldsets)

To reduce response payload, clients may request specific fields:

```
GET /api/v1/hosts?fields=id,name,hostname,status
```

This is a performance optimization hint; the server may return additional fields if they are always required (e.g., `id`).

1.6 Error Response Format

All errors follow RFC 7807 "Problem Details for HTTP APIs". The `Content-Type` is `application/problem+json`.

```
{
  "type": "https://errors.helixterminator.io/v1/validation-error",
  "title": "Validation Error",
  "status": 422,
  "detail": "The request body contains invalid field values.",
  "instance": "/api/v1/hosts",
  "trace_id": "7f3a9b2c-1d4e-5f6a-8b9c-0d1e2f3a4b5c",
  "errors": [
    {
      "field": "hostname",
      "code": "required",
      "message": "hostname is required"
    },
    {
      "field": "port",
      "code": "range",
      "message": "port must be between 1 and 65535",
      "value": 99999
```

```
    }
  ]
}
```

Standard error type URIs:

Type URI	HTTP Status	Description
`errors.helixterminator.io/v1/validation-error`	422	Field-level validation failures
`errors.helixterminator.io/v1/authentication-required`	401	No valid authentication credentials
`errors.helixterminator.io/v1/forbidden`	403	Authenticated but lacks permission
`errors.helixterminator.io/v1/not-found`	404	Resource does not exist
`errors.helixterminator.io/v1/conflict`	409	Conflicting state (duplicate, optimistic lock)
`errors.helixterminator.io/v1/rate-limit-exceeded`	429	Rate limit hit
`errors.helixterminator.io/v1/internal-error`	500	Server error (sanitized — no stack trace)
`errors.helixterminator.io/v1/upstream-error`	502	Dependent service unavailable
`errors.helixterminator.io/v1/timeout`	504	Request processing exceeded deadline

1.7 Authentication

All authenticated endpoints require a Bearer token in the `Authorization` header:

```
Authorization: Bearer eyJhbGciOiJFZERTQSIsInR5cCI6IkpXVCJ9...
```

HelixTerminator uses **EdDSA (Ed25519) signed JWTs** (RFC 8037). Token structure:

```
{
  "iss": "https://auth.helixterminator.io",
  "sub": "550e8400-e29b-41d4-a716-446655440000",
```

```
  "aud": ["helixterm:api"],
  "exp": 1751120400,
  "iat": 1751116800,
  "jti": "unique-token-id",
  "scope": "api:read api:write",
  "org_id": "org-uuid",
  "session_id": "session-uuid",
  "mfa_verified": true
}
```

Token lifetimes:

Token Type	Lifetime	Storage
Access token	15 minutes	Memory (client)
Refresh token	30 days (sliding)	HttpOnly Secure cookie or secure storage
API key token	No expiry (until revoked)	Hashed in DB
Session token (WebSocket)	Duration of connection	Redis

API Keys use the format `htk_v1_<base62-secret>` and are authenticated via:

```
Authorization: Bearer
htk_v1_abcdefghijklmnopqrstuvwxyz0123456789ABCDEFGHIJ
```

or

```
X-API-Key: htk_v1_abcdefghijklmnopqrstuvwxyz0123456789ABCDEFGHIJ
```

1.8 Rate Limiting

Rate limits are enforced at multiple layers:

Layer	Limit	Window	Scope
Global (unauthenticated)	60 req	1 minute	Per IP
Global (authenticated)	1000 req	1 minute	Per user
Auth endpoints	10 req	1 minute	Per IP
Sensitive operations	5 req	15 minutes	Per user
AI endpoints	100 req	1 hour	Per user

Layer	Limit	Window	Scope
File upload	50 req	1 hour	Per user
WebSocket connections	20 concurrent	—	Per user

Rate limit response headers (always present on every response):

```
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 847
X-RateLimit-Reset: 1751117460
X-RateLimit-Policy: authenticated;q=1000;w=60
Retry-After: 47
```

On `429 Too Many Requests`:

```
{
  "type": "https://errors.helixterminator.io/v1/rate-limit-
        exceeded",
  "title": "Rate Limit Exceeded",
  "status": 429,
  "detail":
        "You have exceeded the rate limit of 1000 requests per 60
        seconds.",
  "retry_after": 47,
  "limit": 1000,
  "window_seconds": 60
}
```

Rate limiting uses a **sliding window counter** stored in Redis with the key pattern `rl:{scope}:{identifier}:{window_start}`.

1.9 HATEOAS Links

Responses include a `_links` object following the HAL (Hypertext Application Language) specification (draft-kelly-json-hal):

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "prod-web-01",
  "hostname": "192.168.1.100",
  "_links": {
    "self": {
      "href": "https://api.helixterminator.io/api/v1/hosts/
        550e8400-e29b-41d4-a716-446655440000"
    },
    "connections": {
```

```
      "href": "https://api.helixterminator.io/api/v1/hosts/
        550e8400-e29b-41d4-a716-446655440000/connections"
    },
    "group": {
      "href": "https://api.helixterminator.io/api/v1/groups/
        88e7f30c-1234-5678-abcd-ef0123456789"
    },
    "vault": {
      "href": "https://api.helixterminator.io/api/v1/vaults/
        aabbccdd-0011-2233-4455-66778899aabb"
    }
  }
}
```

Collection responses include pagination links:

```
{
  "_links": {
    "self": { "href": "https://api.helixterminator.io/api/v1/
        hosts?cursor=abc&limit=25" },
    "next": { "href": "https://api.helixterminator.io/api/v1/
        hosts?cursor=def&limit=25" },
    "prev": { "href": "https://api.helixterminator.io/api/v1/
        hosts?cursor=xyz&limit=25&direction=prev" }
  }
}
```

1.10 Request ID and Tracing

Every request is assigned a unique trace ID: - If the client sends `X-Request-ID: <uuid>`, that value is used. - Otherwise, the server generates a UUID v4.

The trace ID is echoed back in every response:

```
X-Request-ID: 7f3a9b2c-1d4e-5f6a-8b9c-0d1e2f3a4b5c
X-Trace-ID: 7f3a9b2c-1d4e-5f6a-8b9c-0d1e2f3a4b5c
```

All log entries and error responses include this ID for correlation.

1.11 Content Negotiation

All API requests and responses use `application/json` unless stated otherwise.

Endpoint Type	Request Content-Type	Response Content-Type
Standard REST	`application/json`	`application/json`
File upload	`multipart/form-data`	`application/json`
File download	(none)	`application/octet-stream` or specific MIME
Error responses	—	`application/problem+json`
Session recording	(none)	`application/x-asciicast` (asciinema v2)
SSH config export	(none)	`text/plain; charset=utf-8`

1.12 OpenAPI 3.1 Specification Overview

The machine-readable OpenAPI 3.1 specification is available at:

```
GET /api/v1/openapi.json   — OpenAPI 3.1 document
GET /api/v1/openapi.yaml   — YAML version
GET /api/v1/docs           — Swagger UI
GET /api/v1/redoc          — ReDoc UI
```

The OpenAPI document uses the following component structure:

```yaml
openapi: "3.1.0"
info:
  title: HelixTerminator API
  version: "1.0.0"
  contact:
    name: HelixTerminator Engineering
    email: api@helixterminator.io
  license:
    name: Proprietary

servers:
  - url: https://api.helixterminator.io/api/v1
    description: Production
  - url: https://staging-api.helixterminator.io/api/v1
    description: Staging
  - url: http://localhost:8080/api/v1
    description: Local development
```

```yaml
components:
  securitySchemes:
    BearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
    ApiKeyHeader:
      type: apiKey
      in: header
      name: X-API-Key

security:
  - BearerAuth: []
```

> **DEFERRED (next increment):** Only the top-level OpenAPI document structure and security schemes are shown above. The full `components.schemas` request/response JSON Schema definitions for every endpoint in §2–§14 are not yet authored in this document; treat the per-endpoint JSON examples in this spec as illustrative, not as a substitute for a generated/validated schema.

1.13 Idempotency Keys

Mutating operations (POST, PATCH) that create resources or trigger actions support idempotency keys to prevent duplicate processing on retry:

```
Idempotency-Key: <client-generated-uuid>
```

The server caches the response for 24 hours keyed by `{user_id}:{idempotency_key}`. If the same key is replayed, the original response is returned without re-executing the operation. The response includes:

```
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
Idempotency-Replayed: true
```

> **DEFERRED (next increment):** "Support" above is described generically for all mutating operations; a definitive per-endpoint matrix stating which POST/PATCH operations *require* an `Idempotency-Key` (vs merely accept one) is not yet authored across §2–§14. Do not assume every mutating endpoint enforces it until that matrix exists.

1.14 Conditional Requests and ETags

GET responses include `ETag` and `Last-Modified` headers:

```
ETag: "33a64df551425fcc55e4d42a148795d9f25f89d4"
Last-Modified: Wed, 28 Jun 2026 10:00:00 GMT
Cache-Control: private, no-cache
```

Clients use conditional requests to avoid re-downloading unchanged data:

```
GET /api/v1/hosts/550e8400 HTTP/1.1
If-None-Match: "33a64df551425fcc55e4d42a148795d9f25f89d4"
```

Returns `304 Not Modified` if unchanged.

For optimistic concurrency on updates:

```
PUT /api/v1/hosts/550e8400 HTTP/1.1
If-Match: "33a64df551425fcc55e4d42a148795d9f25f89d4"
```

Returns `412 Precondition Failed` if the resource was modified since the ETag was retrieved.

1.15 CORS

Cross-Origin Resource Sharing is configured as follows:

```
Access-Control-Allow-Origin: https://app.helixterminator.io
Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE,
OPTIONS
Access-Control-Allow-Headers: Authorization, Content-Type, X-
Request-ID, Idempotency-Key, X-API-Key
Access-Control-Expose-Headers: X-Request-ID, X-RateLimit-Limit, X-
RateLimit-Remaining, X-RateLimit-Reset, ETag
Access-Control-Max-Age: 86400
Access-Control-Allow-Credentials: true
```

In development, `Access-Control-Allow-Origin: *` is permitted (credentials excluded).

---

2. Auth Service REST API

Base path: `/api/v1/auth`

Service: `auth-service` (port 8081 internal, 443 external via gateway) Database: `auth_db` (PostgreSQL)

POST /api/v1/auth/register

Create a new user account.

Authentication: None required
Rate Limit: 5 requests per IP per 15 minutes

Request Headers:

```
Content-Type: application/json
X-Request-ID: <uuid> (optional)
```

Request Body:

```
{
  "email": "alice@example.com",
  "password": "SecureP@ssw0rd!",
  "display_name": "Alice Smith",
  "invite_code": "HELIX-INVITE-ABCD1234",
  "accept_terms": true,
  "locale": "en-US",
  "timezone": "America/New_York"
}
```

Field	Type	Required	Constraints
`email`	string	Yes	Valid email, max 255 chars
`password`	string	Yes	Min 12 chars, complexity requirements
`display_name`	string	Yes	2–100 chars, UTF-8
`invite_code`	string	No	

Field	Type	Required	Constraints
			Required if registration is invite-only
`accept_terms`	boolean	Yes	Must be `true`
`locale`	string	No	BCP 47 locale tag
`timezone`	string	No	IANA timezone name

Success Response — 201 Created:

```
{
  "user": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "email": "alice@example.com",
    "display_name": "Alice Smith",
    "status": "active",
    "email_verified": false,
    "created_at": "2026-06-28T17:40:00Z"
  },
  "tokens": {
    "access_token": "eyJhbGciOiJFZERTQSIsInR5cCI6IkpXVCJ9...",
    "token_type": "Bearer",
    "expires_in": 900,
    "refresh_token": "rt_v1_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
    "scope": "api:read api:write"
  },
  "requires_email_verification": true,
  "_links": {
    "self": { "href": "https://api.helixterminator.io/api/v1/
        users/me" },
    "verify_email": { "href": "https://api.helixterminator.io/api/
        v1/auth/email/verify" }
  }
}
```

Error Responses:

Status	Error Type	Condition
400	`validation-error`	Missing/malformed fields
409	`conflict`	Email already registered
422	`validation-error`	Password too weak, invalid email

Status	Error Type	Condition
429	`rate-limit-exceeded`	Too many registration attempts from IP

POST /api/v1/auth/login

Authenticate with email and password. Returns tokens or MFA challenge.

Authentication: None required
Rate Limit: 10 requests per IP per minute; 5 per account per minute

Request Headers:

```
Content-Type: application/json
User-Agent: HelixTerminator/1.0 (macOS 14.2)
X-Device-ID: <client-device-uuid> (optional, for trusted device tracking)
```

Request Body:

```
{
  "email": "alice@example.com",
  "password": "SecureP@ssw0rd!",
  "device_name": "Alice's MacBook Pro",
  "device_fingerprint": "sha256:abcdef1234567890...",
  "remember_me": true
}
```

Success Response (no MFA) — 200 OK:

```
{
  "status": "authenticated",
  "user": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "email": "alice@example.com",
    "display_name": "Alice Smith",
    "avatar_url": "https://cdn.helixterminator.io/avatars/
        alice.jpg",
    "status": "active",
    "email_verified": true,
    "last_login_at": "2026-06-27T09:00:00Z",
    "mfa_enabled": false,
    "org_id": "org-550e8400-0000-0000-0000-000000000001"
  },
```

```
  "tokens": {
    "access_token": "eyJhbGciOiJFZERTQSIsInR5cCI6IkpXVCJ9...",
    "token_type": "Bearer",
    "expires_in": 900,
    "refresh_token": "rt_v1_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
    "scope": "api:read api:write"
  },
  "session": {
    "id": "sess-550e8400-0000-0000-0000-aabbccddeeff",
    "device_id": "dev-550e8400-0000-0000-0000-112233445566",
    "created_at": "2026-06-28T17:40:00Z",
    "expires_at": "2026-07-28T17:40:00Z"
  }
}
```

MFA Required Response — 200 OK:

```
{
  "status": "mfa_required",
  "mfa_challenge": {
    "challenge_id": "mfa-chal-550e8400-0000-0000-0000-
        aabbccdd0001",
    "methods": ["totp", "fido2"],
    "expires_at": "2026-06-28T17:45:00Z"
  }
}
```

Error Responses:

Status	Error Type	Condition
400	`validation-error`	Malformed request
401	`authentication-required`	Invalid credentials
403	`forbidden`	Account suspended or deleted
423	`locked`	Account temporarily locked after failed attempts
429	`rate-limit-exceeded`	Too many login attempts

POST /api/v1/auth/logout

Invalidate the current session and access token.

Authentication: Bearer token required

Request Body (optional):

```
{
  "all_sessions": false
}
```

Setting `all_sessions: true` invalidates all active sessions for the user.

Success Response — 204 No Content

Error Responses:

Status	Condition
401	Not authenticated

POST /api/v1/auth/refresh

Exchange a refresh token for a new access token.

Authentication: None required (refresh token in body or cookie)

Request Body:

```
{
  "refresh_token": "rt_v1_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
}
```

If using cookie-based refresh tokens, the token is read from the `__Host-refresh_token` HttpOnly Secure cookie and the body can be empty.

Success Response — 200 OK:

```
{
  "access_token": "eyJhbGciOiJFZERTQSIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 900,
  "refresh_token": "rt_v1_yyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy",
  "scope": "api:read api:write"
}
```

The old refresh token is immediately invalidated (token rotation). The new refresh token extends the sliding window by 30 days.

Error Responses:

Status	Condition
401	Refresh token invalid, expired, or already used
403	Session revoked by admin

POST /api/v1/auth/mfa/totp/setup

Initiate TOTP (Time-based One-Time Password) setup. Returns the secret and provisioning URI.

Authentication: Bearer token required (access token, pre-MFA-verification)

Request Body: Empty `{}`

Success Response — 200 OK:

```
{
  "secret": "JBSWY3DPEHPK3PXP",
  "provisioning_uri": "otpauth://totp/HelixTerminator:alice%40example.com?
        secret=JBSWY3DPEHPK3PXP&issuer=HelixTerminator&algorithm=SHA1&digits=6&per
  "qr_code_url": "https://api.helixterminator.io/api/v1/auth/mfa/
        totp/qr?token=setup-xyz",
  "backup_codes": [
    "AAAA-BBBB-CCCC",
    "DDDD-EEEE-FFFF",
    "GGGG-HHHH-IIII",
    "JJJJ-KKKK-LLLL",
    "MMMM-NNNN-OOOO",
    "PPPP-QQQQ-RRRR",
    "SSSS-TTTT-UUUU",
    "VVVV-WWWW-XXXX"
  ],
  "setup_token": "setup-token-expires-in-10-minutes",
  "expires_at": "2026-06-28T17:50:00Z"
}
```

The secret is not persisted until `POST /api/v1/auth/mfa/totp/verify` succeeds.

Error Responses:

Status	Condition
401	Not authenticated
409	TOTP already enabled for this account

POST /api/v1/auth/mfa/totp/verify

Verify a TOTP code to complete setup or authenticate.

Authentication: Bearer token required OR `mfa_challenge_id` in body

Request Body:

```
{
  "code": "123456",
  "setup_token": "setup-token-expires-in-10-minutes",
  "challenge_id": "mfa-chal-550e8400-0000-0000-0000-aabbccdd0001"
}
```

Provide `setup_token` when completing setup, `challenge_id` when authenticating.

Success Response (setup completion) — 200 OK:

```
{
  "enabled": true,
  "backup_codes_remaining": 8,
  "message": "TOTP successfully enabled."
}
```

Success Response (authentication) — 200 OK:

```
{
  "status": "authenticated",
  "tokens": {
    "access_token": "eyJhbGciOiJFZERTQSIsInR5cCI6IkpXVCJ9...",
    "token_type": "Bearer",
    "expires_in": 900,
    "refresh_token": "rt_v1_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
    "scope": "api:read api:write"
  }
}
```

Error Responses:

Status	Condition
400	Invalid code format
401	TOTP code incorrect or expired
410	Challenge expired
429	Too many TOTP attempts

POST /api/v1/auth/mfa/fido2/register/begin

Begin WebAuthn/FIDO2 credential registration. Returns a challenge for the authenticator.

Authentication: Bearer token required

Request Body:

```
{
  "authenticator_name": "YubiKey 5C",
  "authenticator_attachment": "cross-platform"
}
```

`authenticator_attachment`: `"platform"` (biometrics, TPM), `"cross-platform"` (security key), `null` (any).

Success Response — 200 OK:

```
{
  "registration_id": "reg-550e8400-0000-0000-0000-aabbccdd0002",
  "options": {
    "challenge": "dGhpcyBpcyBhIHRlc3QgY2hhbGxlbmdl",
    "rp": {
      "id": "helixterminator.io",
      "name": "HelixTerminator"
    },
    "user": {
      "id": "VVVlMDg0MDAtZTI5Yi00MWQ0LWE3MTYtNDQ2NjU1NDQwMDAw",
      "name": "alice@example.com",
      "displayName": "Alice Smith"
    },
    "pubKeyCredParams": [
      { "type": "public-key", "alg": -8 },
      { "type": "public-key", "alg": -7 },
      { "type": "public-key", "alg": -257 }
    ],
```

```
    "timeout": 60000,
    "authenticatorSelection": {
      "authenticatorAttachment": "cross-platform",
      "requireResidentKey": false,
      "userVerification": "preferred"
    },
    "attestation": "indirect"
  },
  "expires_at": "2026-06-28T17:41:00Z"
}
```

Error Responses:

Status	Condition
401	Not authenticated

POST /api/v1/auth/mfa/fido2/register/complete

Complete WebAuthn/FIDO2 credential registration with authenticator response.

Authentication: Bearer token required

Request Body:

```
{
  "registration_id": "reg-550e8400-0000-0000-0000-aabbccdd0002",
  "authenticator_name": "YubiKey 5C",
  "credential": {
    "id": "credentialIdBase64url",
    "rawId": "credentialIdBase64url",
    "type": "public-key",
    "response": {
      "clientDataJSON":
        "eyJ0eXBlIjoid2ViYXV0aG4uY3JlYXRlIiwiY2hhbGxlbmdlIjoiZEdocGN5Qn...",
      "attestationObject":
        "o2NmbXRmcGFja2VkZ2F0dFN0bXSiY2FsZyZjc2lnWEYwRAIgM..."
    }
  }
}
```

Success Response — 201 Created:

```
{
  "credential_id": "cred-550e8400-0000-0000-0000-aabbccdd0003",
```

```
  "authenticator_name": "YubiKey 5C",
  "credential_type": "public-key",
  "created_at": "2026-06-28T17:40:30Z",
  "aaguid": "2fc0579f-8113-47ea-b116-bb5a8db9202a",
  "transports": ["usb", "nfc"]
}
```

POST /api/v1/auth/mfa/fido2/authenticate/begin

Begin FIDO2 authentication challenge.

Authentication: None (called after password verification, before full auth)

Request Body:

```
{
  "challenge_id": "mfa-chal-550e8400-0000-0000-0000-aabbccdd0001",
  "user_verification": "preferred"
}
```

Success Response — 200 OK:

```
{
  "fido2_challenge_id": "fido2-chal-550e8400-0000-0000-0000-
       aabbccdd0004",
  "options": {
    "challenge": "bGV0J3MgdGVzdCBGSURPMiBhdXRoZW50aWNhdGlvbg==",
    "timeout": 60000,
    "rpId": "helixterminator.io",
    "allowCredentials": [
      {
        "type": "public-key",
        "id": "credentialIdBase64url",
        "transports": ["usb", "nfc"]
      }
    ],
    "userVerification": "preferred"
  },
  "expires_at": "2026-06-28T17:41:00Z"
}
```

POST /api/v1/auth/mfa/fido2/authenticate/complete

Complete FIDO2 authentication.

Authentication: None (completes the MFA flow)

Request Body:

```
{
  "fido2_challenge_id": "fido2-chal-550e8400-0000-0000-0000-
        aabbccdd0004",
  "credential": {
    "id": "credentialIdBase64url",
    "rawId": "credentialIdBase64url",
    "type": "public-key",
    "response": {
      "clientDataJSON":
        "eyJ0eXBlIjoid2ViYXV0aG4uZ2V0IiwiY2hhbGxlbmdlIjoiYkdWMEp5...",
      "authenticatorData": "SZYN5YgOjGh0NBcPZHZgW4/
        krrmihjLHmVzzuoMdl2MBAAAABg==",
      "signature": "MEQCIBBgCLz3rRBOJb/
        mBLKIVHmRUkD9dR3JCZ0u08hPh8M5AiB4...",
      "userHandle": "VVVlMDg0MDAtZTI5Yi00MWQ0..."
    }
  }
}
```

Success Response — 200 OK:

```
{
  "status": "authenticated",
  "tokens": {
    "access_token": "eyJhbGciOiJFZERTQSIsInR5cCI6IkpXVCJ9...",
    "token_type": "Bearer",
    "expires_in": 900,
    "refresh_token": "rt_v1_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
    "scope": "api:read api:write"
  }
}
```

GET /api/v1/auth/devices

List all trusted devices for the authenticated user.

Authentication: Bearer token required

Query Parameters: | Parameter | Type | Description | |---|---|---| | `cursor` | string | Pagination cursor | | `limit` | integer | Page size (default 25) |

Success Response — 200 OK:

```
{
  "data": [
    {
      "id": "dev-550e8400-0000-0000-0000-112233445566",
      "name": "Alice's MacBook Pro",
      "fingerprint": "sha256:abcdef1234567890...",
      "platform": "macOS 14.2",
      "user_agent": "HelixTerminator/1.0 (macOS 14.2)",
      "trusted": true,
      "last_seen_at": "2026-06-28T17:40:00Z",
      "last_seen_ip": "192.168.1.10",
      "created_at": "2026-01-15T09:00:00Z",
      "is_current": true
    }
  ],
  "pagination": {
    "cursor_next": null,
    "has_next": false,
    "limit": 25,
    "total_count": 1
  }
}
```

DELETE /api/v1/auth/devices/{deviceId}

Revoke a trusted device. Future logins from this device will require full authentication.

Authentication: Bearer token required

Path Parameters: | Parameter | Type | Description | |---|---|---| | `deviceId` | UUID | Device identifier |

Success Response — 204 No Content

Error Responses:

Status	Condition
404	Device not found
403	Device belongs to another user

POST /api/v1/auth/sso/{provider}/authorize

Initiate SSO OAuth2/OIDC authorization flow.

Authentication: None required

Path Parameters: | Parameter | Type | Description | |---|---|---| | `provider` | string | SSO provider slug (e.g., `github`, `google`, `azure`, `okta`) |

Request Body:

```
{
  "redirect_uri": "https://app.helixterminator.io/auth/callback",
  "state": "client-generated-random-state",
  "org_slug": "mycompany"
}
```

Success Response — 200 OK:

```
{
  "authorization_url": "https://github.com/login/oauth/authorize?
        client_id=...&redirect_uri=...&state=...&scope=read%3Auser%20user%3Aemail"
  "state": "server-generated-state-token",
  "expires_at": "2026-06-28T17:50:00Z"
}
```

POST /api/v1/auth/sso/{provider}/callback

Handle SSO callback with authorization code.

Authentication: None required

Request Body:

```
{
  "code": "oauth2-authorization-code",
  "state": "server-generated-state-token",
  "redirect_uri": "https://app.helixterminator.io/auth/callback"
}
```

Success Response — 200 OK:

```
{
  "status": "authenticated",
  "user": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "email": "alice@example.com",
    "display_name": "Alice Smith",
    "sso_provider": "github",
    "sso_subject": "12345678"
  },
  "tokens": {
    "access_token": "eyJhbGciOiJFZERTQSIsInR5cCI6IkpXVCJ9...",
    "token_type": "Bearer",
    "expires_in": 900,
    "refresh_token": "rt_v1_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
  },
  "is_new_user": false
}
```

POST /api/v1/auth/api-keys

Create a new API key.

Authentication: Bearer token required

Request Body:

```
{
  "name": "CI/CD Pipeline Key",
  "scopes": ["hosts:read", "sessions:write", "snippets:execute"],
  "expires_at": "2027-06-28T00:00:00Z",
  "allowed_ips": ["10.0.0.0/8", "192.168.1.0/24"],
  "description": "Used by GitHub Actions for automated
       deployments"
}
```

Field	Type	Required	Description
`name`	string	Yes	Human-readable name (max 100 chars)
`scopes`	array	Yes	Permission scopes
`expires_at`	datetime	No	Expiration; omit for non-expiring

Field	Type	Required	Description
`allowed_ips`	array	No	IP allowlist (CIDR notation)
`description`	string	No	Usage description

Success Response — 201 Created:

```
{
  "id": "key-550e8400-0000-0000-0000-aabbccddeeff",
  "name": "CI/CD Pipeline Key",
  "key": "htk_v1_AbCdEfGhIjKlMnOpQrStUvWxYz0123456789ABCDE",
  "prefix": "htk_v1_AbCd",
  "scopes": ["hosts:read", "sessions:write", "snippets:execute"],
  "expires_at": "2027-06-28T00:00:00Z",
  "allowed_ips": ["10.0.0.0/8", "192.168.1.0/24"],
  "created_at": "2026-06-28T17:40:00Z",
  "last_used_at": null
}
```

IMPORTANT: The full `key` value is only returned once at creation. It cannot be retrieved again. Store it securely immediately.

GET /api/v1/auth/api-keys

List all API keys for the authenticated user.

Authentication: Bearer token required

Success Response — 200 OK:

```
{
  "data": [
    {
      "id": "key-550e8400-0000-0000-0000-aabbccddeeff",
      "name": "CI/CD Pipeline Key",
      "prefix": "htk_v1_AbCd",
      "scopes": ["hosts:read", "sessions:write",
        "snippets:execute"],
      "expires_at": "2027-06-28T00:00:00Z",
      "allowed_ips": ["10.0.0.0/8", "192.168.1.0/24"],
      "created_at": "2026-06-28T17:40:00Z",
      "last_used_at": "2026-06-28T12:00:00Z",
      "last_used_ip": "10.0.0.5",
      "is_expired": false
    }
```

```
  ],
  "pagination": {
    "total_count": 1,
    "has_next": false
  }
}
```

DELETE /api/v1/auth/api-keys/{keyId}

Revoke an API key. All requests using this key will immediately return 401.

Authentication: Bearer token required

Path Parameters: | Parameter | Type | Description | |---|---|---| | `keyId` | UUID | API key identifier |

Success Response — 204 No Content

GET /api/v1/auth/sessions

List all active sessions for the authenticated user.

Authentication: Bearer token required

Success Response — 200 OK:

```
{
  "data": [
    {
      "id": "sess-550e8400-0000-0000-0000-aabbccddeeff",
      "device": {
        "id": "dev-550e8400-0000-0000-0000-112233445566",
        "name": "Alice's MacBook Pro",
        "platform": "macOS 14.2"
      },
      "ip_address": "192.168.1.10",
      "user_agent": "HelixTerminator/1.0 (macOS 14.2)",
      "created_at": "2026-06-28T09:00:00Z",
      "last_active_at": "2026-06-28T17:40:00Z",
      "expires_at": "2026-07-28T09:00:00Z",
      "is_current": true,
      "mfa_verified": true,
      "location": {
```

```
        "city": "San Francisco",
        "country": "US",
        "approximate": true
      }
    }
  ],
  "pagination": {
    "total_count": 1,
    "has_next": false
  }
}
```

DELETE /api/v1/auth/sessions/{sessionId}

Terminate a specific session. The associated refresh token is revoked.

Authentication: Bearer token required

Path Parameters: | Parameter | Type | Description | |—|—|—| | `sessionId` | UUID | Session identifier |

Success Response — 204 No Content

---

3. User Service REST API

Base path: `/api/v1/users`
Service: `user-service` (port 8082 internal)
Database: `user_db` (PostgreSQL)

GET /api/v1/users/me

Get the authenticated user's profile.

Authentication: Bearer token required

Success Response — 200 OK:

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "email": "alice@example.com",
  "email_verified": true,
  "display_name": "Alice Smith",
```

```
  "avatar_url": "https://cdn.helixterminator.io/avatars/
       550e8400.jpg",
  "status": "active",
  "bio": "Infrastructure engineer at ExampleCorp",
  "locale": "en-US",
  "timezone": "America/New_York",
  "created_at": "2026-01-15T09:00:00Z",
  "updated_at": "2026-06-28T10:00:00Z",
  "mfa": {
    "totp_enabled": true,
    "fido2_count": 2,
    "backup_codes_remaining": 6
  },
  "organization": {
    "id": "org-550e8400-0000-0000-0000-000000000001",
    "name": "ExampleCorp",
    "slug": "examplecorp",
    "role": "org_admin"
  },
  "_links": {
    "self": { "href": "https://api.helixterminator.io/api/v1/
        users/me" },
    "preferences": { "href": "https://api.helixterminator.io/api/
        v1/users/me/preferences" },
    "data_export": { "href": "https://api.helixterminator.io/api/
        v1/users/me/data-export" }
  }
}
```

PUT /api/v1/users/me

Update the authenticated user's profile.

Authentication: Bearer token required

Request Body:

```
{
  "display_name": "Alice J. Smith",
  "bio": "Senior Infrastructure Engineer at ExampleCorp",
  "locale": "en-GB",
  "timezone": "Europe/London"
}
```

All fields are optional; unincluded fields are not modified.

Success Response — 200 OK: Returns the updated user object (same schema as `GET /api/v1/users/me`).

PUT /api/v1/users/me/password

Change the authenticated user's password.

Authentication: Bearer token required
Rate Limit: 5 requests per user per 15 minutes

Request Body:

```
{
  "current_password": "OldSecureP@ssw0rd!",
  "new_password": "NewSecureP@ssw0rd!2026",
  "revoke_other_sessions": true
}
```

Success Response — 200 OK:

```
{
  "message": "Password changed successfully.",
  "sessions_revoked": 3
}
```

Error Responses:

Status	Condition
401	Current password incorrect
422	New password does not meet complexity requirements
422	New password matches a previously used password

PUT /api/v1/users/me/email

Initiate an email address change. Sends verification to both old and new addresses.

Authentication: Bearer token required

Request Body:

```
{
  "new_email": "alice.smith@newdomain.com",
```

```
  "password": "SecureP@ssw0rd!"
}
```

Success Response — 200 OK:

```
{
  "message": "Verification emails sent to both addresses. Change
       will take effect once both are confirmed.",
  "pending_email": "alice.smith@newdomain.com",
  "expires_at": "2026-06-29T17:40:00Z"
}
```

GET /api/v1/users/me/preferences

Get the authenticated user's preferences.

Authentication: Bearer token required

Success Response — 200 OK:

```
{
  "theme": "dark",
  "font_size": 14,
  "font_family": "JetBrains Mono",
  "terminal_color_scheme": "dracula",
  "cursor_style": "block",
  "cursor_blink": true,
  "scrollback_lines": 10000,
  "bell_sound": false,
  "notifications": {
    "email_on_new_login": true,
    "email_on_new_device": true,
    "push_session_started": false,
    "push_session_ended": false
  },
  "keyboard_shortcuts": {
    "new_tab": "Ctrl+T",
    "close_tab": "Ctrl+W",
    "split_horizontal": "Ctrl+Shift+H",
    "split_vertical": "Ctrl+Shift+V"
  },
  "default_vault_id": "vault-550e8400-0000-0000-0000-
       aabbccddeeff",
  "startup_workspace_id": null,
  "sidebar_collapsed": false,
```

```
  "updated_at": "2026-06-28T10:00:00Z"
}
```

PUT /api/v1/users/me/preferences

Update user preferences (partial update supported).

Authentication: Bearer token required

Request Body:

```
{
  "theme": "light",
  "font_size": 16,
  "notifications": {
    "email_on_new_login": false
  }
}
```

Success Response — 200 OK: Returns the full updated preferences object.

POST /api/v1/users/me/avatar

Upload a new profile avatar.

Authentication: Bearer token required
Content-Type: `multipart/form-data`
Limits: Max 5 MB; JPEG, PNG, GIF, WebP

Request Body (multipart):

```
Content-Disposition: form-data; name="avatar";
filename="photo.jpg"
Content-Type: image/jpeg

<binary data>
```

Success Response — 200 OK:

```
{
  "avatar_url": "https://cdn.helixterminator.io/avatars/550e8400-
        e29b-41d4-a716-446655440000.jpg",
```

```
  "updated_at": "2026-06-28T17:45:00Z"
}
```

DELETE /api/v1/users/me/avatar

Remove the profile avatar and revert to default.

Authentication: Bearer token required

Success Response — 204 No Content

DELETE /api/v1/users/me

Request account deletion (GDPR Article 17 — Right to Erasure).

Authentication: Bearer token required
Rate Limit: 1 request per user per 24 hours

Request Body:

```
{
  "password": "SecureP@ssw0rd!",
  "reason": "No longer using the service",
  "confirm_deletion": true
}
```

`confirm_deletion` must be `true`.

Success Response — 202 Accepted:

```
{
  "message": "Account deletion scheduled. Your data will be
       permanently deleted within 30 days.",
  "deletion_scheduled_at": "2026-06-28T17:40:00Z",
  "permanent_deletion_at": "2026-07-28T17:40:00Z",
  "cancellation_url": "https://app.helixterminator.io/account/
       cancel-deletion?token=xxxx"
}
```

The account enters `pending_deletion` state. A cancellation link is valid for 14 days.

GET /api/v1/users/me/data-export

Request a GDPR data portability export (Article 20). Triggers an async job.

Authentication: Bearer token required
Rate Limit: 1 request per user per 7 days

Query Parameters: | Parameter | Type | Description | |---|---|---| | `format` | string | `json` (default) or `csv` |

Success Response — 202 Accepted:

```
{
  "export_id": "exp-550e8400-0000-0000-0000-aabbccddeeff",
  "status": "pending",
  "format": "json",
  "requested_at": "2026-06-28T17:40:00Z",
  "estimated_completion_at": "2026-06-28T18:10:00Z",
  "download_expires_at": null
}
```

When ready, the user receives an email with a signed download URL. The download link is valid for 48 hours.

GET /api/v1/users/me/data-export/{exportId}/status checks job status:

```
{
  "export_id": "exp-550e8400-0000-0000-0000-aabbccddeeff",
  "status": "completed",
  "download_url": "https://cdn.helixterminator.io/exports/exp-
       xxx.json.zip?token=signed-url",
  "download_expires_at": "2026-06-30T18:10:00Z",
  "file_size_bytes": 2457600
}
```

---

4. Vault Service REST API

Base path: `/api/v1/vaults`
Service: `vault-service` (port 8083 internal)
Database:

`vault_db` (PostgreSQL)	A **Vault** is an end-to-end encrypted container for secrets, credentials, host configurations, and SSH keys. All vault item data is encrypted client-side before transmission using AES-256-GCM with keys derived from the user's master password via Argon2id. The server never sees plaintext vault content.

GET /api/v1/vaults

List all vaults accessible to the authenticated user (owned + shared).

Authentication: Bearer token required

Query Parameters: | Parameter | Type | Default | Description | |—|—|—|—| | `cursor` | string | — | Pagination cursor | | `limit` | integer | 25 | Page size | | `owned_only` | boolean | false | Only show vaults the user owns | | `sort` | string | `name:asc` | Sort field |

Success Response — 200 OK:

```
{
  "data": [
    {
```

```
      "id": "vault-550e8400-0000-0000-0000-aabbccddeeff",
      "name": "Production Credentials",
      "description": "All production server credentials",
      "color": "#FF6B35",
      "icon": "server",
      "owner_id": "550e8400-e29b-41d4-a716-446655440000",
      "member_count": 5,
      "item_count": 247,
      "my_permission": "admin",
      "encrypted": true,
      "sync_enabled": true,
      "last_synced_at": "2026-06-28T17:00:00Z",
      "created_at": "2026-01-15T09:00:00Z",
      "updated_at": "2026-06-28T17:00:00Z",
      "_links": {
        "self": { "href": "https://api.helixterminator.io/api/v1/
        vaults/vault-550e8400-0000-0000-0000-aabbccddeeff" },
        "members": { "href": "https://api.helixterminator.io/api/
        v1/vaults/vault-550e8400-0000-0000-0000-aabbccddeeff/
        members" }
      }
    }
  ],
  "pagination": {
    "cursor_next": null,
    "has_next": false,
    "limit": 25,
    "total_count": 1
  }
}
```

POST /api/v1/vaults

Create a new vault.

Authentication: Bearer token required

Request Body:

```
{
  "name": "Staging Credentials",
  "description": "Staging environment hosts and keys",
  "color": "#4ECDC4",
  "icon": "database",
  "sync_enabled": true,
```

```
  "encrypted_key_blob": "base64-encoded-encrypted-vault-key",
  "kdf_params": {
    "algorithm": "argon2id",
    "memory_kib": 65536,
    "iterations": 3,
    "parallelism": 4,
    "salt": "base64-encoded-salt"
  }
}
```

Success Response — 201 Created: Full vault object.

GET /api/v1/vaults/{vaultId}

Get a specific vault.

Authentication: Bearer token required

Path Parameters: | Parameter | Type | Description | |—|—|—| | `vaultId` | UUID | Vault identifier |

Success Response — 200 OK: Full vault object.

Error Responses:

Status	Condition
403	User is not a member of this vault
404	Vault not found

PUT /api/v1/vaults/{vaultId}

Update vault metadata (name, description, color, icon, sync settings).

Authentication: Bearer token required; requires `admin` vault permission

Request Body:

```
{
  "name": "Staging Credentials v2",
  "color": "#FFE66D",
  "sync_enabled": false
}
```

Success Response — 200 OK: Updated vault object.

DELETE /api/v1/vaults/{vaultId}

Delete a vault permanently. All items are erased. Requires vault ownership.

Authentication: Bearer token required; must be vault owner

Request Body:

```
{
  "confirm_name": "Staging Credentials v2"
}
```

The `confirm_name` must match the vault name exactly.

Success Response — 204 No Content

POST /api/v1/vaults/{vaultId}/sync

Trigger a vault sync operation. Returns the server-side sync state for delta sync.

Authentication: Bearer token required; requires vault member access

Request Body:

```
{
  "client_cursor": "sync-cursor-opaque-value",
  "changes": [
    {
      "item_id": "item-550e8400-0000-0000-0000-aabbccddeeff",
      "operation": "upsert",
      "encrypted_data": "base64-encoded-encrypted-payload",
      "version": 5,
      "checksum": "sha256:aabbccdd..."
    }
  ]
}
```

Success Response — 200 OK:

```
{
  "server_cursor": "new-sync-cursor-opaque",
```

```
  "applied_changes": 3,
  "conflicts": [
    {
      "item_id": "item-550e8400-0000-0000-0000-aabbccddeeff",
      "conflict_type": "version_mismatch",
      "server_version": 7,
      "server_encrypted_data": "base64-encoded...",
      "resolution": "server_wins"
    }
  ],
  "pending_changes": [
    {
      "item_id": "item-aabbccdd-0000-0000-0000-112233445566",
      "operation": "upsert",
      "encrypted_data": "base64-encoded-encrypted-payload",
      "version": 2
    }
  ]
}
```

GET /api/v1/vaults/{vaultId}/members

List vault members.

Authentication: Bearer token required; requires vault member access

Success Response — 200 OK:

```
{
  "data": [
    {
      "user_id": "550e8400-e29b-41d4-a716-446655440000",
      "email": "alice@example.com",
      "display_name": "Alice Smith",
      "avatar_url": "https://cdn.helixterminator.io/avatars/
        alice.jpg",
      "permission": "admin",
      "invited_by": null,
      "joined_at": "2026-01-15T09:00:00Z",
      "is_owner": true
    },
    {
      "user_id": "660e8400-e29b-41d4-a716-556655440000",
      "email": "bob@example.com",
```

```
      "display_name": "Bob Jones",
      "avatar_url": null,
      "permission": "write",
      "invited_by": "550e8400-e29b-41d4-a716-446655440000",
      "joined_at": "2026-02-01T10:00:00Z",
      "is_owner": false
    }
  ],
  "pagination": {
    "total_count": 2,
    "has_next": false
  }
}
```

POST /api/v1/vaults/{vaultId}/members

Add a member to a vault.

Authentication: Bearer token required; requires vault `admin` permission

Request Body:

```
{
  "user_id": "770e8400-e29b-41d4-a716-666655440000",
  "permission": "read",
  "encrypted_vault_key": "base64-encoded-vault-key-encrypted-for-
        new-member"
}
```

The `encrypted_vault_key` is the vault's symmetric key re-encrypted with the new member's public key. The server stores it as a blob and never decrypts it.

Success Response — 201 Created: New member object.

DELETE /api/v1/vaults/{vaultId}/members/{userId}

Remove a member from a vault.

Authentication: Bearer token required; requires vault `admin` permission (or self-removal)

Success Response — 204 No Content

PUT /api/v1/vaults/{vaultId}/members/{userId}/permissions

Update a vault member's permission level.

Authentication: Bearer token required; requires vault `admin` permission

Request Body:

```
{
  "permission": "write"
}
```

Valid permissions: `read`, `write`, `admin`.

Success Response — 200 OK: Updated member object.

GET /api/v1/vaults/{vaultId}/items

List item metadata within a vault (individual-secret surface — distinct from the opaque bulk `/sync` endpoint above, which is the client's delta-sync transport). Item **content** (`encrypted_data`, `iv`) is always end-to-end encrypted; the server never decrypts it and this endpoint never returns plaintext, matching the zero-knowledge posture (client-side AES-256-GCM, Argon2id-derived keys, §4 intro).

Authentication: Bearer token required; requires vault `read` permission

Query Parameters: | Parameter | Type | Default | Description | |—|—|—|—| | `item_type` | string | — | Filter by `vault_items.item_type` | | `cursor` / `limit` | string / integer | — / 25 | Pagination |

Success Response — 200 OK:

```
{
  "data": [
    {
      "id": "item-550e8400-0000-0000-0000-aabbccddeeff",
      "vault_id": "vault-550e8400-0000-0000-0000-aabbccddeeff",
      "item_type": "password",
      "version": 5,
      "checksum": "sha256:aabbccdd...",
      "created_by": "550e8400-e29b-41d4-a716-446655440000",
```

```
      "updated_by": "660e8400-e29b-41d4-a716-556655440000",
      "created_at": "2026-01-15T09:00:00Z",
      "updated_at": "2026-06-20T10:00:00Z"
    }
  ],
  "pagination": { "total_count": 247, "has_next": true }
}
```

`encrypted_data` and `iv` are intentionally omitted from the list response (bandwidth; clients fetch full item content via the endpoint below only for the item currently being opened).

GET /api/v1/vaults/{vaultId}/items/{itemId}

Fetch a single item's full encrypted payload — the item-level counterpart to the bulk `/sync` transport.

Authentication: Bearer token required; requires vault `read` permission

Success Response — 200 OK:

```
{
  "id": "item-550e8400-0000-0000-0000-aabbccddeeff",
  "vault_id": "vault-550e8400-0000-0000-0000-aabbccddeeff",
  "item_type": "password",
  "encrypted_data": "base64-encoded-encrypted-payload",
  "iv": "base64-encoded-iv",
  "checksum": "sha256:aabbccdd...",
  "version": 5,
  "created_at": "2026-01-15T09:00:00Z",
  "updated_at": "2026-06-20T10:00:00Z"
}
```

Error Responses:

Status	Condition
403	User is not a member of this vault, or lacks `read` permission
404	Item not found or soft-deleted

DELETE /api/v1/vaults/{vaultId}/items/{itemId}

Soft-delete a single vault item (sets `vault_items.is_deleted = TRUE`, `deleted_at = NOW()`) without requiring a full `/sync` round-trip.

Authentication: Bearer token required; requires vault `write` permission

Success Response — 204 No Content

GET /api/v1/vaults/{vaultId}/items/{itemId}/versions

List an item's version history (`vault_item_versions`) — used for conflict resolution and "restore a previous version" UI.

Authentication: Bearer token required; requires vault `read` permission

Success Response — 200 OK:

```
{
  "data": [
    {
      "version": 5,
      "encrypted_data": "base64-encoded-encrypted-payload",
      "iv": "base64-encoded-iv",
      "checksum": "sha256:aabbccdd...",
      "changed_by": "550e8400-e29b-41d4-a716-446655440000",
      "created_at": "2026-06-20T10:00:00Z"
    }
  ],
  "pagination": { "total_count": 5, "has_next": false }
}
```

GET /api/v1/vaults/{vaultId}/items/{itemId}/audit

Per-item audit trail — reads `vault_audit_events` filtered by `item_id` (that table already carries an `item_id` column, §17.2; this endpoint is the REST surface the schema was missing). Distinct from the org-wide `GET /api/v1/audit/events` (§13) which cannot be filtered to a single vault item efficiently at that layer.

Authentication: Bearer token required; requires vault `admin` permission

Success Response — 200 OK:

```
{
  "data": [
    {
      "event_type": "item.viewed",
      "user_id": "660e8400-e29b-41d4-a716-556655440000",
      "ip_address": "203.0.113.42",
      "occurred_at": "2026-06-28T17:40:00Z"
    },
    {
      "event_type": "item.updated",
      "user_id": "550e8400-e29b-41d4-a716-446655440000",
      "ip_address": "203.0.113.10",
      "occurred_at": "2026-06-20T10:00:00Z"
    }
  ],
  "pagination": { "total_count": 12, "has_next": false }
}
```

POST /api/v1/vaults/{vaultId}/rewrap

Key rotation / re-wrap after member removal. Rotating a vault's symmetric key, or re-keying it after a member is removed (so the removed member's last-known copy of the key can no longer decrypt items synced after their removal), MUST be **client-driven end-to-end** — per the zero-knowledge posture (§4 intro), the server never generates, sees, or re-wraps the plaintext vault key itself. This endpoint only accepts and atomically installs a client-prepared replacement wrap set; it is the same trust model as `POST /api/v1/vaults/{vaultId}/members` (`encrypted_vault_key` is opaque to the server there too).

Client-side flow (client, not server, performs steps 1–3): 1. Generate a new vault symmetric key locally. 2. Re-encrypt every vault item with the new key (or lazily re-encrypt on next write — implementation choice left to the client; the server does not require all items re-encrypted atomically with the key rotation). 3. For every **remaining** member (fetched via `GET /api/v1/vaults/{vaultId}/members`), wrap the new key with that member's public key, producing one `encrypted_vault_key` blob per member.

Authentication: Bearer token required; requires vault `admin` permission (or must be vault owner for a post-removal rotation)

Request Body:

```
{
  "reason": "member_removed",
  "removed_user_id": "770e8400-e29b-41d4-a716-666655440000",
  "rewrapped_keys": [
    {
      "user_id": "550e8400-e29b-41d4-a716-446655440000",
      "encrypted_vault_key": "base64-encoded-vault-key-encrypted-
        for-this-member"
    },
    {
      "user_id": "660e8400-e29b-41d4-a716-556655440000",
      "encrypted_vault_key": "base64-encoded-vault-key-encrypted-
        for-this-member"
    }
  ]
}
```

`reason`: `member_removed`, `scheduled_rotation`, or `suspected_compromise`.

Server-side guarantee: `rewrapped_keys` must include an entry for **every** current vault member with `permission != 'read'`-excluded logic aside — i.e. every row currently in `vault_members` for this vault — or the request is rejected with `422` (incomplete rewrap set; a partial rewrap would silently strand some members' access). The update is applied in a single transaction: every `vault_members.encrypted_vault_key` is replaced, `vaults.version` is incremented, and a `vault.key_rotated` event is written to `vault_audit_events` (fail-closed per §17.10.1's pattern, applied here to the vault-level privileged-op audit write).

Success Response — 200 OK:

```
{
  "vault_id": "vault-550e8400-0000-0000-0000-aabbccddeeff",
  "version": 8,
  "rewrapped_member_count": 4,
  "rotated_at": "2026-06-28T17:45:00Z"
}
```

Error Responses:

Status	Condition
403	Not vault owner/admin
422	

Status	Condition
	`rewrapped_keys` missing an entry for a current member (incomplete rewrap)

Known limitation (documented, not silently glossed over): re-wrapping the vault key does not retroactively invalidate a removed member's **already-downloaded** local copy of vault items encrypted before their removal — client-side E2E encryption means the server has no mechanism to remotely wipe data already decrypted and cached on a former member's device. This is the same disclosed limitation present in comparable zero-knowledge password managers; mitigation is procedural (rotate promptly on removal, treat any item a departing member had access to as potentially exposed, per org offboarding policy).

---

5. Host Service REST API

Base path: `/api/v1/hosts`, `/api/v1/groups`
Service: `host-service` (port 8084 internal)
Database: `host_db` (PostgreSQL)

GET /api/v1/hosts

List hosts with filtering, sorting, and pagination.

Authentication: Bearer token required

Query Parameters: | Parameter | Type | Default | Description | |—| —|—|—| | `cursor` | string | — | Pagination cursor | | `limit` | integer | 25 | Page size (max 100) | | `sort` | string | `name:asc` | Sort field:direction | | `q` | string | — | Full-text search | | `vault_id` | UUID | — | Filter by vault | | `group_id` | UUID | — | Filter by group | | `status` | string | — | `active`, `inactive`, `unreachable` | | `os` | string | — | OS filter (e.g., `linux`, `macos`, `windows`) | | `tag` | string | — | Filter by tag (repeatable) | | `jump_enabled` | boolean | — | Filter by jump host capability | | `last_connected_gte` | datetime | — | Last connection after date | | `last_connected_lte` | datetime | — | Last connection before date |

Success Response — 200 OK:

```
{
  "data": [
    {
      "id": "host-550e8400-0000-0000-0000-aabbccddeeff",
      "name": "prod-web-01",
      "hostname": "10.0.1.10",
      "port": 22,
      "username": "deploy",
      "auth_method": "key",
      "key_id": "key-550e8400-0000-0000-0000-112233445566",
      "vault_id": "vault-550e8400-0000-0000-0000-aabbccddeeff",
      "group_id": "group-550e8400-0000-0000-0000-223344556677",
      "tags": ["production", "web", "nginx"],
      "os": "linux",
      "os_version": "Ubuntu 24.04 LTS",
      "arch": "amd64",
      "description": "Primary web server",
      "jump_host_id": null,
      "jump_enabled": false,
      "color": "#FF6B35",
      "icon": "server",
      "status": "active",
      "last_connected_at": "2026-06-28T16:00:00Z",
      "fingerprint_verified": true,
      "created_at": "2026-01-15T09:00:00Z",
      "updated_at": "2026-06-28T10:00:00Z",
      "_links": {
        "self": { "href": "https://api.helixterminator.io/api/v1/
        hosts/host-550e8400-0000-0000-0000-aabbccddeeff" },
        "connections": { "href": "https://api.helixterminator.io/
        api/v1/hosts/host-550e8400-0000-0000-0000-aabbccddeeff/
        connections" }
      }
    }
  ],
  "pagination": {
    "cursor_next": "eyJpZCI6Imhvc3Qtc...",
    "has_next": true,
    "limit": 25,
    "total_count": 347
  }
}
```

POST /api/v1/hosts

Create a new host.

Authentication: Bearer token required

Request Body:

```
{
  "name": "prod-db-01",
  "hostname": "10.0.2.10",
  "port": 22,
  "username": "ubuntu",
  "auth_method": "key",
  "key_id": "key-550e8400-0000-0000-0000-112233445566",
  "vault_id": "vault-550e8400-0000-0000-0000-aabbccddeeff",
  "group_id": "group-550e8400-0000-0000-0000-223344556677",
  "tags": ["production", "database", "postgresql"],
  "description": "Primary PostgreSQL server",
  "jump_host_id": "host-550e8400-0000-0000-0000-334455667788",
  "color": "#4ECDC4",
  "icon": "database",
  "proxy_jump_command": null,
  "environment_variables": {
    "PGPASSWORD": "${secrets.PGPASSWORD}"
  },
  "startup_snippet_id": null,
  "connection_timeout_seconds": 30,
  "keepalive_interval_seconds": 60
}
```

Success Response — 201 Created: Full host object.

GET /api/v1/hosts/{hostId}

Get a specific host.

Authentication: Bearer token required

Path Parameters: | Parameter | Type | Description | |—|—|—| | `hostId` | UUID | Host identifier |

Success Response — 200 OK: Full host object with additional computed fields:

```
{
  "id": "host-550e8400-0000-0000-0000-aabbccddeeff",
  "name": "prod-web-01",
  "hostname": "10.0.1.10",
  "port": 22,
  "username": "deploy",
  "auth_method": "key",
  "key_id": "key-550e8400-0000-0000-0000-112233445566",
  "known_fingerprints": [
    {
      "algorithm": "SHA256",
      "fingerprint": "SHA256:abc123def456...",
      "added_at": "2026-01-15T09:05:00Z",
      "verified_by": "alice@example.com"
    }
  ],
  "jump_chain": [],
  "connection_stats": {
    "total_connections": 847,
    "total_duration_seconds": 284700,
    "last_connected_at": "2026-06-28T16:00:00Z",
    "average_session_seconds": 336
  }
}
```

PUT /api/v1/hosts/{hostId}

Replace a host configuration entirely.

Authentication: Bearer token required

Request Body: Same schema as POST.

Success Response — 200 OK: Updated host object.

DELETE /api/v1/hosts/{hostId}

Delete a host and all its connection history.

Authentication: Bearer token required

Success Response — 204 No Content

POST /api/v1/hosts/bulk

Create multiple hosts at once (batch operation).

Authentication: Bearer token required

Request Body:

```
{
  "hosts": [
    {
      "name": "prod-app-01",
      "hostname": "10.0.3.10",
      "port": 22,
      "username": "ubuntu",
      "auth_method": "key",
      "key_id": "key-550e8400-0000-0000-0000-112233445566",
      "vault_id": "vault-550e8400-0000-0000-0000-aabbccddeeff"
    },
    {
      "name": "prod-app-02",
      "hostname": "10.0.3.11",
      "port": 22,
      "username": "ubuntu",
      "auth_method": "key",
      "key_id": "key-550e8400-0000-0000-0000-112233445566",
      "vault_id": "vault-550e8400-0000-0000-0000-aabbccddeeff"
    }
  ],
  "group_id": "group-550e8400-0000-0000-0000-223344556677",
  "on_conflict": "skip"
}
```

`on_conflict`: `skip` (skip duplicates), `update` (update existing), `error` (fail on duplicate).

Success Response — 207 Multi-Status:

```
{
  "results": [
    { "index": 0, "status": 201, "id": "host-111", "name": "prod-
        app-01" },
    { "index": 1, "status": 409, "error": "hostname '10.0.3.11'
        already exists in vault" }
  ],
  "created": 1,
```

```
  "skipped": 0,
  "errors": 1
}
```

DELETE /api/v1/hosts/bulk

Delete multiple hosts by IDs.

Authentication: Bearer token required

Request Body:

```
{
  "host_ids": [
    "host-550e8400-0000-0000-0000-aabbccddeeff",
    "host-660e8400-0000-0000-0000-bbccddeeaabb"
  ]
}
```

Success Response — 200 OK:

```
{
  "deleted": 2,
  "not_found": 0,
  "forbidden": 0
}
```

GET /api/v1/hosts/{hostId}/connections

Get connection history for a specific host.

Authentication: Bearer token required

Query Parameters: | Parameter | Type | Description | |—|—|—| | `cursor` | string | Pagination cursor | | `limit` | integer | Page size | | `user_id` | UUID | Filter by user | | `started_at_gte` | datetime | Filter by start time |

Success Response — 200 OK:

```
{
  "data": [
    {
      "id": "conn-550e8400-0000-0000-0000-aabbccddeeff",
```

```
      "host_id": "host-550e8400-0000-0000-0000-aabbccddeeff",
      "user_id": "550e8400-e29b-41d4-a716-446655440000",
      "user_email": "alice@example.com",
      "session_id": "sess-550e8400-0000-0000-0000-aabbccddeeff",
      "client_ip": "192.168.1.10",
      "started_at": "2026-06-28T16:00:00Z",
      "ended_at": "2026-06-28T16:15:30Z",
      "duration_seconds": 930,
      "bytes_sent": 48392,
      "bytes_received": 1204847,
      "exit_code": 0,
      "recording_available": true
    }
  ],
  "pagination": {
    "cursor_next": null,
    "has_next": false,
    "total_count": 847
  }
}
```

POST /api/v1/hosts/import

Import hosts from CSV, SSH config file, or Ansible inventory.

Authentication: Bearer token required
Content-Type: `multipart/form-data`

Request Body (multipart):

```
Content-Disposition: form-data; name="file"; filename="hosts.csv"
Content-Type: text/csv

name,hostname,port,username,auth_method
prod-web-01,10.0.1.10,22,ubuntu,key
prod-web-02,10.0.1.11,22,ubuntu,key
```

Query Parameters: | Parameter | Type | Description | |---|---|---| | `format` | string | `csv`, `ssh_config`, `ansible` | | `vault_id` | UUID | Target vault | | `group_id` | UUID | Target group | | `dry_run` | boolean | Validate without importing |

Success Response — 202 Accepted:

```
{
  "import_id": "imp-550e8400-0000-0000-0000-aabbccddeeff",
  "status": "processing",
  "total_rows": 150,
  "dry_run": false
}
```

GET /api/v1/hosts/export

Export hosts to SSH config format.

Authentication: Bearer token required

Query Parameters: | Parameter | Type | Description | |---|---|---| | `vault_id` | UUID | Export from specific vault | | `group_id` | UUID | Export from specific group | | `format` | string | `ssh_config` (default), `csv`, `json` |

Success Response — 200 OK:

```
Content-Type: text/plain; charset=utf-8
Content-Disposition: attachment; filename="helixterm-hosts.conf"

Host prod-web-01
    HostName 10.0.1.10
    Port 22
    User ubuntu
    IdentityFile ~/.ssh/helixterm_key

Host prod-db-01
    HostName 10.0.2.10
    Port 22
    User ubuntu
    ProxyJump prod-web-01
    IdentityFile ~/.ssh/helixterm_key
```

GET /api/v1/groups

List host groups.

Authentication: Bearer token required

Query Parameters: | Parameter | Type | Description | |---|---|---| | `vault_id` | UUID | Filter by vault | | `parent_id` | UUID | Filter by parent group | | `q` | string | Search query |

Success Response — 200 OK:

```
{
  "data": [
    {
      "id": "group-550e8400-0000-0000-0000-223344556677",
      "name": "Production",
      "description": "All production hosts",
      "parent_id": null,
      "vault_id": "vault-550e8400-0000-0000-0000-aabbccddeeff",
      "color": "#FF6B35",
      "icon": "folder",
      "host_count": 47,
      "child_group_count": 3,
      "inherit_settings": true,
      "default_key_id":
        "key-550e8400-0000-0000-0000-112233445566",
      "default_username": "ubuntu",
      "created_at": "2026-01-15T09:00:00Z"
    }
  ],
  "pagination": {
    "total_count": 12,
    "has_next": false
  }
}
```

POST /api/v1/groups

Create a host group.

Authentication: Bearer token required

Request Body:

```
{
  "name": "Staging",
  "description": "Staging environment hosts",
  "parent_id": null,
  "vault_id": "vault-550e8400-0000-0000-0000-aabbccddeeff",
  "color": "#4ECDC4",
```

```
  "icon": "folder-open",
  "inherit_settings": true,
  "default_key_id": "key-550e8400-0000-0000-0000-112233445566",
  "default_username": "ubuntu",
  "default_port": 22
}
```

Success Response — 201 Created: Full group object.

GET /api/v1/groups/{groupId}

Get a specific group with its full configuration.

Authentication: Bearer token required

Success Response — 200 OK: Full group object.

PUT /api/v1/groups/{groupId}

Update a group.

Authentication: Bearer token required

Request Body: Same as POST, all fields optional.

Success Response — 200 OK: Updated group object.

DELETE /api/v1/groups/{groupId}

Delete a group. Hosts within the group are not deleted but become ungrouped.

Authentication: Bearer token required

Query Parameters: | Parameter | Type | Description | |—|—|—| | `migrate_hosts_to` | UUID | Move hosts to this group instead of ungrouping |

Success Response — 204 No Content

POST /api/v1/groups/{groupId}/hosts

Add a host to a group.

Authentication: Bearer token required

Request Body:

```
{
  "host_id": "host-550e8400-0000-0000-0000-aabbccddeeff"
}
```

Success Response — 204 No Content

DELETE /api/v1/groups/{groupId}/hosts/{hostId}

Remove a host from a group.

Authentication: Bearer token required

Success Response — 204 No Content

PUT /api/v1/groups/{groupId}/inherit

Configure inheritance settings for a group. Child groups and hosts inherit credentials, jump chains, and connection settings.

Authentication: Bearer token required

Request Body:

```
{
  "inherit_from_parent": true,
  "inherit_key": true,
  "inherit_username": true,
  "inherit_port": false,
  "inherit_jump_host": true,
  "inherit_environment_variables": true,
  "inherit_startup_snippet": false
}
```

Success Response — 200 OK: Updated inheritance settings.

---

6. SSH Proxy & Terminal Service REST API

Base path: `/api/v1/sessions`
Service: `session-service` (port 8085 internal), `ssh-proxy-service` (port 8086)
Database: `session_db` (PostgreSQL)

POST /api/v1/sessions/ssh

Initiate an SSH session. Returns a session token and WebSocket URL for terminal I/O.

Authentication: Bearer token required

Request Body:

```
{
  "host_id": "host-550e8400-0000-0000-0000-aabbccddeeff",
  "terminal": {
    "cols": 220,
    "rows": 50,
    "term": "xterm-256color"
  },
  "recording_enabled": true,
  "collab_enabled": false,
  "read_only": false,
  "startup_snippet_id": null,
  "reason": "Investigating high CPU alert",
  "ticket_ref": "INC-20260628-001"
}
```

Field	Type	Required	Description
`host_id`	UUID	Yes	Target host
`terminal.cols`	integer	Yes	Terminal width in columns
`terminal.rows`	integer	Yes	Terminal height in rows
`terminal.term`	string	No	Terminal type (default `xterm-256color`)
`recording_enabled`	boolean	No	Enable session recording

Field	Type	Required	Description
			(default per org policy)
`collab_enabled`	boolean	No	Enable collaboration channel
`read_only`	boolean	No	Read-only mode (no input sent to host)
`startup_snippet_id`	UUID	No	Run snippet immediately on connect
`reason`	string	No	Reason for access (required by org policy)
`ticket_ref`	string	No	Incident/ticket reference

Success Response — 201 Created:

```
{
  "session_id": "sess-550e8400-0000-0000-0000-aabbccddeeff",
  "session_token": "st_v1_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
  "websocket_url": "wss://proxy.helixterminator.io/api/v1/
        sessions/sess-550e8400/terminal",
  "collab_url": "wss://proxy.helixterminator.io/api/v1/sessions/
        sess-550e8400/collab",
  "status": "connecting",
  "host": {
    "id": "host-550e8400-0000-0000-0000-aabbccddeeff",
    "name": "prod-web-01",
    "hostname": "10.0.1.10",
    "port": 22
  },
  "recording_enabled": true,
  "expires_at": "2026-06-29T17:40:00Z",
  "created_at": "2026-06-28T17:40:00Z"
}
```

Error Responses:

Status	Condition
403	No permission to connect to this host
404	Host not found
409	Max concurrent sessions reached
422	Host unreachable, connection refused

GET /api/v1/sessions

List sessions (active and historical) for the authenticated user.

Authentication: Bearer token required

Query Parameters: | Parameter | Type | Description | |---|---|---| | `status` | string | `active`, `closed`, `error` | | `host_id` | UUID | Filter by host | | `cursor` | string | Pagination cursor | | `limit` | integer | Page size | | `started_at_gte` | datetime | Filter by start time |

Success Response — 200 OK:

```
{
  "data": [
    {
      "id": "sess-550e8400-0000-0000-0000-aabbccddeeff",
      "host_id": "host-550e8400-0000-0000-0000-aabbccddeeff",
      "host_name": "prod-web-01",
      "user_id": "550e8400-e29b-41d4-a716-446655440000",
      "status": "active",
      "recording_enabled": true,
      "collab_enabled": false,
      "started_at": "2026-06-28T17:40:00Z",
      "ended_at": null,
      "duration_seconds": null,
      "reason": "Investigating high CPU alert",
      "ticket_ref": "INC-20260628-001"
    }
  ],
  "pagination": {
    "total_count": 42,
    "has_next": false
  }
}
```

GET /api/v1/sessions/{sessionId}

Get details of a specific session.

Authentication: Bearer token required

Success Response — 200 OK: Full session object including connection metadata.

DELETE /api/v1/sessions/{sessionId}

Terminate an active SSH session.

Authentication: Bearer token required (session owner or org admin)

Success Response — 204 No Content

The SSH connection is terminated. If recording is enabled, the recording is finalized asynchronously.

POST /api/v1/sessions/{sessionId}/resize

Resize the terminal window for an active session.

Authentication: Bearer token required (must be session owner)

Request Body:

```
{
  "cols": 240,
  "rows": 60
}
```

Success Response — 204 No Content

POST /api/v1/sessions/{sessionId}/broadcast

Broadcast input to multiple sessions simultaneously (requires org_admin permission).

Authentication: Bearer token required; requires org `org_admin` role

Authorization + blast-radius gate. Multi-session broadcast is the highest-blast-radius terminal capability in the API (one command, many hosts, `org_admin`-only by design). In addition to the role check:

- Every `target_session_ids` entry MUST resolve to a session whose `org_id` equals the caller's current org (§17.0 RLS enforces this at the database layer even if application code omitted the filter); a cross-tenant session ID in the list is dropped from the batch and reported in `results` with `status: "denied_cross_tenant"`, not silently skipped.
- `target_session_ids.length` is capped by `organizations.settings.broadcast_max_sessions` (default `25`); exceeding it returns `422` before any session receives input (all-or-nothing — a broadcast is never partially dispatched because the request itself was oversized).
- The broadcast writes a fail-closed `session.broadcast_executed` audit event (§17.10.1 pattern, synchronous commit) recording the full `target_session_ids` list and the exact `command` bytes **before** dispatch to any session begins — an incident review can always reconstruct exactly what was about to be sent even if delivery to some sessions subsequently failed or timed out.
- `require_confirmation: true` causes the server to hold the command in a `pending_confirmation` state and return `202 Accepted` with a `confirmation_token`; the actual send only happens once the caller (or a second approver, for two-person-control-configured orgs — see `05_security_zero_trust`'s break-glass/JIT controls) calls back with that token, giving a human a chance to abort an oversized or mistaken broadcast before it reaches any host.

Request Body:

```
{
  "target_session_ids": [
    "sess-aabbccdd-0000-0000-0000-112233445566",
    "sess-bbccddee-0000-0000-0000-223344556677"
  ],
  "command": "sudo systemctl restart nginx\n",
  "require_confirmation": false
}
```

Success Response — 200 OK:

```
{
  "broadcast": 2,
  "failed": 0,
```

```
  "results": [
    { "session_id": "sess-aabbccdd", "status": "sent" },
    { "session_id": "sess-bbccddee", "status": "sent" }
  ]
}
```

Error Responses:

Status	Condition
403	Caller lacks org `org_admin` role
422	`target_session_ids.length` exceeds `organizations.settings.broadcast_max_sessions`

GET /api/v1/sessions/{sessionId}/log

Download the full session log as plain text.

Authentication: Bearer token required (session owner or org admin)

Query Parameters: | Parameter | Type | Description | |---|---|---| | `format` | string | `text` (default), `ansi`, `html` | | `include_timestamps` | boolean | Prefix each line with timestamp |

Success Response — 200 OK:

```
Content-Type: text/plain; charset=utf-8
Content-Disposition: attachment; filename="session-
sess-550e8400-2026-06-28.log"

[2026-06-28 17:40:05] ubuntu@prod-web-01:~$ top
[2026-06-28 17:40:05] top - 17:40:05 up 42 days,  6:13,  1 user,
load average: 0.12, 0.08, 0.05
...
```

GET /api/v1/sessions/{sessionId}/recording

Download the session recording in asciinema v2 format.

Authentication: Bearer token required (session owner or org admin)

Query Parameters: | Parameter | Type | Description | |---|---|---| | `format` | string | `asciicast` (default), `gif`, `mp4` |

Success Response — 200 OK:

```
Content-Type: application/x-asciicast
Content-Disposition: attachment; filename="session-
sess-550e8400-2026-06-28.cast"

{"version": 2, "width": 220, "height": 50, "timestamp":
1751125200, "title": "prod-web-01 session", "env": {"TERM":
"xterm-256color"}}
[0.0, "o", "\u001b[?1049h\u001b[22;0;0t"]
[0.234, "o", "ubuntu@prod-web-01:~$ "]
...
```

---

7. SFTP Service REST API

Base path: `/api/v1/sftp`
Service: `sftp-service` (port 8087 internal)
Database: `session_db` (shared, sftp tables)

POST /api/v1/sftp/sessions

Open an SFTP session to a host.

Authentication: Bearer token required

Request Body:

```
{
  "host_id": "host-550e8400-0000-0000-0000-aabbccddeeff",
  "initial_path": "/var/www",
  "transfer_mode": "binary"
}
```

Success Response — 201 Created:

```
{
  "sftp_session_id": "sftp-550e8400-0000-0000-0000-aabbccddeeff",
  "host_id": "host-550e8400-0000-0000-0000-aabbccddeeff",
  "host_name": "prod-web-01",
  "cwd": "/var/www",
  "status": "connected",
  "transfer_mode": "binary",
  "server_version": "SSH-2.0-OpenSSH_9.7",
```

```
  "created_at": "2026-06-28T17:40:00Z",
  "expires_at": "2026-06-28T23:40:00Z"
}
```

GET /api/v1/sftp/sessions/{sftpSessionId}/ls

List directory contents.

Authentication: Bearer token required

Query Parameters: | Parameter | Type | Description | |---|---|---| | `path` | string | Absolute path to list (default: `/`) | | `show_hidden` | boolean | Include hidden files (default: false) |

Success Response — 200 OK:

```
{
  "path": "/var/www",
  "entries": [
    {
      "name": "html",
      "path": "/var/www/html",
      "type": "directory",
      "size": 4096,
      "permissions": "drwxr-xr-x",
      "permissions_octal": "0755",
      "owner": "www-data",
      "group": "www-data",
      "modified_at": "2026-06-20T14:00:00Z",
      "is_symlink": false,
      "symlink_target": null
    },
    {
      "name": "index.html",
      "path": "/var/www/index.html",
      "type": "file",
      "size": 8192,
      "permissions": "-rw-r--r--",
      "permissions_octal": "0644",
      "owner": "www-data",
      "group": "www-data",
      "modified_at": "2026-06-28T10:00:00Z",
      "is_symlink": false,
      "symlink_target": null
```

```
    }
  ],
  "total_entries": 2
}
```

POST /api/v1/sftp/sessions/{sftpSessionId}/upload

Upload a file to the remote host.

Authentication: Bearer token required
Content-Type: `multipart/form-data`
Max size: 10 GB per file (chunked)

Request Body (multipart):

```
Content-Disposition: form-data; name="file"; filename="app.tar.gz"
Content-Type: application/gzip
```

Query Parameters: | Parameter | Type | Description | |---|---|---| | `path` | string | Remote destination path | | `overwrite` | boolean | Overwrite if exists (default: false) | | `chmod` | string | Set permissions after upload (e.g., `0755`) |

Success Response — 201 Created:

```
{
  "transfer_id": "xfer-550e8400-0000-0000-0000-aabbccddeeff",
  "local_filename": "app.tar.gz",
  "remote_path": "/var/www/app.tar.gz",
  "bytes_transferred": 25600000,
  "checksum_sha256": "sha256:abcdef...",
  "duration_ms": 4800,
  "transferred_at": "2026-06-28T17:40:05Z"
}
```

GET /api/v1/sftp/sessions/{sftpSessionId}/download

Download a file from the remote host.

Authentication: Bearer token required

Query Parameters: | Parameter | Type | Description | |---|---|---| | `path` | string | Remote file path |

Success Response — 200 OK:

```
Content-Type: application/octet-stream
Content-Disposition: attachment; filename="nginx.conf"
Content-Length: 4096
X-File-Permissions: 0644
X-File-Owner: root
X-File-Modified: Wed, 28 Jun 2026 10:00:00 GMT

<binary data>
```

POST /api/v1/sftp/sessions/{sftpSessionId}/mkdir

Create a remote directory.

Authentication: Bearer token required

Request Body:

```
{
  "path": "/var/www/newdir",
  "permissions": "0755",
  "recursive": true
}
```

Success Response — 201 Created:

```
{
  "path": "/var/www/newdir",
  "created_at": "2026-06-28T17:40:00Z"
}
```

DELETE /api/v1/sftp/sessions/{sftpSessionId}/rm

Delete a remote file or directory.

Authentication: Bearer token required

Query Parameters: | Parameter | Type | Description | |—|—|—| | `path` | string | Remote path to delete | | `recursive` | boolean | Delete directories recursively (default: false) |

Success Response — 204 No Content

POST /api/v1/sftp/sessions/{sftpSessionId}/rename

Rename or move a remote file or directory.

Authentication: Bearer token required

Request Body:

```
{
  "source": "/var/www/old-name.html",
  "destination": "/var/www/new-name.html"
}
```

Success Response — 200 OK:

```
{
  "source": "/var/www/old-name.html",
  "destination": "/var/www/new-name.html",
  "renamed_at": "2026-06-28T17:40:00Z"
}
```

POST /api/v1/sftp/sessions/{sftpSessionId}/chmod

Change permissions on a remote file or directory.

Authentication: Bearer token required

Request Body:

```
{
  "path": "/var/www/script.sh",
  "permissions": "0755",
  "recursive": false
}
```

Success Response — 200 OK:

```
{
  "path": "/var/www/script.sh",
  "old_permissions": "0644",
  "new_permissions": "0755"
}
```

GET /api/v1/sftp/sessions/{sftpSessionId}/stat

Get file or directory metadata.

Authentication: Bearer token required

Query Parameters: | Parameter | Type | Description | |—|—|—| | `path` | string | Remote path | | `follow_symlinks` | boolean | Resolve symlinks (default: true) |

Success Response — 200 OK:

```
{
  "path": "/var/www/html/index.html",
  "type": "file",
  "size": 8192,
  "permissions": "-rw-r--r--",
  "permissions_octal": "0644",
  "owner": "www-data",
  "group": "www-data",
  "uid": 33,
  "gid": 33,
  "accessed_at": "2026-06-28T16:00:00Z",
  "modified_at": "2026-06-28T10:00:00Z",
  "changed_at": "2026-06-28T10:00:00Z",
  "is_symlink": false,
  "symlink_target": null
}
```

DELETE /api/v1/sftp/sessions/{sftpSessionId}

Close an SFTP session and release resources.

Authentication: Bearer token required

Success Response — 204 No Content

---

8. Port Forwarding Service REST API

Base path: `/api/v1/port-forwards`
Service: `portforward-service` (port 8088 internal)
Database: `session_db` (port forwarding tables)

Authorization & blast-radius gating. A `dynamic` rule (SOCKS5 proxy) is the highest-blast-radius port forward type — it turns the tunnel into a general-purpose proxy for arbitrary destinations reachable from the target host's network, not a single fixed `remote_address:remote_port`. Accordingly:

- Creating (`POST`) or activating (`POST .../start`) a `dynamic` rule requires `write` permission on the target host's vault **and** the org-level policy flag `organizations.settings.allow_dynamic_port_forward` (default `false`) — a `local`/`remote` rule requires only ordinary `write` permission.
- Every `start`/`stop` of a `dynamic` rule writes a fail-closed `port_forward.dynamic_started` / `port_forward.dynamic_stopped` audit event (§17.10.1 pattern) recording the target host and the requesting user, independent of the ordinary `port_forward.status` transition already implied by `port_forward_connections`.
- `organizations.settings.allow_dynamic_port_forward = false` causes both the create and the start call to return `403` with `code: "dynamic_forwarding_disabled_by_org_policy"`, rather than silently downgrading the rule to a different `type`.

GET /api/v1/port-forwards

List all port forwarding rules.

Authentication: Bearer token required

Query Parameters: | Parameter | Type | Description | |---|---|---| | `host_id` | UUID | Filter by host | | `type` | string | `local`, `remote`, `dynamic` | | `status` | string | `active`, `inactive` |

Success Response — 200 OK:


```
{
  "data": [
    {
      "id": "pf-550e8400-0000-0000-0000-aabbccddeeff",
      "name": "PostgreSQL Dev Access",
      "host_id": "host-550e8400-0000-0000-0000-aabbccddeeff",
      "type": "local",
      "local_address": "127.0.0.1",
      "local_port": 15432,
      "remote_address": "localhost",
      "remote_port": 5432,
```

```
      "status": "active",
      "auto_start": true,
      "created_at": "2026-01-15T09:00:00Z"
    }
  ],
  "pagination": { "total_count": 5, "has_next": false }
}
```

POST /api/v1/port-forwards

Create a port forwarding rule.

Authentication: Bearer token required; `write` permission on the target host's vault. Creating a `type: dynamic` rule additionally requires `organizations.settings.allow_dynamic_port_forward = true` (see "Authorization & blast-radius gating" above).

Request Body:

```
{
  "name": "Redis Dev Access",
  "host_id": "host-550e8400-0000-0000-0000-aabbccddeeff",
  "type": "local",
  "local_address": "127.0.0.1",
  "local_port": 16379,
  "remote_address": "localhost",
  "remote_port": 6379,
  "auto_start": true,
  "bind_address": "127.0.0.1",
  "description": "Forward remote Redis port to local dev"
}
```

`type` values: - `local`: Forward `local_port` on client to `remote_address:remote_port` on server - `remote`: Forward `remote_port` on server to `local_address:local_port` on client
- `dynamic`: SOCKS5 proxy on `local_port` — gated, see above

Success Response — 201 Created: Full rule object.

Error Responses:

Status	Condition
403	

Status	Condition
	`type: dynamic` requested but `allow_dynamic_port_forward` is `false` for this org

GET /api/v1/port-forwards/{ruleId}

Get a specific port forwarding rule.

Authentication: Bearer token required

Success Response — 200 OK: Full rule object.

PUT /api/v1/port-forwards/{ruleId}

Update a port forwarding rule. The rule must be stopped first.

Authentication: Bearer token required

Request Body: Same as POST, all fields optional.

Success Response — 200 OK: Updated rule object.

DELETE /api/v1/port-forwards/{ruleId}

Delete a port forwarding rule. Rule is automatically stopped first.

Authentication: Bearer token required

Success Response — 204 No Content

POST /api/v1/port-forwards/{ruleId}/start

Activate a port forwarding rule (establishes SSH tunnel).

Authentication: Bearer token required; for a `type: dynamic` rule, re-checked against `organizations.settings.allow_dynamic_port_forward` at activation time (not just at rule-creation time — an org may disable dynamic forwarding after the rule was created).

Error Responses:

Status	Condition
403	Rule is `type: dynamic` and `allow_dynamic_port_forward` is now `false` for this org

Success Response — 200 OK:

```
{
  "id": "pf-550e8400-0000-0000-0000-aabbccddeeff",
  "status": "active",
  "connection_id": "pfconn-550e8400-0000-0000-0000-aabbccddeeff",
  "started_at": "2026-06-28T17:40:00Z",
  "local_endpoint": "127.0.0.1:16379"
}
```

POST /api/v1/port-forwards/{ruleId}/stop

Deactivate a port forwarding rule (closes SSH tunnel).

Authentication: Bearer token required

Success Response — 200 OK:

```
{
  "id": "pf-550e8400-0000-0000-0000-aabbccddeeff",
  "status": "inactive",
  "stopped_at": "2026-06-28T17:45:00Z",
  "bytes_transferred": 4096000
}
```

GET /api/v1/port-forwards/{ruleId}/status

Get real-time status of a port forwarding connection.

Authentication: Bearer token required

Success Response — 200 OK:

```
{
  "id": "pf-550e8400-0000-0000-0000-aabbccddeeff",
  "status": "active",
  "started_at": "2026-06-28T17:40:00Z",
  "bytes_sent": 102400,
```

```
  "bytes_received": 2048000,
  "active_connections": 3,
  "last_activity_at": "2026-06-28T17:44:55Z"
}
```

---

9. Keychain Service REST API

Base path: `/api/v1/keys` Service: `keychain-service` (port 8089 internal) Database: `keychain_db` (PostgreSQL)

GET /api/v1/keys

List SSH keys and certificates.

Authentication: Bearer token required

Query Parameters: | Parameter | Type | Description | |---|---|---| | `vault_id` | UUID | Filter by vault | | `type` | string | `ssh_key`, `certificate`, `pgp` | | `q` | string | Search by name |

Success Response — 200 OK:

```
{
  "data": [
    {
      "id": "key-550e8400-0000-0000-0000-112233445566",
      "name": "Production Deploy Key",
      "type": "ssh_key",
      "algorithm": "ed25519",
      "bits": null,
      "fingerprint":
        "SHA256:abcdef1234567890abcdef1234567890abcdef12",
      "comment": "deploy@helixterm",
      "vault_id": "vault-550e8400-0000-0000-0000-aabbccddeeff",
      "has_passphrase": false,
```

```
      "deployments_count": 47,
      "last_used_at": "2026-06-28T16:00:00Z",
      "expires_at": null,
      "created_at": "2026-01-15T09:00:00Z",
      "_links": {
        "self": { "href": "https://api.helixterminator.io/api/v1/
        keys/key-550e8400-0000-0000-0000-112233445566" },
        "public_key": { "href": "https://api.helixterminator.io/
        api/v1/keys/key-550e8400-0000-0000-0000-112233445566/
        public" }
      }
    }
  ],
  "pagination": { "total_count": 12, "has_next": false }
}
```

POST /api/v1/keys

Create or register a new SSH key.

Authentication: Bearer token required

Request Body:

```
{
  "name": "Staging Deploy Key",
  "type": "ssh_key",
  "vault_id": "vault-550e8400-0000-0000-0000-aabbccddeeff",
  "comment": "staging@helixterm",
  "encrypted_private_key": "base64-encoded-encrypted-pem",
  "public_key": "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIB...
        staging@helixterm",
  "passphrase_protected": false
}
```

The private key is always stored encrypted. The `encrypted_private_key` must be encrypted with the vault's key.

Success Response — 201 Created: Key object (without private key material).

GET /api/v1/keys/{keyId}

Get key metadata (never returns the private key).

Authentication: Bearer token required

Success Response — 200 OK: Full key object including deployment list.

PUT /api/v1/keys/{keyId}

Update key name, comment, or vault association.

Authentication: Bearer token required

Request Body:

```
{
  "name": "Updated Key Name",
  "comment": "new-comment@helixterm"
}
```

Success Response — 200 OK: Updated key object.

DELETE /api/v1/keys/{keyId}

Delete a key. Hosts using this key will require re-association.

Authentication: Bearer token required

Success Response — 204 No Content

POST /api/v1/keys/generate

Generate a new SSH key pair server-side (or client-side via this endpoint's parameter guidance).

Authentication: Bearer token required

Request Body:

```
{
  "name": "Auto-Generated Key 2026",
  "algorithm": "ed25519",
  "comment": "generated@helixterm-2026-06-28",
  "vault_id": "vault-550e8400-0000-0000-0000-aabbccddeeff",
  "bits": null
}
```

`algorithm` values: `ed25519` (recommended), `ecdsa`, `rsa4096`.
`bits`: Only applicable for RSA (2048, 3072, 4096). Ed25519 ignores this.

Success Response — 201 Created:

```
{
  "id": "key-aabbccdd-0000-0000-0000-112233445566",
  "name": "Auto-Generated Key 2026",
  "algorithm": "ed25519",
  "fingerprint":
        "SHA256:newkeyfingerprinthere12345678901234567890",
  "public_key": "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIB2...
        generated@helixterm-2026-06-28",
  "encrypted_private_key": "base64-encoded-encrypted-private-key",
  "created_at": "2026-06-28T17:40:00Z"
}
```

The private key is returned once at generation, encrypted with the vault key. Store it securely.

GET /api/v1/keys/{keyId}/public

Get the public key in OpenSSH authorized_keys format.

Authentication: Bearer token required

Success Response — 200 OK:

```
Content-Type: text/plain; charset=utf-8

ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIB2abcdef1234567890abcdef1234567890abcd
deploy@helixterm
```

POST /api/v1/keys/{keyId}/deploy

Deploy the public key to a host's `authorized_keys` file via SSH.

Authentication: Bearer token required

Request Body:

```
{
  "host_id": "host-550e8400-0000-0000-0000-aabbccddeeff",
  "target_user": "ubuntu",
```

```
  "auth_key_id": "key-aabbccdd-0000-0000-0000-existing-key",
  "options": {
    "command": null,
    "restrict": false,
    "no_port_forwarding": false,
    "no_agent_forwarding": false
  }
}
```

`auth_key_id` is the key used to authenticate the deployment operation itself.

Success Response — 200 OK:

```
{
  "deployed_to": "host-550e8400-0000-0000-0000-aabbccddeeff",
  "host_name": "prod-web-01",
  "target_user": "ubuntu",
  "authorized_keys_path": "/home/ubuntu/.ssh/authorized_keys",
  "deployed_at": "2026-06-28T17:40:00Z",
  "was_already_present": false
}
```

POST /api/v1/keys/import

Import an SSH key from PEM, OpenSSH, or PKCS#8 format.

Authentication: Bearer token required

Request Body:

```
{
  "name": "Imported Legacy Key",
  "vault_id": "vault-550e8400-0000-0000-0000-aabbccddeeff",
  "private_key_pem": "-----BEGIN OPENSSH PRIVATE KEY-----
        \nb3BlbnNzaC1rZXkAAAA...\n-----END OPENSSH PRIVATE
        KEY-----",
  "passphrase": "optional-key-passphrase",
  "comment": "legacy-server@old-domain"
}
```

Success Response — 201 Created: Key object.

---

10. Snippet Service REST API

Base path: `/api/v1/snippets`
Service: `snippet-service` (port 8090 internal)
Database: `snippet_db` (PostgreSQL)

GET /api/v1/snippets

List snippets accessible to the user.

Authentication: Bearer token required

Query Parameters: | Parameter | Type | Description | |—|—|—| | `vault_id` | UUID | Filter by vault | | `category_id` | UUID | Filter by category | | `language` | string | Filter by language (e.g., `bash`, `python`) | | `q` | string | Full-text search |

Success Response — 200 OK:


```
{
  "data": [
    {
      "id": "snip-550e8400-0000-0000-0000-aabbccddeeff",
      "name": "Restart NGINX",
      "description": "Restart the NGINX web server and verify it
        started",
      "content": "sudo systemctl restart nginx && sudo systemctl
        status nginx",
      "language": "bash",
      "category_id": "cat-550e8400-0000-0000-0000-aabbccddeeff",
      "category_name": "System Administration",
      "vault_id": "vault-550e8400-0000-0000-0000-aabbccddeeff",
      "tags": ["nginx", "systemctl", "webserver"],
      "shared": false,
      "parameters": [],
      "executions_count": 142,
      "last_executed_at": "2026-06-28T15:00:00Z",
      "created_by": "550e8400-e29b-41d4-a716-446655440000",
      "created_at": "2026-01-15T09:00:00Z",
      "updated_at": "2026-06-20T10:00:00Z"
    }
  ],
  "pagination": { "total_count": 87, "has_next": false }
}
```

POST /api/v1/snippets

Create a new snippet.

Authentication: Bearer token required

Request Body:

```
{
  "name": "Check Disk Usage",
  "description":
        "Show disk usage sorted by size for the specified
        directory",
  "content":
        "du -sh {{directory}}/* 2>/dev/null | sort -rh | head
        -20",
  "language": "bash",
  "category_id": "cat-550e8400-0000-0000-0000-aabbccddeeff",
  "vault_id": "vault-550e8400-0000-0000-0000-aabbccddeeff",
  "tags": ["disk", "storage", "monitoring"],
  "shared": false,
  "parameters": [
    {
      "name": "directory",
      "description": "Directory to check",
      "type": "string",
      "default": "/var",
      "required": true
    }
  ]
}
```

Parameters in `content` use `{{parameter_name}}` syntax for variable substitution. **This is never raw shell string interpolation** — see "Parameter substitution safety" under the `execute` endpoint below for the exact mechanism that prevents a parameter value from being interpreted as shell metacharacters.

Success Response — 201 Created: Full snippet object.

GET /api/v1/snippets/{snippetId}

Get a specific snippet.

Authentication: Bearer token required

Success Response — 200 OK: Full snippet with execution history summary.

PUT /api/v1/snippets/{snippetId}

Update a snippet.

Authentication: Bearer token required

Request Body: Same as POST, all fields optional.

Success Response — 200 OK: Updated snippet.

DELETE /api/v1/snippets/{snippetId}

Delete a snippet.

Authentication: Bearer token required

Success Response — 204 No Content

POST /api/v1/snippets/{snippetId}/execute

Execute a snippet on one or more hosts.

Authentication: Bearer token required

Request Body:

```
{
  "host_ids": [
    "host-550e8400-0000-0000-0000-aabbccddeeff",
    "host-660e8400-0000-0000-0000-bbccddeeaabb"
  ],
  "parameters": {
    "directory": "/home"
  },
  "execution_mode": "parallel",
  "timeout_seconds": 60,
  "record": false
}
```

`execution_mode`: `parallel` (all hosts simultaneously) or `sequential` (one at a time).

Parameter substitution safety. `{{parameter_name}}` tokens in `content` are **never** resolved via raw string interpolation into a shell command line — that would let a parameter value such as `/var; rm -rf / #` execute as a second, attacker-controlled command. The snippet execution engine instead:

1. Parses `content` into an argument-vector template at snippet-creation time (each `{{name}}` token records its position and the parameter's declared `type`: `string`, `path`, `integer`, `enum`).
2. At execution time, validates every supplied parameter value against its declared `type` and (if present) `allowed_values` / `pattern` constraint — a value containing a shell metacharacter (`; | & $ ` \ < > ( )` newline) is **rejected outright** for `type: path` or `type: string` parameters unless the parameter is explicitly declared `type: raw_unsafe` (an opt-in escape hatch requiring vault `admin` permission on the owning vault, itself flagged in `snippet_execution_results`).
3. Builds the remote command as an **argument vector** (`exec.Command(shell, "-c", template, "--", arg1, arg2, …)` with parameters passed as positional shell arguments, not string-concatenated into the template) so the underlying `/bin/sh -c` invocation never sees attacker-controlled bytes inside the command-template portion of the string — equivalent to parameterized-query binding for shell exec. For the common case, values are additionally single-quote-escaped (`' → '\''`) as a second, defense-in-depth layer before substitution, so a snippet author who intentionally still uses `{{param}}` inline inside a larger shell pipeline (rather than a trailing positional argument) is also protected.
4. Rejects a template whose substitution would change the number of shell tokens the template author wrote (e.g. a value containing an unescaped, unquoted space expanding into two arguments where one was expected) unless the parameter is declared `type: path` with `quote: true` (the default for `path`).

Authorization + blast-radius gate. Multi-host execution is a privileged, high-blast-radius action:

- `host_ids.length > 1` requires the caller to hold `write` permission on **every** targeted host's vault (not merely the snippet's own vault) — checked per-host, not just once for the snippet.
- An org-level policy setting (`organizations.settings.snippet_max_broadcast_hosts`, default `10`)

caps `host_ids.length`; exceeding it returns `422` with the configured limit, unless the caller holds `org_admin`.
- Every execution writes a fail-closed `snippet.executed` audit event (§17.10.1 pattern) **before** dispatch begins, recording the full `host_ids` list, `execution_mode`, and the resolved (post-validation, pre-substitution) parameter values — so a blast-radius incident is fully reconstructable even if the execution itself later fails or times out.

Success Response — 202 Accepted:

```
{
  "execution_id": "exec-550e8400-0000-0000-0000-aabbccddeeff",
  "snippet_id": "snip-550e8400-0000-0000-0000-aabbccddeeff",
  "status": "running",
  "host_count": 2,
  "started_at": "2026-06-28T17:40:00Z"
}
```

Error Responses:

Status	Condition
400	Parameter value rejected (shell metacharacter in a non-`raw_unsafe` parameter)
403	Caller lacks `write` permission on one or more targeted hosts' vault
422	`host_ids.length` exceeds `snippet_max_broadcast_hosts` and caller is not `org_admin`

Poll `GET /api/v1/snippets/executions/{executionId}` for results.

GET /api/v1/snippets/search

Full-text search across snippet names, descriptions, and content.

Authentication: Bearer token required

Query Parameters: | Parameter | Type | Description | |—|—|—| | `q` | string | Search query (required) | | `limit` | integer | Max results (default 20, max 50) |

Success Response — 200 OK:

```
{
  "query": "nginx restart",
  "results": [
    {
      "id": "snip-550e8400-0000-0000-0000-aabbccddeeff",
      "name": "Restart NGINX",
      "_score": 0.97,
      "_highlights": {
        "name": "<mark>Restart NGINX</mark>",
        "content": "sudo systemctl <mark>restart nginx</mark>
        && ..."
      }
    }
  ],
  "total_results": 3
}
```


---

11. Workspace Service REST API

Base path: `/api/v1/workspaces`
Service: `workspace-service` (port 8091 internal)
Database: `workspace_db` (PostgreSQL)

A **Workspace** is a saved layout of terminal tabs, panels, and connections — analogous to a saved IDE project layout.

GET /api/v1/workspaces

List all workspaces.

Authentication: Bearer token required

Success Response — 200 OK:


```
{
  "data": [
    {
      "id": "ws-550e8400-0000-0000-0000-aabbccddeeff",
      "name": "Production Debug Session",
      "description": "4-pane layout for production debugging",
      "thumbnail_url": "https://cdn.helixterminator.io/ws-
        thumbnails/ws-550e8400.png",
      "layout": {
```

```
        "type": "grid",
        "panels": 4,
        "arrangement": "2x2"
      },
      "session_count": 4,
      "is_template": false,
      "template_id": null,
      "pinned": true,
      "last_opened_at": "2026-06-28T16:00:00Z",
      "created_at": "2026-01-15T09:00:00Z"
    }
  ],
  "pagination": { "total_count": 8, "has_next": false }
}
```

POST /api/v1/workspaces

Create a new workspace.

Authentication: Bearer token required

Request Body:


```
{
  "name": "Database Monitoring",
  "description": "Monitor all PostgreSQL instances",
  "layout": {
    "type": "split",
    "direction": "horizontal",
    "panes": [
      {
        "id": "pane-1",
        "type": "terminal",
        "host_id": "host-550e8400-0000-0000-0000-aabbccddeeff",
        "startup_snippet_id": "snip-
        aabbccdd-0000-0000-0000-112233445566",
        "size_percent": 50
      },
      {
        "id": "pane-2",
        "type": "terminal",
        "host_id": "host-660e8400-0000-0000-0000-bbccddeeaabb",
        "startup_snippet_id": null,
        "size_percent": 50
```

```
      }
    ]
  },
  "auto_connect": true,
  "template_id": null
}
```

Success Response — 201 Created: Full workspace object.

GET /api/v1/workspaces/{workspaceId}

Get a specific workspace.

Authentication: Bearer token required

Success Response — 200 OK: Full workspace object.

PUT /api/v1/workspaces/{workspaceId}

Update a workspace layout or metadata.

Authentication: Bearer token required

Success Response — 200 OK: Updated workspace.

DELETE /api/v1/workspaces/{workspaceId}

Delete a workspace.

Authentication: Bearer token required

Success Response — 204 No Content

POST /api/v1/workspaces/{workspaceId}/restore

Restore a workspace to a previous saved state.

Authentication: Bearer token required

Request Body:

```
{
  "snapshot_id": "snap-550e8400-0000-0000-0000-aabbccddeeff"
}
```

Success Response — 200 OK: Restored workspace.

GET /api/v1/workspace-templates

List available workspace templates.

Authentication: Bearer token required

Success Response — 200 OK:

```
{
  "data": [
    {
      "id": "tmpl-550e8400-0000-0000-0000-aabbccddeeff",
      "name": "4-Pane Server Monitor",
      "description": "Connect to 4 servers in a 2x2 grid",
      "category": "monitoring",
      "pane_count": 4,
      "preview_url": "https://cdn.helixterminator.io/templates/
        4pane-preview.png",
      "usage_count": 1247,
      "created_by": "system",
      "created_at": "2026-01-01T00:00:00Z"
    }
  ]
}
```

POST /api/v1/workspace-templates

Create a workspace template from an existing workspace.

Authentication: Bearer token required

Request Body:

```
{
  "workspace_id": "ws-550e8400-0000-0000-0000-aabbccddeeff",
  "name": "My Monitoring Template",
  "description": "4-pane monitoring template",
```

```
  "public": false
}
```

Success Response — 201 Created: Template object.

---

12. Organization & Team Service REST API

Base path: `/api/v1/orgs`
Service: `org-service` (port 8092 internal)
Database: `org_db` (PostgreSQL)

GET /api/v1/orgs/me

Get the authenticated user's organization.

Authentication: Bearer token required

Success Response — 200 OK:

```
{
  "id": "org-550e8400-0000-0000-0000-000000000001",
  "name": "ExampleCorp",
  "slug": "examplecorp",
  "plan": "enterprise",
  "member_count": 87,
  "max_members": 500,
  "domain": "examplecorp.com",
  "domain_verified": true,
  "sso_enabled": true,
  "sso_provider": "azure",
  "enforce_mfa": true,
  "session_recording_required": true,
  "vault_count": 12,
  "host_count": 347,
  "created_at": "2025-06-01T00:00:00Z"
}
```

GET /api/v1/orgs/me/members

List organization members.

Authentication: Bearer token required; requires org `org_admin` or `member` role

Query Parameters: | Parameter | Type | Description | |---|---|---| | `role` | string | Filter by role: `org_admin`, `team_admin`, `member`, `auditor`, `api_user` | | `team_id` | UUID | Filter by team membership | | `q` | string | Search by name or email | | `cursor` | string | Pagination cursor | | `limit` | integer | Page size |

Success Response — 200 OK:

```
{
  "data": [
    {
      "id": "mem-550e8400-0000-0000-0000-aabbccddeeff",
      "user_id": "550e8400-e29b-41d4-a716-446655440000",
      "email": "alice@example.com",
      "display_name": "Alice Smith",
      "role": "org_admin",
      "teams": ["platform", "sre"],
      "invited_by": null,
      "joined_at": "2025-06-01T00:00:00Z",
      "last_active_at": "2026-06-28T17:00:00Z",
      "status": "active"
    }
  ],
  "pagination": { "total_count": 87, "has_next": true }
}
```

POST /api/v1/orgs/me/invitations

Invite a new member to the organization.

Authentication: Bearer token required; requires `org_admin` role

Request Body:

```
{
  "email": "newmember@examplecorp.com",
  "role": "member",
  "team_ids": ["team-550e8400-0000-0000-0000-aabbccddeeff"],
  "message": "Welcome to HelixTerminator!"
}
```

Success Response — 201 Created:

```
{
  "invitation_id": "inv-550e8400-0000-0000-0000-aabbccddeeff",
  "email": "newmember@examplecorp.com",
  "role": "member",
  "invited_by": "alice@example.com",
  "expires_at": "2026-07-05T17:40:00Z",
  "status": "pending"
}
```

GET /api/v1/orgs/me/teams

List teams in the organization.

Authentication: Bearer token required

Success Response — 200 OK:

```
{
  "data": [
    {
      "id": "team-550e8400-0000-0000-0000-aabbccddeeff",
      "name": "Platform Engineering",
      "slug": "platform",
      "description": "Infrastructure and platform team",
      "member_count": 12,
      "vault_access": [
        {
          "vault_id": "vault-550e8400-0000-0000-0000-
        aabbccddeeff",
          "vault_name": "Production Credentials",
          "permission": "write"
        }
      ],
      "created_at": "2025-06-01T00:00:00Z"
    }
  ],
  "pagination": { "total_count": 5, "has_next": false }
}
```

POST /api/v1/orgs/me/teams

Create a new team.

Authentication: Bearer token required; requires `org_admin` role

Request Body:

```
{
  "name": "Security Engineering",
  "slug": "security",
  "description": "Security and compliance team"
}
```

Success Response — 201 Created: Full team object.

GET /api/v1/orgs/me/teams/{teamId}

Get a specific team.

Authentication: Bearer token required

Success Response — 200 OK: Full team with members.

PUT /api/v1/orgs/me/teams/{teamId}

Update a team.

Authentication: Bearer token required; requires `org_admin` role

Success Response — 200 OK: Updated team.

DELETE /api/v1/orgs/me/teams/{teamId}

Delete a team.

Authentication: Bearer token required; requires `org_admin` role

Success Response — 204 No Content

POST /api/v1/orgs/me/teams/{teamId}/members

Add a member to a team.

Authentication: Bearer token required; requires `org_admin` role

Request Body:

```
{
  "user_id": "660e8400-e29b-41d4-a716-556655440000",
  "role": "member"
}
```

Success Response — 201 Created: Team member object.

DELETE /api/v1/orgs/me/teams/{teamId}/members/{userId}

Remove a member from a team.

Authentication: Bearer token required; requires `org_admin` role

Success Response — 204 No Content

---

13. Audit Service REST API

Base path: `/api/v1/audit`
Service: `audit-service` (port 8093 internal)
Database: `audit_db` (PostgreSQL, append-only, partitioned)

GET /api/v1/audit/events

Query audit events with comprehensive filtering.

Authentication: Bearer token required; requires org `org_admin` role

Query Parameters: | Parameter | Type | Description | |—|—|—| | `cursor` | string | Pagination cursor | | `limit` | integer | Page size (max 100) | | `user_id` | UUID | Filter by user | | `event_type` | string | Filter by event type (repeatable) | | `resource_type` | string | `host`, `vault`, `session`, `key`, etc. | | `resource_id` | UUID | Filter by specific resource | |

`outcome` | string | `success`, `failure` | | `ip_address` | string | Filter by source IP | | `created_at_gte` | datetime | After date | | `created_at_lte` | datetime | Before date | | `sort` | string | `created_at:desc` (default) |

Success Response — 200 OK:

```
{
  "data": [
    {
      "id": "audit-550e8400-0000-0000-0000-aabbccddeeff",
      "event_type": "session.ssh.started",
      "user_id": "550e8400-e29b-41d4-a716-446655440000",
      "user_email": "alice@example.com",
      "resource_type": "host",
      "resource_id": "host-550e8400-0000-0000-0000-aabbccddeeff",
      "resource_name": "prod-web-01",
      "outcome": "success",
      "ip_address": "192.168.1.10",
      "user_agent": "HelixTerminator/1.0",
      "session_id": "sess-550e8400-0000-0000-0000-aabbccddeeff",
      "metadata": {
        "host_name": "prod-web-01",
        "jump_chain": [],
        "recording_enabled": true,
        "ticket_ref": "INC-20260628-001"
      },
      "hash": "sha256:event-integrity-hash",
      "prev_hash": "sha256:previous-event-hash",
      "created_at": "2026-06-28T17:40:00Z"
    }
  ],
  "pagination": {
    "cursor_next": "eyJpZCI6...",
    "has_next": true,
    "total_count": 48291
  }
}
```

GET /api/v1/audit/events/{eventId}

Get a specific audit event by ID.

Authentication: Bearer token required; requires org `org_admin` role

Success Response — 200 OK: Full audit event object.

GET /api/v1/audit/export

Export audit events to a file.

Authentication: Bearer token required; requires org `org_admin` role

Query Parameters: | Parameter | Type | Description | |---|---|---| | `format` | string | `json`, `csv`, `syslog` | | `created_at_gte` | datetime | Start date (required) | | `created_at_lte` | datetime | End date (required) | | `event_type` | string | Filter by event type |

Success Response — 202 Accepted:

```
{
  "export_id": "aexp-550e8400-0000-0000-0000-aabbccddeeff",
  "status": "processing",
  "estimated_rows": 48291,
  "format": "json",
  "requested_at": "2026-06-28T17:40:00Z"
}
```

---

14. AI Service REST API

Base path: `/api/v1/ai`
Service: `ai-service` (port 8094 internal)
Rate Limit: 100 requests per user per hour

POST /api/v1/ai/complete

AI-powered command completion in the terminal context.

Authentication: Bearer token required

Request Body:

```
{
  "context": {
    "cwd": "/var/www/html",
    "hostname": "prod-web-01",
    "os": "Ubuntu 24.04",
```

```
    "shell": "bash",
    "history": [
      "ls -la",
      "cat nginx.conf",
      "sudo systemctl status nginx"
    ]
  },
  "partial_command": "sudo journalctl -u nginx",
  "max_suggestions": 5
}
```

Success Response — 200 OK:

```
{
  "suggestions": [
    {
      "completion":
        "sudo journalctl -u nginx --since '1 hour ago' | tail
        -100",
      "description": "View nginx logs from the last hour",
      "confidence": 0.95
    },
    {
      "completion": "sudo journalctl -u nginx -n 50 --no-pager",
      "description": "Show last 50 log entries",
      "confidence": 0.88
    }
  ],
  "model": "helixterm-cmd-v1",
  "latency_ms": 124
}
```

POST /api/v1/ai/explain

Explain a command in plain English.

Authentication: Bearer token required

Request Body:

```
{
  "command": "find /var/log -name '*.log' -mtime +7 -exec gzip {}
        \\;",
  "context": {
    "os": "Ubuntu 24.04",
```

```
    "shell": "bash"
  },
  "detail_level": "detailed"
}
```

`detail_level`: `brief`, `standard`, `detailed`.

Success Response — 200 OK:

```
{
  "command": "find /var/log -name '*.log' -mtime +7 -exec gzip {}
      \\;",
  "explanation": {
    "summary":
        "Compresses all .log files in /var/log that haven't been
        modified in more than 7 days.",
    "parts": [
      {
        "token": "find /var/log",
        "description": "Start searching from the /var/log
        directory"
      },
      {
        "token": "-name '*.log'",
        "description": "Match only files ending in .log"
      },
      {
        "token": "-mtime +7",
        "description": "Modified more than 7 days ago"
      },
      {
        "token": "-exec gzip {} \\;",
        "description": "Execute gzip on each matching file"
      }
    ],
    "risk_level": "low",
    "side_effects": ["Creates .log.gz files", "Deletes
        original .log files (gzip default)"],
    "man_page_refs": ["find(1)", "gzip(1)"]
  },
  "model": "helixterm-explain-v1",
  "latency_ms": 89
}
```

POST /api/v1/ai/suggest

Get AI suggestions for the next action based on terminal context.

Authentication: Bearer token required

Request Body:

```
{
  "context": {
    "cwd": "/var/www",
    "hostname": "prod-web-01",
    "os": "Ubuntu 24.04",
    "shell": "bash",
    "recent_output": "nginx: [warn] could not build optimal
        variables_hash...\nnginx: the configuration file /etc/
        nginx/nginx.conf syntax is ok",
    "history": [
      "sudo nginx -t",
      "sudo cat /etc/nginx/nginx.conf"
    ]
  },
  "goal": "Fix the nginx configuration warning"
}
```

Success Response — 200 OK:

```
{
  "suggestions": [
    {
      "type": "command",
      "command": "sudo sed -i 's/variables_hash_bucket_size.*/
        variables_hash_bucket_size 128;/' /etc/nginx/nginx.conf",
      "description": "Increase variables_hash_bucket_size to
        resolve the warning",
      "confidence": 0.91,
      "explanation": "The warning indicates
        variables_hash_bucket_size needs to be increased. The
        default is 64 bytes; setting it to 128 typically resolves
        this."
    },
    {
      "type": "documentation",
      "url": "https://nginx.org/en/docs/hash.html",
      "description": "NGINX hash configuration documentation"
    }
  ],
  "model": "helixterm-suggest-v1",
```

```
  "latency_ms": 156
}
```

---

15. WebSocket API

HelixTerminator uses WebSocket connections for real-time terminal I/O, collaboration, and vault sync.

Authentication

All WebSocket connections require authentication via: 1. **Query parameter:** `wss://proxy.helixterminator.io/...?token=<session_token>` 2. **First message:** Send `{"type":"auth","token":"<session_token>"}` within 5 seconds of connecting.

WS /api/v1/sessions/{sessionId}/terminal

Bidirectional terminal I/O stream. Protocol: Binary and text frames.

Connection URL:

```
wss://proxy.helixterminator.io/api/v1/sessions/sess-550e8400/terminal?token=st_v1_xxx
```

Client → Server messages:

Input data (terminal keystrokes):

```
{"type": "input", "data": "bHMgLWxhCg=="}
```

`data` is base64-encoded raw bytes.

Terminal resize:

```
{"type": "resize", "cols": 240, "rows": 60}
```

Ping/keepalive:

```
{"type": "ping"}
```

Server → Client messages:

Output data:

```
{"type": "output", "data": "dG90YWwgNDgK..."}
```

Session state:

```
{"type": "state", "status": "connected", "host": "prod-web-01",
        "latency_ms": 12}
```

Error:

```
{"type": "error", "code": "host_unreachable", "message":
        "Connection to host timed out after 30 seconds"}
```

Close:

```
{"type": "close", "code": 0, "message": "Session ended by user"}
```

WebSocket close codes:

Code	Meaning
4000	Normal close (session ended)
4001	Authentication failed
4002	Session expired
4003	Host connection lost
4004	Server error
4005	Rate limit exceeded
4006	Idle timeout (5 minutes of no input)
4007	Reconnect window expired (see below)

Reconnect / resume semantics. A dropped WebSocket (client network blip, load balancer idle timeout, mobile app backgrounding) MUST NOT discard in-flight terminal output or force the SSH session itself to terminate — the underlying SSH connection to the host and the WebSocket transport to the client are independent lifecycles.

- **Resume token.** The initial `state` message (and every subsequent one) includes a `resume_token`:

  ```
  {"type": "state", "status": "connected", "host": "prod-
          web-01", "latency_ms": 12, "resume_token":
          "rt_ws_v1_xxxxxxxxxxxxxxxxxxx"}
  ```

  The client persists the most recent `resume_token` (and the `last_event_id` below) in memory across a disconnect.

- **Last-event-id.** Every `output` frame carries a monotonically increasing `event_id` (backed by `session_events.id`, a `BIGSERIAL`, §17.5):

  ```
  {"type": "output", "data": "dG90YWwgNDgK...", "event_id":
          184213}
  ```

  The client tracks the highest `event_id` it has successfully rendered.

- **Reconnect request.** On reconnect, the client opens a new WebSocket to the same `wss://proxy.helixterminator.io/api/v1/sessions/{sessionId}/terminal` URL and sends, as its first message instead of (or in addition to) `auth`:

  ```
  {"type": "resume", "resume_token":
          "rt_ws_v1_xxxxxxxxxxxxxxxxxxxx", "last_event_id":
          184213}
  ```

- **Server-side resume window.** The proxy node holds the SSH connection and a bounded ring buffer of recent `session_events` (default: 5 minutes or 10,000 events, whichever is smaller — configurable per org) open for a **grace period** (default 60 seconds, `session:{sessionId}:state` TTL-refreshed per §18.1) after the WebSocket drops, keyed by `resume_token`. A `resume` request within the grace period and with a `resume_token` matching the held session:

  1. Re-authenticates the token (same validity rules as `auth`).
  2. **Output replay:** streams every buffered `output` event with `event_id > last_event_id` in order, each still tagged with its original `event_id`, before resuming live output — the client never misses or duplicates a byte of terminal output across the gap.
  3. Replies with a fresh `state` message (new `resume_token`, same underlying SSH session) once replay completes.

- **Grace period expiry:** if no `resume` arrives within the grace period, the proxy tears down the underlying SSH connection and the ring buffer, exactly as today (close code `4007` is sent to any late `resume` attempt, distinguishing "your reconnect was too slow" from `4002`/`4003`). A `resume` with an unrecognized/expired `resume_token` MUST fall back to the normal `POST /api/v1/sessions/ssh` new-session flow — it is never treated as an implicit new session under the old session's identity (that would let a stale/leaked `resume_token` silently attach a new client to someone else's session).

- **`recording_enabled` sessions:** replayed events are also fed to the same recording pipeline exactly once (deduplicated by `event_id`), so a reconnect never produces a truncated or duplicated `session_recordings` artifact.

WS /api/v1/sessions/{sessionId}/collab

Collaboration channel for shared terminal sessions.

Connection URL:

```
wss://proxy.helixterminator.io/api/v1/sessions/sess-550e8400/collab?token=st_v1_xxx
```

Server → Client messages:

Participant joined:

```
{"type": "participant_joined", "user_id": "660e8400-...",
        "display_name": "Bob Jones", "role": "viewer"}
```

Participant left:

```
{"type": "participant_left", "user_id": "660e8400-..."}
```

Cursor position (other participants' cursors):

```
{"type": "cursor", "user_id": "660e8400-...", "position": {"row":
        5, "col": 12}, "color": "#FF6B35"}
```

Chat message:

```
{"type": "chat", "user_id": "660e8400-...", "display_name": "Bob
        Jones", "message": "Check line 42", "timestamp":
        "2026-06-28T17:45:00Z"}
```

Client → Server messages:

Send chat:

```
{"type": "chat", "message": "I see the issue!"}
```

Request control:

```
{"type": "request_control"}
```

WS /api/v1/sync

Vault sync channel for real-time vault synchronization across clients.

Connection URL:

```
wss://api.helixterminator.io/api/v1/sync?
token=<access_token>&vault_id=<vault_id>
```

Client → Server messages:

Sync push:

```
{
  "type": "push",
  "vault_id": "vault-550e8400-...",
  "changes": [
    {
      "item_id": "item-550e8400-...",
      "operation": "upsert",
      "encrypted_data": "base64...",
      "version": 6,
      "checksum": "sha256:..."
    }
  ],
  "client_cursor": "cursor-value"
}
```

Server → Client messages:

Sync pull (server-initiated changes):

```
{
  "type": "pull",
  "vault_id": "vault-550e8400-...",
  "changes": [...],
  "server_cursor": "new-cursor"
}
```

Conflict notification:

```
{
  "type": "conflict",
  "item_id": "item-550e8400-...",
  "your_version": 5,
```

```
  "server_version": 7
}
```

---

16. gRPC Service Definitions

Internal gRPC services communicate over mTLS on the internal Kubernetes network. All proto files live at `internal/proto/` in the monorepo.

Base configuration:

```
syntax = "proto3";
package helixterm.v1;
option go_package = "helixterminator.io/core/internal/
        proto;proto";
```

---

auth.proto

```
syntax = "proto3";
package helixterm.v1;
option go_package = "helixterminator.io/core/internal/
        proto;proto";

import "google/protobuf/timestamp.proto";
import "google/protobuf/empty.proto";

// AuthService provides internal authentication operations called
        by other microservices.
service AuthService {
  // ValidateToken validates a JWT access token and returns the
        claims.
  rpc ValidateToken(ValidateTokenRequest) returns
        (ValidateTokenResponse);

  // ValidateApiKey validates an API key and returns the
        associated user and scopes.
  rpc ValidateApiKey(ValidateApiKeyRequest) returns
        (ValidateApiKeyResponse);

  // GetUserContext retrieves user context (user ID, org, roles)
        for an authenticated request.
  rpc GetUserContext(GetUserContextRequest) returns (UserContext);

  // RevokeToken adds a token to the blocklist.
```

```
  rpc RevokeToken(RevokeTokenRequest) returns
        (google.protobuf.Empty);

  // GetSession retrieves session details by session ID.
  rpc GetSession(GetSessionRequest) returns (Session);

  // InvalidateSession terminates a session.
  rpc InvalidateSession(InvalidateSessionRequest) returns
        (google.protobuf.Empty);

  // CheckPermission verifies a user has a specific permission on
        a resource.
  rpc CheckPermission(CheckPermissionRequest) returns
        (CheckPermissionResponse);

  // BulkCheckPermission checks multiple permissions at once.
  rpc BulkCheckPermission(BulkCheckPermissionRequest) returns
        (BulkCheckPermissionResponse);
}

message ValidateTokenRequest {
  string token = 1;
  repeated string required_scopes = 2;
  string audience = 3;
}

message ValidateTokenResponse {
  bool valid = 1;
  string user_id = 2;
  string session_id = 3;
  string org_id = 4;
  repeated string scopes = 5;
  bool mfa_verified = 6;
  google.protobuf.Timestamp expires_at = 7;
  string error_code = 8;
  string error_message = 9;
}

message ValidateApiKeyRequest {
  string api_key = 1;
  repeated string required_scopes = 2;
}

message ValidateApiKeyResponse {
  bool valid = 1;
  string user_id = 2;
```

```
  string key_id = 3;
  string org_id = 4;
  repeated string scopes = 5;
  repeated string allowed_ips = 6;
  string error_code = 7;
}

message GetUserContextRequest {
  string user_id = 1;
}

message UserContext {
  string user_id = 1;
  string email = 2;
  string display_name = 3;
  string org_id = 4;
  string org_slug = 5;
  string org_role = 6;
  repeated string team_ids = 7;
  repeated string permissions = 8;
  bool mfa_enabled = 9;
  string locale = 10;
  string timezone = 11;
  string status = 12;
}

message RevokeTokenRequest {
  string token_jti = 1;
  google.protobuf.Timestamp expires_at = 2;
  string reason = 3;
}

message GetSessionRequest {
  string session_id = 1;
}

message Session {
  string id = 1;
  string user_id = 2;
  string device_id = 3;
  string ip_address = 4;
  string user_agent = 5;
  bool mfa_verified = 6;
  google.protobuf.Timestamp created_at = 7;
```

```
  google.protobuf.Timestamp last_active_at = 8;
  google.protobuf.Timestamp expires_at = 9;
  string status = 10;
}

message InvalidateSessionRequest {
  string session_id = 1;
  string reason = 2;
}

message CheckPermissionRequest {
  string user_id = 1;
  string resource_type = 2;
  string resource_id = 3;
  string action = 4;
  string org_id = 5;
}

message CheckPermissionResponse {
  bool allowed = 1;
  string reason = 2;
}

message BulkCheckPermissionRequest {
  repeated CheckPermissionRequest checks = 1;
}

message BulkCheckPermissionResponse {
  repeated CheckPermissionResponse results = 1;
}
```

---

vault.proto

```
syntax = "proto3";
package helixterm.v1;
option go_package = "helixterminator.io/core/internal/
        proto;proto";

import "google/protobuf/timestamp.proto";
import "google/protobuf/empty.proto";

// VaultService provides internal vault operations for other
        microservices.
service VaultService {
```

```
  // GetVault retrieves vault metadata (not encrypted contents).
  rpc GetVault(GetVaultRequest) returns (Vault);

  // CheckVaultAccess verifies a user's access to a vault.
  rpc CheckVaultAccess(CheckVaultAccessRequest) returns
        (CheckVaultAccessResponse);

  // GetVaultMember retrieves vault membership for a specific
        user.
  rpc GetVaultMember(GetVaultMemberRequest) returns (VaultMember);

  // RecordVaultEvent records an event in vault audit log.
  rpc RecordVaultEvent(RecordVaultEventRequest) returns
        (google.protobuf.Empty);

  // GetVaultSyncState gets the current sync cursor for a vault.
  rpc GetVaultSyncState(GetVaultSyncStateRequest) returns
        (VaultSyncState);

  // UpdateVaultSyncState updates the sync cursor after successful
        sync.
  rpc UpdateVaultSyncState(UpdateVaultSyncStateRequest) returns
        (google.protobuf.Empty);

  // GetEncryptedVaultKey retrieves the vault key blob encrypted
        for a specific user.
  rpc GetEncryptedVaultKey(GetEncryptedVaultKeyRequest) returns
        (GetEncryptedVaultKeyResponse);

  // ListVaultMembers lists all members of a vault.
  rpc ListVaultMembers(ListVaultMembersRequest) returns
        (ListVaultMembersResponse);
}

message GetVaultRequest {
  string vault_id = 1;
}

message Vault {
  string id = 1;
  string name = 2;
  string owner_id = 3;
  string org_id = 4;
  bool sync_enabled = 5;
  bool encrypted = 6;
  google.protobuf.Timestamp created_at = 7;
  google.protobuf.Timestamp updated_at = 8;
```

```
}

message CheckVaultAccessRequest {
  string vault_id = 1;
  string user_id = 2;
  string required_permission = 3;
}

message CheckVaultAccessResponse {
  bool allowed = 1;
  string permission = 2;
  string reason = 3;
}

message GetVaultMemberRequest {
  string vault_id = 1;
  string user_id = 2;
}

message VaultMember {
  string vault_id = 1;
  string user_id = 2;
  string permission = 3;
  bytes encrypted_vault_key = 4;
  google.protobuf.Timestamp joined_at = 5;
}

message RecordVaultEventRequest {
  string vault_id = 1;
  string user_id = 2;
  string event_type = 3;
  string resource_type = 4;
  string resource_id = 5;
  bytes metadata_json = 6;
  string ip_address = 7;
}

message GetVaultSyncStateRequest {
  string vault_id = 1;
  string client_id = 2;
}

message VaultSyncState {
  string vault_id = 1;
```

```
  string client_id = 2;
  string cursor = 3;
  google.protobuf.Timestamp last_synced_at = 4;
  int64 server_version = 5;
}

message UpdateVaultSyncStateRequest {
  string vault_id = 1;
  string client_id = 2;
  string cursor = 3;
  int64 server_version = 4;
}

message GetEncryptedVaultKeyRequest {
  string vault_id = 1;
  string user_id = 2;
}

message GetEncryptedVaultKeyResponse {
  bytes encrypted_key_blob = 1;
  bytes kdf_params_json = 2;
}

message ListVaultMembersRequest {
  string vault_id = 1;
}

message ListVaultMembersResponse {
  repeated VaultMember members = 1;
}
```

---

pki.proto

```
syntax = "proto3";
package helixterm.v1;
option go_package = "helixterminator.io/core/internal/
        proto;proto";

import "google/protobuf/timestamp.proto";
import "google/protobuf/duration.proto";
import "google/protobuf/empty.proto";

// PKIService provides SSH certificate issuance and management.
```

```
// HelixTerminator supports SSH certificate-based authentication
        as an alternative to authorized_keys.
service PKIService {
  // SignUserCertificate signs a user's public key with the CA.
  rpc SignUserCertificate(SignUserCertificateRequest) returns
        (SignUserCertificateResponse);

  // SignHostCertificate signs a host public key with the CA.
  rpc SignHostCertificate(SignHostCertificateRequest) returns
        (SignHostCertificateResponse);

  // GetCACertificate returns the CA public key for trust
        distribution.
  rpc GetCACertificate(GetCACertificateRequest) returns
        (GetCACertificateResponse);

  // RevokeCertificate revokes a previously issued certificate.
  rpc RevokeCertificate(RevokeCertificateRequest) returns
        (google.protobuf.Empty);

  // GetCRL returns the current certificate revocation list.
  rpc GetCRL(GetCRLRequest) returns (GetCRLResponse);

  // CheckCertificate validates whether a certificate is valid and
        not revoked.
  rpc CheckCertificate(CheckCertificateRequest) returns
        (CheckCertificateResponse);

  // ListCertificates lists issued certificates for a user or
        host.
  rpc ListCertificates(ListCertificatesRequest) returns
        (ListCertificatesResponse);

  // RotateCA rotates the CA key (requires admin authority).
  rpc RotateCA(RotateCARequest) returns (RotateCAResponse);
}

message SignUserCertificateRequest {
  string public_key_openssh = 1;
  string user_id = 2;
  string username = 3;
  repeated string principals = 4;
  repeated CertExtension extensions = 5;
  google.protobuf.Duration validity = 6;
  string source_address = 7;
  bool force_command = 8;
  string forced_command = 9;
}
```

```
message SignUserCertificateResponse {
  string certificate_openssh = 1;
  string certificate_id = 2;
  google.protobuf.Timestamp valid_after = 3;
  google.protobuf.Timestamp valid_before = 4;
  string serial = 5;
}

message SignHostCertificateRequest {
  string public_key_openssh = 1;
  string host_id = 2;
  repeated string hostnames = 3;
  google.protobuf.Duration validity = 4;
}

message SignHostCertificateResponse {
  string certificate_openssh = 1;
  string certificate_id = 2;
  google.protobuf.Timestamp valid_before = 3;
  string serial = 4;
}

message CertExtension {
  string name = 1;
  string value = 2;
  bool critical = 3;
}

message GetCACertificateRequest {
  string ca_type = 1; // "user" or "host"
}

message GetCACertificateResponse {
  string public_key_openssh = 1;
  string fingerprint = 2;
  google.protobuf.Timestamp created_at = 3;
  google.protobuf.Timestamp rotated_at = 4;
  string version = 5;
}

message RevokeCertificateRequest {
  string certificate_id = 1;
  string reason = 2;
```

```
  string revoked_by = 3;
}

message GetCRLRequest {
  string ca_type = 1;
}

message GetCRLResponse {
  repeated string revoked_serials = 1;
  string krl_binary_base64 = 2;
  google.protobuf.Timestamp generated_at = 3;
}

message CheckCertificateRequest {
  string certificate_openssh = 1;
}

message CheckCertificateResponse {
  bool valid = 1;
  bool revoked = 2;
  bool expired = 3;
  string certificate_id = 4;
  string reason = 5;
  google.protobuf.Timestamp expires_at = 6;
}

message ListCertificatesRequest {
  string entity_type = 1; // "user" or "host"
  string entity_id = 2;
  bool include_revoked = 3;
  bool include_expired = 4;
}

message Certificate {
  string id = 1;
  string entity_type = 2;
  string entity_id = 3;
  string serial = 4;
  string fingerprint = 5;
  repeated string principals = 6;
  google.protobuf.Timestamp valid_after = 7;
  google.protobuf.Timestamp valid_before = 8;
  bool revoked = 9;
  google.protobuf.Timestamp revoked_at = 10;
```

```
  string revocation_reason = 11;
  google.protobuf.Timestamp created_at = 12;
}

message ListCertificatesResponse {
  repeated Certificate certificates = 1;
  int32 total_count = 2;
}

message RotateCARequest {
  string ca_type = 1;
  string reason = 2;
  string rotated_by = 3;
}

message RotateCAResponse {
  string old_public_key = 1;
  string new_public_key = 2;
  string new_fingerprint = 3;
  google.protobuf.Timestamp rotated_at = 4;
}
```

---

session.proto

```
syntax = "proto3";
package helixterm.v1;
option go_package = "helixterminator.io/core/internal/
        proto;proto";

import "google/protobuf/timestamp.proto";
import "google/protobuf/empty.proto";

// SessionService manages SSH session lifecycle for internal
        service communication.
service SessionService {
  // CreateSession creates a new session record.
  rpc CreateSession(CreateSessionRequest) returns
        (CreateSessionResponse);

  // GetSession retrieves session details.
  rpc GetSession(GetSessionRequest) returns (SSHSession);

  // UpdateSessionStatus updates the status of a session.
  rpc UpdateSessionStatus(UpdateSessionStatusRequest) returns
        (google.protobuf.Empty);
```

```
  // TerminateSession forcibly terminates an active session.
  rpc TerminateSession(TerminateSessionRequest) returns
        (google.protobuf.Empty);

  // ListActiveSessions lists currently active sessions,
        optionally filtered.
  rpc ListActiveSessions(ListActiveSessionsRequest) returns
        (ListActiveSessionsResponse);

  // RecordSessionEvent appends an event to the session event log.
  rpc RecordSessionEvent(RecordSessionEventRequest) returns
        (google.protobuf.Empty);

  // GetSessionStats returns aggregate statistics for a session.
  rpc GetSessionStats(GetSessionStatsRequest) returns
        (SessionStats);

  // FinalizeRecording marks a session recording as complete and
        triggers processing.
  rpc FinalizeRecording(FinalizeRecordingRequest) returns
        (google.protobuf.Empty);

  // BroadcastToSession sends a command to one or more sessions.
  rpc BroadcastToSession(BroadcastRequest) returns
        (BroadcastResponse);

  // GetPortForward retrieves port forwarding rule details.
  rpc GetPortForward(GetPortForwardRequest) returns
        (PortForwardRule);

  // UpdatePortForwardStatus updates the status of a port
        forwarding connection.
  rpc UpdatePortForwardStatus(UpdatePortForwardStatusRequest)
        returns (google.protobuf.Empty);
}

message CreateSessionRequest {
  string user_id = 1;
  string host_id = 2;
  string vault_id = 3;
  string client_ip = 4;
  string user_agent = 5;
  int32 terminal_cols = 6;
  int32 terminal_rows = 7;
  string terminal_type = 8;
  bool recording_enabled = 9;
  bool collab_enabled = 10;
```

```
  bool read_only = 11;
  string reason = 12;
  string ticket_ref = 13;
  string startup_snippet_id = 14;
}

message CreateSessionResponse {
  string session_id = 1;
  string session_token = 2;
  google.protobuf.Timestamp expires_at = 3;
}

message GetSessionRequest {
  string session_id = 1;
}

message SSHSession {
  string id = 1;
  string user_id = 2;
  string host_id = 3;
  string vault_id = 4;
  string status = 5;
  string client_ip = 6;
  int32 terminal_cols = 7;
  int32 terminal_rows = 8;
  bool recording_enabled = 9;
  bool collab_enabled = 10;
  bool read_only = 11;
  string reason = 12;
  string ticket_ref = 13;
  google.protobuf.Timestamp started_at = 14;
  google.protobuf.Timestamp ended_at = 15;
  int64 bytes_sent = 16;
  int64 bytes_received = 17;
}

message UpdateSessionStatusRequest {
  string session_id = 1;
  string status = 2;
  string error_message = 3;
  int32 exit_code = 4;
}

message TerminateSessionRequest {
```

```
  string session_id = 1;
  string terminated_by = 2;
  string reason = 3;
}

message ListActiveSessionsRequest {
  string user_id = 1;
  string host_id = 2;
  string org_id = 3;
}

message ListActiveSessionsResponse {
  repeated SSHSession sessions = 1;
}

message RecordSessionEventRequest {
  string session_id = 1;
  string event_type = 2;
  google.protobuf.Timestamp occurred_at = 3;
  bytes data = 4;
  string direction = 5; // "i" (input) or "o" (output)
}

message GetSessionStatsRequest {
  string session_id = 1;
}

message SessionStats {
  string session_id = 1;
  int64 bytes_sent = 2;
  int64 bytes_received = 3;
  int64 events_count = 4;
  google.protobuf.Timestamp duration = 5;
  int32 resize_count = 6;
}

message FinalizeRecordingRequest {
  string session_id = 1;
  string recording_path = 2;
  int64 file_size_bytes = 3;
}

message BroadcastRequest {
  repeated string session_ids = 1;
```

```
  bytes data = 2;
  bool require_confirmation = 3;
}

message BroadcastResponse {
  int32 sent = 1;
  int32 failed = 2;
  repeated string failed_session_ids = 3;
}

message GetPortForwardRequest {
  string rule_id = 1;
}

message PortForwardRule {
  string id = 1;
  string user_id = 2;
  string host_id = 3;
  string name = 4;
  string type = 5;
  string local_address = 6;
  int32 local_port = 7;
  string remote_address = 8;
  int32 remote_port = 9;
  bool auto_start = 10;
  string status = 11;
}

message UpdatePortForwardStatusRequest {
  string rule_id = 1;
  string connection_id = 2;
  string status = 3;
  int64 bytes_sent = 4;
  int64 bytes_received = 5;
}
```

---

audit.proto

```
syntax = "proto3";
package helixterm.v1;
option go_package = "helixterminator.io/core/internal/
        proto;proto";
```

```protobuf
import "google/protobuf/timestamp.proto";
import "google/protobuf/empty.proto";

// AuditService receives and stores audit events from all
        microservices.
// It maintains a cryptographic hash chain for tamper evidence.
service AuditService {
  // RecordEvent appends an audit event to the immutable log.
  rpc RecordEvent(RecordEventRequest) returns
        (RecordEventResponse);

  // RecordEventBatch appends multiple events in a single RPC
        (efficient bulk logging).
  rpc RecordEventBatch(RecordEventBatchRequest) returns
        (RecordEventBatchResponse);

  // QueryEvents queries the audit log with filters (admin only).
  rpc QueryEvents(QueryEventsRequest) returns
        (QueryEventsResponse);

  // GetEvent retrieves a single audit event.
  rpc GetEvent(GetEventRequest) returns (AuditEvent);

  // VerifyIntegrity verifies the hash chain for a given time
        range.
  rpc VerifyIntegrity(VerifyIntegrityRequest) returns
        (VerifyIntegrityResponse);

  // ExportEvents triggers an async export job.
  rpc ExportEvents(ExportEventsRequest) returns
        (ExportEventsResponse);
}

message RecordEventRequest {
  string event_type = 1;
  string user_id = 2;
  string org_id = 3;
  string resource_type = 4;
  string resource_id = 5;
  string resource_name = 6;
  string outcome = 7;
  string ip_address = 8;
  string user_agent = 9;
  string session_id = 10;
  bytes metadata_json = 11;
  google.protobuf.Timestamp occurred_at = 12;
  string source_service = 13;
```

```
}

message RecordEventResponse {
  string event_id = 1;
  string hash = 2;
  google.protobuf.Timestamp recorded_at = 3;
}

message RecordEventBatchRequest {
  repeated RecordEventRequest events = 1;
}

message RecordEventBatchResponse {
  int32 recorded = 1;
  int32 failed = 2;
  repeated RecordEventResponse results = 3;
}

message QueryEventsRequest {
  string org_id = 1;
  string user_id = 2;
  string resource_type = 3;
  string resource_id = 4;
  string event_type = 5;
  string outcome = 6;
  google.protobuf.Timestamp created_at_gte = 7;
  google.protobuf.Timestamp created_at_lte = 8;
  string cursor = 9;
  int32 limit = 10;
  string sort_direction = 11;
}

message QueryEventsResponse {
  repeated AuditEvent events = 1;
  string next_cursor = 2;
  bool has_next = 3;
  int64 total_count = 4;
}

message GetEventRequest {
  string event_id = 1;
}

message AuditEvent {
```

```
  string id = 1;
  string event_type = 2;
  string user_id = 3;
  string org_id = 4;
  string resource_type = 5;
  string resource_id = 6;
  string resource_name = 7;
  string outcome = 8;
  string ip_address = 9;
  string user_agent = 10;
  string session_id = 11;
  bytes metadata_json = 12;
  string hash = 13;
  string prev_hash = 14;
  string source_service = 15;
  google.protobuf.Timestamp occurred_at = 16;
  google.protobuf.Timestamp recorded_at = 17;
}

message VerifyIntegrityRequest {
  string org_id = 1;
  google.protobuf.Timestamp from = 2;
  google.protobuf.Timestamp to = 3;
}

message VerifyIntegrityResponse {
  bool valid = 1;
  int64 events_checked = 2;
  string first_broken_event_id = 3;
  string error_description = 4;
}

message ExportEventsRequest {
  string org_id = 1;
  string format = 2;
  google.protobuf.Timestamp created_at_gte = 3;
  google.protobuf.Timestamp created_at_lte = 4;
  string requested_by = 5;
}

message ExportEventsResponse {
  string export_id = 1;
  string status = 2;
}
```

---

17. PostgreSQL Database Schemas

Each microservice owns its own PostgreSQL 17.2 database. In production, each runs on a dedicated cluster or schema. In development, all schemas live in a single PostgreSQL instance with separate databases.

> **RESOLVED (this increment):** Multi-tenant isolation is no longer app-layer `WHERE org_id = …` only. > Every multi-tenant table in §17 now carries `ROW LEVEL SECURITY` + a `CREATE POLICY` — see **§17.0** for > the mandatory pattern, the session-variable wiring, and the full per-database policy enumeration. A > missing/incorrect `WHERE` clause in application code is now a defense-in-depth gap, not

a full IDOR: the > database itself refuses cross-tenant rows even if the application forgets the filter. > > **DEFERRED (next increment):** No backup/restore or Point-In-Time-Recovery (PITR) procedure is specified > for any of the per-service PostgreSQL databases below (or for Redis in §18). Authoring RPO/RTO targets > and a concrete backup/PITR runbook is deferred; until then, assume no tested recovery path exists.

Conventions: - All primary keys are `UUID` using `gen_random_uuid()` (pgcrypto / pg 13+). - All timestamps are `TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()`. - Soft delete: `deleted_at TIMESTAMP WITH TIME ZONE` — `NULL` means not

deleted. - Partial indexes on `deleted_at IS NULL` for all soft-delete tables. - All `ENUM`-like status columns use `VARCHAR` with `CHECK` constraints (easier migrations than SQL ENUMs). - `JSONB` for flexible metadata fields with GIN indexes. - `BRIN` indexes on all `created_at` / `occurred_at` for append-heavy tables.

17.0 Row-Level Security (RLS) — Mandatory On Every Multi-Tenant Table

Every table in §17 that carries tenant-scoping data — directly via an `org_id` column, indirectly via a same-database parent (e.g. `vault_id → vaults.org_id`), or per-user via `user_id` where a row is never org-partitioned (a user can belong to multiple orgs, §17.9) — is protected by PostgreSQL **Row Level Security** in addition to (never instead of) the application-layer `WHERE` clause. RLS is defense in depth: a missing/incorrect `WHERE org_id = …` in application code today is a cross-tenant IDOR; after this section, the same bug returns zero rows because the database itself refuses them.

17.0.1 Session-variable convention

Two session variables carry request-scoped tenant/identity context, set **per transaction** (never per-session) to avoid leaking one tenant's context onto the next request when the connection is reused by a pool (PgBouncer transaction-pooling mode, or a Go `pgxpool` connection returned to the pool between requests):

Variable	Set by	Scopes
`app.current_org`	Every request handler that operates within an organization	Tables with `org_id` (direct or via same-DB join)
`app.current_user_id`	Every authenticated request handler	`auth_db` tables that are not org-partitioned (§17.0.4)

Both are read inside policies via `current_setting('app.<name>', true)` — the second argument (`true`, "missing_ok") makes an **unset** variable evaluate to `NULL` instead of raising an error. `NULL = anything` is `NULL` (never `TRUE`) in a `USING`/`WITH CHECK` expression, so a connection that never set the session variable — the exact failure mode of a forgotten application-layer filter, or of code that reached the database outside the mandated wrapper below — sees **zero rows**, not all rows. This is the fail-closed default: RLS degrades a missing tenant context from "cross-tenant leak" to "empty result set."

17.0.2 Connection-checkout wiring (Go / pgx)

`SET LOCAL` only takes effect inside a transaction and automatically resets at `COMMIT`/`ROLLBACK` — this is what makes it safe under transaction-pooling (a session-level bare `SET` would otherwise persist on the pooled physical connection and leak into the next, unrelated request). Every RLS-scoped query MUST run through this wrapper; a `go vet`-based custom analyzer (`internal/dbctx/lint`) flags any direct `pool.Query`/`pool.Exec` call against an RLS-protected database package that bypasses it.

```
// internal/dbctx/tenant.go
package dbctx

import (
    "context"

    "github.com/google/uuid"
    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgxpool"
)

// WithOrgScope runs fn inside a transaction with app.current_org
        set to orgID
```

```
// for the lifetime of that transaction only (SET LOCAL). Commits
        on success,
// rolls back (and therefore discards the session variable) on any
        error.
func WithOrgScope(ctx context.Context, pool *pgxpool.Pool, orgID
        uuid.UUID, fn func(pgx.Tx) error) error {
    tx, err := pool.Begin(ctx)
    if err != nil {
        return err
    }
    defer tx.Rollback(ctx) // no-op once committed

    if _, err := tx.Exec(ctx, "SELECT
        set_config('app.current_org', $1, true)",
        orgID.String()); err != nil {
        return err
    }
    if err := fn(tx); err != nil {
        return err
    }
    return tx.Commit(ctx)
}

// WithUserScope is the self-scoped equivalent for auth_db tables
        (§17.0.4).
func WithUserScope(ctx context.Context, pool
        *pgxpool.Pool, userID uuid.UUID, fn func(pgx.Tx) error)
        error {
    tx, err := pool.Begin(ctx)
    if err != nil {
        return err
    }
    defer tx.Rollback(ctx)

    if _, err := tx.Exec(ctx, "SELECT
        set_config('app.current_user_id', $1, true)",
        userID.String()); err != nil {
        return err
    }
    if err := fn(tx); err != nil {
        return err
    }
    return tx.Commit(ctx)
}
```

`set_config(name, value, is_local=true)` is used instead of raw `SET LOCAL app.current_org = $1` because `SET` does not accept a bind parameter in the pgx simple/extended protocol; `set_config` is a normal SQL function call and accepts one safely (no string-formatting, no injection surface).

17.0.3 `FORCE ROW LEVEL SECURITY` — why it is mandatory, not optional

By default PostgreSQL RLS does **not** apply to the table owner. Application services connect using the same role that owns the table via migrations (e.g. `svc_vault_rw` owns and queries `vault_db`), so without `FORCE ROW LEVEL SECURITY` the policy would be silently bypassed for exactly the connection that matters — `ENABLE ROW LEVEL SECURITY` alone would only protect against a *different*, non-owning role, which is not how these services are deployed. Every `CREATE POLICY` in this section is therefore always paired with:

```
ALTER TABLE <table> ENABLE ROW LEVEL SECURITY;
ALTER TABLE <table> FORCE ROW LEVEL SECURITY;
```

Migrations themselves run under a separate, non-request-serving role (`migrator`) that is never used by request-handling code, so schema changes are unaffected by `FORCE`.

17.0.4 Two scoping modes + the narrow `BYPASSRLS` exception

- **ORG-SCOPED** (`app.current_org`) — the default for every table below that carries `org_id`, directly or via a same-database parent join. Cross-**database** joins do not exist in this database-per-service topology (§17 intro), so tables such as `hosts`, `ssh_keys`, `snippets`, `ssh_sessions` carry a **denormalized** `org_id` column populated at row-creation time from the vault-membership check already performed at the API layer (§4, §5) — RLS treats that denormalized column as authoritative within its own database.
- **SELF-SCOPED** (`app.current_user_id`) — `auth_db`'s user-owned tables (`users`, `user_sessions`, `refresh_tokens`, `device_tokens`, the three `mfa_*` tables, `login_history`, `password_history`, `sso_identities`) have no `org_id` at all, because a user can belong to multiple organizations (`org_db.org_members`, a *different* database) — there is no single tenant to scope by. These are self-only: a row is visible only to the user who owns it.

- **`BYPASSRLS`** — granted to exactly three narrowly-scoped, single-purpose service roles, each audited (every statement they execute is logged to `audit_events` with `source_service` set to the role name), never used by a general request-handling connection pool:

Role	Sole purpose	Grants
`svc_auth_authmw`	JWT-blocklist check on every authenticated request, **before** tenant/user context is known	`SELECT` on `jwt_blocklist` only
`svc_auth_admin_ro`	Cross-user directory reads for `org_admin`/`auditor` RBAC-verified requests (the caller's role is checked in the API handler **before** this connection is ever opened)	`SELECT` on `auth_db` tables only
`svc_audit_writer`	Cross-tenant `INSERT` into `audit_events` from every service (§17.10.1)	`INSERT` on `audit_events` only

  An un-enumerated or un-audited `BYPASSRLS` grant is a release blocker; the enforcement test in §17.0.6 asserts the live role list matches exactly this table.

17.0.5 Worked patterns

Direct (mirrors the `audit_events` pattern, §17.10):

```
ALTER TABLE vaults ENABLE ROW LEVEL SECURITY;
ALTER TABLE vaults FORCE ROW LEVEL SECURITY;

CREATE POLICY vaults_org_isolation ON vaults
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);
```

Indirect, same-database join (a child table with no `org_id` of its own — worked example `vault_items`, scoped through its parent `vaults` row, both tables living in `vault_db`):

```
ALTER TABLE vault_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE vault_items FORCE ROW LEVEL SECURITY;

CREATE POLICY vault_items_org_isolation ON vault_items
  USING (vault_id IN (
    SELECT id FROM vaults WHERE org_id =
        current_setting('app.current_org', true)::uuid
  ))
  WITH CHECK (vault_id IN (
    SELECT id FROM vaults WHERE org_id =
        current_setting('app.current_org', true)::uuid
  ));
```

Self-scoped (worked example `users`; `BYPASSRLS` roles per §17.0.4 read across users when their own independent RBAC check has already passed):

```
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
ALTER TABLE users FORCE ROW LEVEL SECURITY;

CREATE POLICY users_self_only ON users
  USING (id = current_setting('app.current_user_id', true)::uuid)
  WITH CHECK (id = current_setting('app.current_user_id',
        true)::uuid);
```

17.0.6 Enforcement-test approach

Every service ships `internal/db/<service>/rls_test.go`, run as a post-build DB-integration test (§11.4.4(b) layer 3, real PostgreSQL 17.2 in a disposable Testcontainer, never mocked per §11.4.27):

1. **Cross-tenant denial:** connect as the service's own request role (never a superuser), seed one row under org **A** and one under org **B**, call `WithOrgScope(ctx, pool, orgA, …)`, `SELECT` org B's row → assert **0 rows**; `UPDATE`/`DELETE` org B's row → assert **0 rows affected**.
2. **Fail-closed on missing context:** run the same `SELECT` against a transaction that never called `WithOrgScope` at all (the exact shape of a bug that forgot to wire the tenant context) → assert **0 rows**, proving the fail-closed default from §17.0.1, not merely that org B is denied.
3. **Owner-role bypass proof:** run test 1 while connected as the table-owning migration role directly (not through the request role) →

still assert **0 rows** — proves `FORCE ROW LEVEL SECURITY` is actually wired, not merely `ENABLE`.

4. **`BYPASSRLS` roster proof:** `SELECT rolname FROM pg_roles WHERE rolbypassrls` → assert the result set is **exactly** the three roles in §17.0.4's table — no more, no fewer.

Paired §1.1 meta-test mutation: comment out the `FORCE ROW LEVEL SECURITY` line for one table → test 3 must fail; the paired mutation for test 4 grants `BYPASSRLS` to a fourth, unlisted role → the roster assertion must fail. A mutation that does not flip the corresponding test to FAIL is itself a finding (§11.4 anti-bluff covenant — the gate must actually bite).

17.0.7 Full per-database policy enumeration

The pattern above applied to every multi-tenant table across all 11 databases. Partition children inherit `ROW LEVEL SECURITY` from their partitioned parent automatically (PostgreSQL applies the parent's policies to every partition); only the parent table needs `ENABLE`/`FORCE`/`CREATE POLICY`.

```
-- ============================================================
-- auth_db — self-scoped (no org_id; a user spans multiple orgs)
-- ============================================================
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
        ALTER TABLE users FORCE ROW LEVEL SECURITY;
CREATE POLICY users_self_only ON users
  USING (id = current_setting('app.current_user_id', true)::uuid)
  WITH CHECK (id = current_setting('app.current_user_id',
        true)::uuid);

ALTER TABLE user_sessions ENABLE ROW LEVEL SECURITY;
        ALTER TABLE user_sessions FORCE ROW LEVEL SECURITY;
CREATE POLICY user_sessions_self_only ON user_sessions
  USING (user_id = current_setting('app.current_user_id',
        true)::uuid)
  WITH CHECK (user_id = current_setting('app.current_user_id',
        true)::uuid);

ALTER TABLE refresh_tokens ENABLE ROW LEVEL SECURITY;
        ALTER TABLE refresh_tokens FORCE ROW LEVEL SECURITY;
CREATE POLICY refresh_tokens_self_only ON refresh_tokens
  USING (user_id = current_setting('app.current_user_id',
        true)::uuid)
  WITH CHECK (user_id = current_setting('app.current_user_id',
        true)::uuid);
```

```
ALTER TABLE device_tokens ENABLE ROW LEVEL SECURITY;
        ALTER TABLE device_tokens FORCE ROW LEVEL SECURITY;
CREATE POLICY device_tokens_self_only ON device_tokens
  USING (user_id = current_setting('app.current_user_id',
        true)::uuid)
  WITH CHECK (user_id = current_setting('app.current_user_id',
        true)::uuid);

ALTER TABLE mfa_totp_credentials ENABLE ROW LEVEL SECURITY;
        ALTER TABLE mfa_totp_credentials FORCE ROW LEVEL SECURITY;
CREATE POLICY mfa_totp_credentials_self_only ON
        mfa_totp_credentials
  USING (user_id = current_setting('app.current_user_id',
        true)::uuid)
  WITH CHECK (user_id = current_setting('app.current_user_id',
        true)::uuid);

ALTER TABLE mfa_totp_backup_codes ENABLE ROW LEVEL SECURITY;
        ALTER TABLE mfa_totp_backup_codes FORCE ROW LEVEL
        SECURITY;
CREATE POLICY mfa_totp_backup_codes_self_only ON
        mfa_totp_backup_codes
  USING (user_id = current_setting('app.current_user_id',
        true)::uuid)
  WITH CHECK (user_id = current_setting('app.current_user_id',
        true)::uuid);

ALTER TABLE mfa_fido2_credentials ENABLE ROW LEVEL SECURITY;
        ALTER TABLE mfa_fido2_credentials FORCE ROW LEVEL
        SECURITY;
CREATE POLICY mfa_fido2_credentials_self_only ON
        mfa_fido2_credentials
  USING (user_id = current_setting('app.current_user_id',
        true)::uuid)
  WITH CHECK (user_id = current_setting('app.current_user_id',
        true)::uuid);

ALTER TABLE login_history ENABLE ROW LEVEL SECURITY;
        ALTER TABLE login_history FORCE ROW LEVEL SECURITY;
CREATE POLICY login_history_self_only ON login_history
  USING (user_id = current_setting('app.current_user_id',
        true)::uuid)
  WITH CHECK (user_id = current_setting('app.current_user_id',
        true)::uuid);

ALTER TABLE password_history ENABLE ROW LEVEL SECURITY;
        ALTER TABLE password_history FORCE ROW LEVEL SECURITY;
CREATE POLICY password_history_self_only ON password_history
  USING (user_id = current_setting('app.current_user_id',
        true)::uuid)
  WITH CHECK (user_id = current_setting('app.current_user_id',
        true)::uuid);
```

```
ALTER TABLE sso_identities ENABLE ROW LEVEL SECURITY;
        ALTER TABLE sso_identities FORCE ROW LEVEL SECURITY;
CREATE POLICY sso_identities_self_only ON sso_identities
  USING (user_id = current_setting('app.current_user_id',
        true)::uuid)
  WITH CHECK (user_id = current_setting('app.current_user_id',
        true)::uuid);

-- api_keys.org_id is nullable: NULL = a personal key (owner-
        only), NOT NULL = an
-- org-issued key (visible to the org's admin surface for
        revocation/rotation).
ALTER TABLE api_keys ENABLE ROW LEVEL SECURITY;
        ALTER TABLE api_keys FORCE ROW LEVEL SECURITY;
CREATE POLICY api_keys_self_or_org ON api_keys
  USING (
    (org_id IS NULL AND user_id =
        current_setting('app.current_user_id', true)::uuid)
    OR (org_id IS NOT NULL AND org_id =
        current_setting('app.current_org', true)::uuid)
  )
  WITH CHECK (
    (org_id IS NULL AND user_id =
        current_setting('app.current_user_id', true)::uuid)
    OR (org_id IS NOT NULL AND org_id =
        current_setting('app.current_org', true)::uuid)
  );

ALTER TABLE sso_providers ENABLE ROW LEVEL SECURITY;
        ALTER TABLE sso_providers FORCE ROW LEVEL SECURITY;
CREATE POLICY sso_providers_org_isolation ON sso_providers
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

-- jwt_blocklist has no per-user read API; only svc_auth_authmw
        (BYPASSRLS,
-- §17.0.4) reads it, on every authenticated request before tenant
        context
-- exists. RLS is still enabled so a stray non-BYPASSRLS
        connection sees zero
-- rows rather than the full blocklist.
ALTER TABLE jwt_blocklist ENABLE ROW LEVEL SECURITY;
        ALTER TABLE jwt_blocklist FORCE ROW LEVEL SECURITY;
CREATE POLICY jwt_blocklist_self_only ON jwt_blocklist
  USING (user_id = current_setting('app.current_user_id',
        true)::uuid)
  WITH CHECK (user_id = current_setting('app.current_user_id',
        true)::uuid);
```

```
-- ============================================================
-- vault_db — org-scoped (direct on vaults, joined for children)
-- ============================================================
ALTER TABLE vaults ENABLE ROW LEVEL SECURITY;
        ALTER TABLE vaults FORCE ROW LEVEL SECURITY;
CREATE POLICY vaults_org_isolation ON vaults
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE vault_members ENABLE ROW LEVEL SECURITY;
        ALTER TABLE vault_members FORCE ROW LEVEL SECURITY;
CREATE POLICY vault_members_org_isolation ON vault_members
  USING (vault_id IN (SELECT id FROM vaults WHERE org_id =
        current_setting('app.current_org', true)::uuid))
  WITH CHECK (vault_id IN (SELECT id FROM vaults WHERE org_id =
        current_setting('app.current_org', true)::uuid));

ALTER TABLE vault_items ENABLE ROW LEVEL SECURITY;
        ALTER TABLE vault_items FORCE ROW LEVEL SECURITY;
CREATE POLICY vault_items_org_isolation ON vault_items
  USING (vault_id IN (SELECT id FROM vaults WHERE org_id =
        current_setting('app.current_org', true)::uuid))
  WITH CHECK (vault_id IN (SELECT id FROM vaults WHERE org_id =
        current_setting('app.current_org', true)::uuid));

ALTER TABLE vault_item_versions ENABLE ROW LEVEL SECURITY;
        ALTER TABLE vault_item_versions FORCE ROW LEVEL SECURITY;
CREATE POLICY vault_item_versions_org_isolation ON
        vault_item_versions
  USING (vault_id IN (SELECT id FROM vaults WHERE org_id =
        current_setting('app.current_org', true)::uuid))
  WITH CHECK (vault_id IN (SELECT id FROM vaults WHERE org_id =
        current_setting('app.current_org', true)::uuid));

ALTER TABLE vault_sync_states ENABLE ROW LEVEL SECURITY;
        ALTER TABLE vault_sync_states FORCE ROW LEVEL SECURITY;
CREATE POLICY vault_sync_states_org_isolation ON vault_sync_states
  USING (vault_id IN (SELECT id FROM vaults WHERE org_id =
        current_setting('app.current_org', true)::uuid))
  WITH CHECK (vault_id IN (SELECT id FROM vaults WHERE org_id =
        current_setting('app.current_org', true)::uuid));

ALTER TABLE vault_audit_events ENABLE ROW LEVEL SECURITY;
        ALTER TABLE vault_audit_events FORCE ROW LEVEL SECURITY;
CREATE POLICY vault_audit_events_org_isolation ON
        vault_audit_events
  USING (vault_id IN (SELECT id FROM vaults WHERE org_id =
        current_setting('app.current_org', true)::uuid))
  WITH CHECK (vault_id IN (SELECT id FROM vaults WHERE org_id =
        current_setting('app.current_org', true)::uuid));
```

```
-- ============================================================
-- host_db — org-scoped (denormalized org_id direct on parents,
-- joined through hosts/host_groups for children)
-- ============================================================
ALTER TABLE hosts ENABLE ROW LEVEL SECURITY;
        ALTER TABLE hosts FORCE ROW LEVEL SECURITY;
CREATE POLICY hosts_org_isolation ON hosts
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE host_groups ENABLE ROW LEVEL SECURITY;
        ALTER TABLE host_groups FORCE ROW LEVEL SECURITY;
CREATE POLICY host_groups_org_isolation ON host_groups
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE host_group_members ENABLE ROW LEVEL SECURITY;
        ALTER TABLE host_group_members FORCE ROW LEVEL SECURITY;
CREATE POLICY host_group_members_org_isolation ON
        host_group_members
  USING (host_id IN (SELECT id FROM hosts WHERE org_id =
        current_setting('app.current_org', true)::uuid))
  WITH CHECK (host_id IN (SELECT id FROM hosts WHERE org_id =
        current_setting('app.current_org', true)::uuid));

ALTER TABLE host_labels ENABLE ROW LEVEL SECURITY;
        ALTER TABLE host_labels FORCE ROW LEVEL SECURITY;
CREATE POLICY host_labels_org_isolation ON host_labels
  USING (host_id IN (SELECT id FROM hosts WHERE org_id =
        current_setting('app.current_org', true)::uuid))
  WITH CHECK (host_id IN (SELECT id FROM hosts WHERE org_id =
        current_setting('app.current_org', true)::uuid));

ALTER TABLE host_known_fingerprints ENABLE ROW LEVEL SECURITY;
        ALTER TABLE host_known_fingerprints FORCE ROW LEVEL
        SECURITY;
CREATE POLICY host_known_fingerprints_org_isolation ON
        host_known_fingerprints
  USING (host_id IN (SELECT id FROM hosts WHERE org_id =
        current_setting('app.current_org', true)::uuid))
  WITH CHECK (host_id IN (SELECT id FROM hosts WHERE org_id =
        current_setting('app.current_org', true)::uuid));

ALTER TABLE host_connection_history ENABLE ROW LEVEL SECURITY;
        ALTER TABLE host_connection_history FORCE ROW LEVEL
        SECURITY;
CREATE POLICY host_connection_history_org_isolation ON
        host_connection_history
```

```
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE jump_host_chains ENABLE ROW LEVEL SECURITY;
        ALTER TABLE jump_host_chains FORCE ROW LEVEL SECURITY;
CREATE POLICY jump_host_chains_org_isolation ON jump_host_chains
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);


-- ============================================================
-- keychain_db — org-scoped (direct on ssh_keys, joined for
        children)
-- ============================================================
ALTER TABLE ssh_keys ENABLE ROW LEVEL SECURITY;
        ALTER TABLE ssh_keys FORCE ROW LEVEL SECURITY;
CREATE POLICY ssh_keys_org_isolation ON ssh_keys
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE key_deployments ENABLE ROW LEVEL SECURITY;
        ALTER TABLE key_deployments FORCE ROW LEVEL SECURITY;
CREATE POLICY key_deployments_org_isolation ON key_deployments
  USING (key_id IN (SELECT id FROM ssh_keys WHERE org_id =
        current_setting('app.current_org', true)::uuid))
  WITH CHECK (key_id IN (SELECT id FROM ssh_keys WHERE org_id =
        current_setting('app.current_org', true)::uuid));


-- key_usage_log.key_id / certificate_store.key_id are nullable-
        by-reference in
-- practice (a key can be deleted while usage history / certs are
        retained for
-- audit) — the join predicate is written so a since-deleted key
        still resolves
-- via ssh_keys' own soft-delete (deleted_at), never silently
        exposing rows
-- whose parent key no longer resolves to any org.
ALTER TABLE key_usage_log ENABLE ROW LEVEL SECURITY;
        ALTER TABLE key_usage_log FORCE ROW LEVEL SECURITY;
CREATE POLICY key_usage_log_org_isolation ON key_usage_log
  USING (key_id IN (SELECT id FROM ssh_keys WHERE org_id =
        current_setting('app.current_org', true)::uuid))
  WITH CHECK (key_id IN (SELECT id FROM ssh_keys WHERE org_id =
        current_setting('app.current_org', true)::uuid));

ALTER TABLE certificate_store ENABLE ROW LEVEL SECURITY;
        ALTER TABLE certificate_store FORCE ROW LEVEL SECURITY;
CREATE POLICY certificate_store_org_isolation ON certificate_store
```

```
  USING (key_id IN (SELECT id FROM ssh_keys WHERE org_id =
        current_setting('app.current_org', true)::uuid))
  WITH CHECK (key_id IN (SELECT id FROM ssh_keys WHERE org_id =
        current_setting('app.current_org', true)::uuid));

-- ============================================================
-- session_db — org-scoped (direct on session-root tables, joined
-- for their per-event/per-transfer/per-connection children)
-- ============================================================
ALTER TABLE ssh_sessions ENABLE ROW LEVEL SECURITY;
        ALTER TABLE ssh_sessions FORCE ROW LEVEL SECURITY;
CREATE POLICY ssh_sessions_org_isolation ON ssh_sessions
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE session_events ENABLE ROW LEVEL SECURITY;
        ALTER TABLE session_events FORCE ROW LEVEL SECURITY;
CREATE POLICY session_events_org_isolation ON session_events
  USING (session_id IN (SELECT id FROM ssh_sessions WHERE org_id
        = current_setting('app.current_org', true)::uuid))
  WITH CHECK (session_id IN (SELECT id FROM ssh_sessions WHERE
        org_id = current_setting('app.current_org', true)::uuid));

ALTER TABLE session_recordings ENABLE ROW LEVEL SECURITY;
        ALTER TABLE session_recordings FORCE ROW LEVEL SECURITY;
CREATE POLICY session_recordings_org_isolation ON
        session_recordings
  USING (session_id IN (SELECT id FROM ssh_sessions WHERE org_id
        = current_setting('app.current_org', true)::uuid))
  WITH CHECK (session_id IN (SELECT id FROM ssh_sessions WHERE
        org_id = current_setting('app.current_org', true)::uuid));

ALTER TABLE sftp_sessions ENABLE ROW LEVEL SECURITY;
        ALTER TABLE sftp_sessions FORCE ROW LEVEL SECURITY;
CREATE POLICY sftp_sessions_org_isolation ON sftp_sessions
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE sftp_transfers ENABLE ROW LEVEL SECURITY;
        ALTER TABLE sftp_transfers FORCE ROW LEVEL SECURITY;
CREATE POLICY sftp_transfers_org_isolation ON sftp_transfers
  USING (sftp_session_id IN (SELECT id FROM sftp_sessions WHERE
        org_id = current_setting('app.current_org', true)::uuid))
  WITH CHECK (sftp_session_id IN (SELECT id FROM sftp_sessions
        WHERE org_id = current_setting('app.current_org',
        true)::uuid));

ALTER TABLE port_forward_rules ENABLE ROW LEVEL SECURITY;
        ALTER TABLE port_forward_rules FORCE ROW LEVEL SECURITY;
```

```
CREATE POLICY port_forward_rules_org_isolation ON
        port_forward_rules
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE port_forward_connections ENABLE ROW LEVEL SECURITY;
        ALTER TABLE port_forward_connections FORCE ROW LEVEL
        SECURITY;
CREATE POLICY port_forward_connections_org_isolation ON
        port_forward_connections
  USING (rule_id IN (SELECT id FROM port_forward_rules WHERE
        org_id = current_setting('app.current_org', true)::uuid))
  WITH CHECK (rule_id IN (SELECT id FROM port_forward_rules WHERE
        org_id = current_setting('app.current_org', true)::uuid));


-- ============================================================
-- snippet_db — org-scoped (direct on all three root tables)
-- ============================================================
ALTER TABLE snippet_categories ENABLE ROW LEVEL SECURITY;
        ALTER TABLE snippet_categories FORCE ROW LEVEL SECURITY;
CREATE POLICY snippet_categories_org_isolation ON
        snippet_categories
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE snippets ENABLE ROW LEVEL SECURITY;
        ALTER TABLE snippets FORCE ROW LEVEL SECURITY;
CREATE POLICY snippets_org_isolation ON snippets
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE snippet_executions ENABLE ROW LEVEL SECURITY;
        ALTER TABLE snippet_executions FORCE ROW LEVEL SECURITY;
CREATE POLICY snippet_executions_org_isolation ON
        snippet_executions
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE snippet_execution_results ENABLE ROW LEVEL SECURITY;
        ALTER TABLE snippet_execution_results FORCE ROW LEVEL
        SECURITY;
CREATE POLICY snippet_execution_results_org_isolation ON
        snippet_execution_results
  USING (execution_id IN (SELECT id FROM snippet_executions WHERE
        org_id = current_setting('app.current_org', true)::uuid))
```

```
  WITH CHECK (execution_id IN (SELECT id FROM snippet_executions
        WHERE org_id = current_setting('app.current_org',
        true)::uuid));


-- ============================================================
-- workspace_db — org-scoped, with a nullable-org "public
        template"
-- carve-out on workspace_templates
-- ============================================================
ALTER TABLE workspaces ENABLE ROW LEVEL SECURITY;
        ALTER TABLE workspaces FORCE ROW LEVEL SECURITY;
CREATE POLICY workspaces_org_isolation ON workspaces
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE workspace_snapshots ENABLE ROW LEVEL SECURITY;
        ALTER TABLE workspace_snapshots FORCE ROW LEVEL SECURITY;
CREATE POLICY workspace_snapshots_org_isolation ON
        workspace_snapshots
  USING (workspace_id IN (SELECT id FROM workspaces WHERE org_id
        = current_setting('app.current_org', true)::uuid))
  WITH CHECK (workspace_id IN (SELECT id FROM workspaces WHERE
        org_id = current_setting('app.current_org', true)::uuid));

ALTER TABLE workspace_sessions ENABLE ROW LEVEL SECURITY;
        ALTER TABLE workspace_sessions FORCE ROW LEVEL SECURITY;
CREATE POLICY workspace_sessions_org_isolation ON
        workspace_sessions
  USING (workspace_id IN (SELECT id FROM workspaces WHERE org_id
        = current_setting('app.current_org', true)::uuid))
  WITH CHECK (workspace_id IN (SELECT id FROM workspaces WHERE
        org_id = current_setting('app.current_org', true)::uuid));

-- workspace_templates.org_id is nullable: NULL + public = TRUE is
        a
-- platform-wide template (visible to every org); NULL + public =
        FALSE never
-- happens in practice (enforced by a CHECK, not by RLS) but the
        policy is
-- written defensively regardless.
ALTER TABLE workspace_templates ENABLE ROW LEVEL SECURITY;
        ALTER TABLE workspace_templates FORCE ROW LEVEL SECURITY;
CREATE POLICY workspace_templates_org_or_public ON
        workspace_templates
  USING (
    org_id = current_setting('app.current_org', true)::uuid
    OR (org_id IS NULL AND public = TRUE)
  )
  WITH CHECK (
    org_id = current_setting('app.current_org', true)::uuid
```

```
    OR (org_id IS NULL AND public = TRUE)
  );

-- ============================================================
-- collab_db — org-scoped (direct on collaboration_sessions, joined
-- for participants/events)
-- ============================================================
ALTER TABLE collaboration_sessions ENABLE ROW LEVEL SECURITY;
        ALTER TABLE collaboration_sessions FORCE ROW LEVEL SECURITY;
CREATE POLICY collaboration_sessions_org_isolation ON collaboration_sessions
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org', true)::uuid);

ALTER TABLE collaboration_participants ENABLE ROW LEVEL SECURITY;
        ALTER TABLE collaboration_participants FORCE ROW LEVEL SECURITY;
CREATE POLICY collaboration_participants_org_isolation ON collaboration_participants
  USING (collab_id IN (SELECT id FROM collaboration_sessions
        WHERE org_id = current_setting('app.current_org', true)::uuid))
  WITH CHECK (collab_id IN (SELECT id FROM collaboration_sessions
        WHERE org_id = current_setting('app.current_org', true)::uuid));

ALTER TABLE collaboration_events ENABLE ROW LEVEL SECURITY;
        ALTER TABLE collaboration_events FORCE ROW LEVEL SECURITY;
CREATE POLICY collaboration_events_org_isolation ON collaboration_events
  USING (collab_id IN (SELECT id FROM collaboration_sessions
        WHERE org_id = current_setting('app.current_org', true)::uuid))
  WITH CHECK (collab_id IN (SELECT id FROM collaboration_sessions
        WHERE org_id = current_setting('app.current_org', true)::uuid));


-- ============================================================
-- org_db — the organization row IS the tenant (id, not org_id);
-- every other table here already carries org_id directly
-- ============================================================
ALTER TABLE organizations ENABLE ROW LEVEL SECURITY;
        ALTER TABLE organizations FORCE ROW LEVEL SECURITY;
CREATE POLICY organizations_is_current_org ON organizations
  USING (id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (id = current_setting('app.current_org', true)::uuid);
```

```sql
ALTER TABLE org_members ENABLE ROW LEVEL SECURITY;
        ALTER TABLE org_members FORCE ROW LEVEL SECURITY;
CREATE POLICY org_members_org_isolation ON org_members
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE teams ENABLE ROW LEVEL SECURITY;
        ALTER TABLE teams FORCE ROW LEVEL SECURITY;
CREATE POLICY teams_org_isolation ON teams
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE team_members ENABLE ROW LEVEL SECURITY;
        ALTER TABLE team_members FORCE ROW LEVEL SECURITY;
CREATE POLICY team_members_org_isolation ON team_members
  USING (team_id IN (SELECT id FROM teams WHERE org_id =
        current_setting('app.current_org', true)::uuid))
  WITH CHECK (team_id IN (SELECT id FROM teams WHERE org_id =
        current_setting('app.current_org', true)::uuid));

ALTER TABLE roles ENABLE ROW LEVEL SECURITY;
        ALTER TABLE roles FORCE ROW LEVEL SECURITY;
CREATE POLICY roles_org_isolation ON roles
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE role_assignments ENABLE ROW LEVEL SECURITY;
        ALTER TABLE role_assignments FORCE ROW LEVEL SECURITY;
CREATE POLICY role_assignments_org_isolation ON role_assignments
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE invitations ENABLE ROW LEVEL SECURITY;
        ALTER TABLE invitations FORCE ROW LEVEL SECURITY;
CREATE POLICY invitations_org_isolation ON invitations
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

-- ============================================================
-- audit_db — direct on all four tables (incl. audit_pii_keys,
-- §17.10.1); see §17.10.1 for the privileged svc_audit_writer
-- BYPASSRLS write path
-- ============================================================
```

```
ALTER TABLE audit_events ENABLE ROW LEVEL SECURITY;
        ALTER TABLE audit_events FORCE ROW LEVEL SECURITY;
CREATE POLICY audit_events_org_isolation ON audit_events
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE audit_pii_keys ENABLE ROW LEVEL SECURITY;
        ALTER TABLE audit_pii_keys FORCE ROW LEVEL SECURITY;
CREATE POLICY audit_pii_keys_org_isolation ON audit_pii_keys
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE audit_event_hash_chain ENABLE ROW LEVEL SECURITY;
        ALTER TABLE audit_event_hash_chain FORCE ROW LEVEL
        SECURITY;
CREATE POLICY audit_event_hash_chain_org_isolation ON
        audit_event_hash_chain
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE audit_exports ENABLE ROW LEVEL SECURITY;
        ALTER TABLE audit_exports FORCE ROW LEVEL SECURITY;
CREATE POLICY audit_exports_org_isolation ON audit_exports
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);


-- ============================================================
-- pki_db — direct on all four tables
-- ============================================================
ALTER TABLE ca_keys ENABLE ROW LEVEL SECURITY;
        ALTER TABLE ca_keys FORCE ROW LEVEL SECURITY;
CREATE POLICY ca_keys_org_isolation ON ca_keys
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE certificates ENABLE ROW LEVEL SECURITY;
        ALTER TABLE certificates FORCE ROW LEVEL SECURITY;
CREATE POLICY certificates_org_isolation ON certificates
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE certificate_revocations ENABLE ROW LEVEL SECURITY;
        ALTER TABLE certificate_revocations FORCE ROW LEVEL
        SECURITY;
```

```
CREATE POLICY certificate_revocations_org_isolation ON
        certificate_revocations
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);

ALTER TABLE crl_entries ENABLE ROW LEVEL SECURITY;
        ALTER TABLE crl_entries FORCE ROW LEVEL SECURITY;
CREATE POLICY crl_entries_org_isolation ON crl_entries
  USING (org_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (org_id = current_setting('app.current_org',
        true)::uuid);
```

17.1 auth_db

```
-- Enable required extensions
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";
CREATE EXTENSION IF NOT EXISTS "citext";

-- ============================================================
-- TABLE: users
-- Core user identity. Owns authentication credentials.
-- ============================================================
CREATE TABLE users (
  id                        UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
  email                     CITEXT NOT NULL,
  email_verified_at         TIMESTAMP WITH TIME ZONE,
  email_pending             CITEXT,
  email_pending_token       VARCHAR(255),
  email_pending_expires_at  TIMESTAMP WITH TIME ZONE,
  password_hash             VARCHAR(255),
  display_name              VARCHAR(100) NOT NULL,
  avatar_url                TEXT,
  bio                       TEXT,
  locale                    VARCHAR(20) NOT NULL DEFAULT 'en-US',
  timezone                  VARCHAR(100) NOT NULL DEFAULT 'UTC',
  status                    VARCHAR(20) NOT NULL DEFAULT 'active'
                              CHECK (status IN ('active',
        'suspended', 'pending_deletion', 'deleted')),
  failed_login_attempts     INTEGER NOT NULL DEFAULT 0,
  locked_until              TIMESTAMP WITH TIME ZONE,
  last_login_at             TIMESTAMP WITH TIME ZONE,
  last_login_ip             INET,
```

```
  password_changed_at       TIMESTAMP WITH TIME ZONE,
  terms_accepted_at         TIMESTAMP WITH TIME ZONE,
  terms_version             VARCHAR(20),
  deletion_requested_at     TIMESTAMP WITH TIME ZONE,
  deletion_scheduled_at     TIMESTAMP WITH TIME ZONE,
  created_at                TIMESTAMP WITH TIME ZONE NOT NULL
        DEFAULT NOW(),
  updated_at                TIMESTAMP WITH TIME ZONE NOT NULL
        DEFAULT NOW(),
  deleted_at                TIMESTAMP WITH TIME ZONE
);

CREATE UNIQUE INDEX idx_users_email ON users(email) WHERE
        deleted_at IS NULL;
CREATE INDEX idx_users_email_pending ON users(email_pending)
        WHERE email_pending IS NOT NULL;
CREATE INDEX idx_users_status ON users(status) WHERE deleted_at
        IS NULL;
CREATE INDEX idx_users_created_at ON users USING BRIN
        (created_at);
CREATE INDEX idx_users_deletion_scheduled ON
        users(deletion_scheduled_at)
  WHERE deletion_scheduled_at IS NOT NULL;

-- Trigger: update updated_at on row modification
CREATE OR REPLACE FUNCTION trigger_set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_users_updated_at
  BEFORE UPDATE ON users
  FOR EACH ROW EXECUTE FUNCTION trigger_set_updated_at();



-- ============================================================
-- TABLE: user_sessions
-- Active authentication sessions (login sessions).
-- ============================================================
CREATE TABLE user_sessions (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
  device_id         UUID,
```

```
  ip_address        INET NOT NULL,
  user_agent        TEXT,
  location_city     VARCHAR(100),
  location_country  VARCHAR(10),
  mfa_verified      BOOLEAN NOT NULL DEFAULT FALSE,
  mfa_method        VARCHAR(20) CHECK (mfa_method IN ('totp',
        'fido2', 'backup_code', NULL)),
  status            VARCHAR(20) NOT NULL DEFAULT 'active'
                      CHECK (status IN ('active', 'expired',
        'revoked')),
  revoked_reason    VARCHAR(100),
  last_active_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  created_at        TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  expires_at        TIMESTAMP WITH TIME ZONE NOT NULL
);

CREATE INDEX idx_user_sessions_user_id ON user_sessions(user_id);
CREATE INDEX idx_user_sessions_status ON user_sessions(status,
        expires_at)
  WHERE status = 'active';
CREATE INDEX idx_user_sessions_expires_at ON user_sessions USING
        BRIN (expires_at);



-- ============================================================
-- TABLE: refresh_tokens
-- Refresh token storage for token rotation.
-- ============================================================
CREATE TABLE refresh_tokens (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id      UUID NOT NULL REFERENCES user_sessions(id) ON
        DELETE CASCADE,
  user_id         UUID NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
  token_hash      VARCHAR(255) NOT NULL UNIQUE,
  family          UUID NOT NULL,
  generation      INTEGER NOT NULL DEFAULT 1,
  ip_address      INET,
  user_agent      TEXT,
  used_at         TIMESTAMP WITH TIME ZONE,
  revoked         BOOLEAN NOT NULL DEFAULT FALSE,
  revoked_at      TIMESTAMP WITH TIME ZONE,
  revoked_reason  VARCHAR(100),
  created_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  expires_at      TIMESTAMP WITH TIME ZONE NOT NULL
```

```
);

CREATE INDEX idx_refresh_tokens_token_hash ON
        refresh_tokens(token_hash);
CREATE INDEX idx_refresh_tokens_session_id ON
        refresh_tokens(session_id);
CREATE INDEX idx_refresh_tokens_family ON refresh_tokens(family);
CREATE INDEX idx_refresh_tokens_expires_at ON refresh_tokens
        USING BRIN (expires_at);



-- ============================================================
-- TABLE: device_tokens
-- Trusted device registrations.
-- ============================================================
CREATE TABLE device_tokens (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id          UUID NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
  name             VARCHAR(255) NOT NULL,
  fingerprint      VARCHAR(512) NOT NULL,
  platform         VARCHAR(255),
  user_agent       TEXT,
  trusted          BOOLEAN NOT NULL DEFAULT TRUE,
  last_seen_at     TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  last_seen_ip     INET,
  revoked          BOOLEAN NOT NULL DEFAULT FALSE,
  revoked_at       TIMESTAMP WITH TIME ZONE,
  created_at       TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE UNIQUE INDEX idx_device_tokens_user_fingerprint ON
        device_tokens(user_id, fingerprint)
  WHERE revoked = FALSE;
CREATE INDEX idx_device_tokens_user_id ON device_tokens(user_id);



-- ============================================================
-- TABLE: mfa_totp_credentials
-- TOTP (authenticator app) MFA credentials.
-- ============================================================
CREATE TABLE mfa_totp_credentials (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id          UUID NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
  encrypted_secret TEXT NOT NULL,
```

```
  issuer           VARCHAR(100) NOT NULL DEFAULT
        'HelixTerminator',
  algorithm        VARCHAR(20) NOT NULL DEFAULT 'SHA1',
  digits           INTEGER NOT NULL DEFAULT 6,
  period           INTEGER NOT NULL DEFAULT 30,
  enabled          BOOLEAN NOT NULL DEFAULT TRUE,
  last_used_at     TIMESTAMP WITH TIME ZONE,
  last_used_code   VARCHAR(10),
  created_at       TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  updated_at       TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE UNIQUE INDEX idx_mfa_totp_user_enabled ON
        mfa_totp_credentials(user_id)
  WHERE enabled = TRUE;



-- ============================================================
-- TABLE: mfa_totp_backup_codes
-- One-time backup codes for TOTP recovery.
-- ============================================================
CREATE TABLE mfa_totp_backup_codes (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id      UUID NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
  code_hash    VARCHAR(255) NOT NULL,
  used         BOOLEAN NOT NULL DEFAULT FALSE,
  used_at      TIMESTAMP WITH TIME ZONE,
  used_ip      INET,
  created_at   TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_backup_codes_user_id ON
        mfa_totp_backup_codes(user_id)
  WHERE used = FALSE;



-- ============================================================
-- TABLE: mfa_fido2_credentials
-- WebAuthn/FIDO2 credential registrations.
-- ============================================================
CREATE TABLE mfa_fido2_credentials (
  id                   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id              UUID NOT NULL REFERENCES users(id) ON
        DELETE CASCADE,
```

```
  credential_id        BYTEA NOT NULL UNIQUE,
  credential_id_b64    TEXT NOT NULL,
  name                 VARCHAR(255) NOT NULL,
  public_key_cbor      BYTEA NOT NULL,
  aaguid               UUID,
  sign_count           BIGINT NOT NULL DEFAULT 0,
  transports           TEXT[] DEFAULT '{}',
  backup_eligible      BOOLEAN NOT NULL DEFAULT FALSE,
  backup_state         BOOLEAN NOT NULL DEFAULT FALSE,
  attestation_type     VARCHAR(50),
  attestation_data     JSONB,
  last_used_at         TIMESTAMP WITH TIME ZONE,
  enabled              BOOLEAN NOT NULL DEFAULT TRUE,
  created_at           TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  updated_at           TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW()
);

CREATE INDEX idx_fido2_user_id ON mfa_fido2_credentials(user_id)
        WHERE enabled = TRUE;
CREATE INDEX idx_fido2_credential_id ON
        mfa_fido2_credentials(credential_id_b64);



-- ============================================================
-- TABLE: api_keys
-- API key management.
-- ============================================================
CREATE TABLE api_keys (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
  org_id            UUID,
  name              VARCHAR(100) NOT NULL,
  description       TEXT,
  key_hash          VARCHAR(255) NOT NULL UNIQUE,
  key_prefix        VARCHAR(20) NOT NULL,
  scopes            TEXT[] NOT NULL DEFAULT '{}',
  allowed_ips       CIDR[] DEFAULT '{}',
  last_used_at      TIMESTAMP WITH TIME ZONE,
  last_used_ip      INET,
  revoked           BOOLEAN NOT NULL DEFAULT FALSE,
  revoked_at        TIMESTAMP WITH TIME ZONE,
  revoked_reason    VARCHAR(255),
  expires_at        TIMESTAMP WITH TIME ZONE,
```

```
  created_at       TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  updated_at       TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_api_keys_user_id ON api_keys(user_id) WHERE
        revoked = FALSE;
CREATE INDEX idx_api_keys_key_hash ON api_keys(key_hash);



-- ============================================================
-- TABLE: login_history
-- Immutable login event log.
-- ============================================================
CREATE TABLE login_history (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
  event_type        VARCHAR(50) NOT NULL
                      CHECK (event_type IN (
                        'login_success', 'login_failure',
        'logout',
                        'mfa_success', 'mfa_failure',
        'token_refresh',
                        'api_key_used', 'sso_login',
        'password_reset'
                      )),
  ip_address        INET,
  user_agent        TEXT,
  device_id         UUID,
  session_id        UUID,
  failure_reason    VARCHAR(255),
  metadata          JSONB DEFAULT '{}',
  occurred_at       TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW()
);

CREATE INDEX idx_login_history_user_id ON login_history(user_id);
CREATE INDEX idx_login_history_occurred_at ON login_history USING
        BRIN (occurred_at);
CREATE INDEX idx_login_history_event_type ON
        login_history(event_type, occurred_at DESC);



-- ============================================================
-- TABLE: password_history
-- Stores hashes of previous passwords (prevents reuse).
```

```sql
-- ============================================================
CREATE TABLE password_history (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id       UUID NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
  password_hash VARCHAR(255) NOT NULL,
  created_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_password_history_user_id ON
        password_history(user_id);



-- ============================================================
-- TABLE: sso_providers
-- Configured SSO provider integrations per organization.
-- ============================================================
CREATE TABLE sso_providers (
  id                    UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
  org_id                UUID NOT NULL,
  provider              VARCHAR(50) NOT NULL
                          CHECK (provider IN ('github', 'google',
        'azure', 'okta', 'saml', 'oidc')),
  slug                  VARCHAR(100) NOT NULL,
  display_name          VARCHAR(255),
  client_id             VARCHAR(512),
  encrypted_client_secret TEXT,
  discovery_url         TEXT,
  authorization_url     TEXT,
  token_url             TEXT,
  userinfo_url          TEXT,
  jwks_uri              TEXT,
  scopes                TEXT[] DEFAULT ARRAY['openid', 'email',
        'profile'],
  attribute_mapping     JSONB DEFAULT '{}',
  enabled               BOOLEAN NOT NULL DEFAULT TRUE,
  enforce_for_domain    VARCHAR(255),
  created_at            TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  updated_at            TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW()
);

CREATE UNIQUE INDEX idx_sso_providers_org_provider ON
        sso_providers(org_id, provider)
  WHERE enabled = TRUE;
```

```
CREATE INDEX idx_sso_providers_slug ON sso_providers(slug);


-- ============================================================
-- TABLE: sso_identities
-- Links a local user to a remote SSO identity.
-- ============================================================
CREATE TABLE sso_identities (
  id                      UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
  user_id                 UUID NOT NULL REFERENCES users(id) ON
        DELETE CASCADE,
  provider_id             UUID NOT NULL REFERENCES
        sso_providers(id) ON DELETE CASCADE,
  subject                 VARCHAR(512) NOT NULL,
  -- Envelope-encrypted IdP OAuth tokens (never plaintext at
        rest). Encrypted
  -- with pgcrypto pgp_sym_encrypt() using the org's KEK-wrapped
        DEK (same

        -- envelope-encryption model as §17.10.1's audit PII, §4's
        vault keys); the
  -- application decrypts only inside the request that needs to
        call the IdP
  -- (token refresh, SCIM sync), never returns these bytes to any
        client API.
  encrypted_access_token  BYTEA,
  encrypted_refresh_token BYTEA,
  token_key_id            UUID,
  token_expires_at        TIMESTAMP WITH TIME ZONE,
  profile_data            JSONB DEFAULT '{}',
  created_at              TIMESTAMP WITH TIME ZONE NOT NULL
        DEFAULT NOW(),
  updated_at              TIMESTAMP WITH TIME ZONE NOT NULL
        DEFAULT NOW()
);

CREATE UNIQUE INDEX idx_sso_identities_provider_subject ON
        sso_identities(provider_id, subject);
CREATE INDEX idx_sso_identities_user_id ON
        sso_identities(user_id);


-- ============================================================
-- TABLE: jwt_blocklist
-- Blocklisted JTIs (revoked tokens before expiry).
-- ============================================================
CREATE TABLE jwt_blocklist (
  jti         VARCHAR(255) PRIMARY KEY,
```

```
  user_id     UUID NOT NULL,
  revoked_at  TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  expires_at  TIMESTAMP WITH TIME ZONE NOT NULL,
  reason      VARCHAR(100)
);

CREATE INDEX idx_jwt_blocklist_expires_at ON jwt_blocklist USING
        BRIN (expires_at);
-- Periodically delete expired entries: DELETE FROM jwt_blocklist
        WHERE expires_at < NOW();
```

17.2 vault_db

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";

-- ============================================================
-- TABLE: vaults
-- Vault containers (E2E encrypted).
-- ============================================================
CREATE TABLE vaults (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id          UUID NOT NULL,
  owner_id        UUID NOT NULL,
  name            VARCHAR(255) NOT NULL,
  description     TEXT,
  color           VARCHAR(20),
  icon            VARCHAR(50),
  encrypted       BOOLEAN NOT NULL DEFAULT TRUE,
  sync_enabled    BOOLEAN NOT NULL DEFAULT TRUE,
  item_count      INTEGER NOT NULL DEFAULT 0,
  storage_bytes   BIGINT NOT NULL DEFAULT 0,
  version         BIGINT NOT NULL DEFAULT 1,
  created_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  updated_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  deleted_at      TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_vaults_org_id ON vaults(org_id) WHERE deleted_at
        IS NULL;
CREATE INDEX idx_vaults_owner_id ON vaults(owner_id) WHERE
        deleted_at IS NULL;
CREATE INDEX idx_vaults_name_trgm ON vaults USING GIN (name
        gin_trgm_ops) WHERE deleted_at IS NULL;
```

```sql
-- ============================================================
-- TABLE: vault_members
-- Users with access to each vault.
-- ============================================================
CREATE TABLE vault_members (
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vault_id              UUID NOT NULL REFERENCES vaults(id) ON
        DELETE CASCADE,
  user_id               UUID NOT NULL,
  permission            VARCHAR(20) NOT NULL CHECK (permission IN
        ('read', 'write', 'admin')),
  is_owner              BOOLEAN NOT NULL DEFAULT FALSE,
  invited_by            UUID,
  encrypted_vault_key   BYTEA NOT NULL,
  kdf_params            JSONB NOT NULL DEFAULT '{}',
  joined_at             TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  updated_at            TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW()
);

CREATE UNIQUE INDEX idx_vault_members_vault_user ON
        vault_members(vault_id, user_id);
CREATE INDEX idx_vault_members_user_id ON vault_members(user_id);


-- ============================================================
-- TABLE: vault_items
-- Individual encrypted items within a vault.
-- ============================================================
CREATE TABLE vault_items (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vault_id         UUID NOT NULL REFERENCES vaults(id) ON DELETE
        CASCADE,
  item_type        VARCHAR(50) NOT NULL
                     CHECK (item_type IN (
                       'host', 'ssh_key', 'password', 'note',
                       'certificate', 'totp_secret',
        'api_credential', 'file'
                     )),
  encrypted_data   BYTEA NOT NULL,
  checksum         VARCHAR(128) NOT NULL,
  iv               BYTEA NOT NULL,
  version          INTEGER NOT NULL DEFAULT 1,
  is_deleted       BOOLEAN NOT NULL DEFAULT FALSE,
  deleted_at       TIMESTAMP WITH TIME ZONE,
```

```
  created_by      UUID NOT NULL,
  updated_by      UUID,
  created_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  updated_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_vault_items_vault_id ON vault_items(vault_id)
        WHERE is_deleted = FALSE;
CREATE INDEX idx_vault_items_updated_at ON vault_items USING BRIN
        (updated_at);



-- ============================================================
-- TABLE: vault_item_versions
-- Version history for vault items (for conflict resolution).
-- ============================================================
CREATE TABLE vault_item_versions (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  item_id         UUID NOT NULL REFERENCES vault_items(id) ON
        DELETE CASCADE,
  vault_id        UUID NOT NULL,
  version         INTEGER NOT NULL,
  encrypted_data  BYTEA NOT NULL,
  checksum        VARCHAR(128) NOT NULL,
  iv              BYTEA NOT NULL,
  changed_by      UUID NOT NULL,
  created_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_vault_item_versions_item_id ON
        vault_item_versions(item_id, version DESC);
CREATE INDEX idx_vault_item_versions_vault_id ON
        vault_item_versions(vault_id);



-- ============================================================
-- TABLE: vault_sync_states
-- Per-client sync cursors for delta synchronization.
-- ============================================================
CREATE TABLE vault_sync_states (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vault_id        UUID NOT NULL REFERENCES vaults(id) ON DELETE
        CASCADE,
  user_id         UUID NOT NULL,
  client_id       VARCHAR(255) NOT NULL,
  cursor          TEXT NOT NULL DEFAULT '',
```

```
  server_version  BIGINT NOT NULL DEFAULT 0,
  last_synced_at  TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  client_platform VARCHAR(100),
  app_version     VARCHAR(50)
);

CREATE UNIQUE INDEX idx_vault_sync_states_vault_client ON
        vault_sync_states(vault_id, client_id);
CREATE INDEX idx_vault_sync_states_user_id ON
        vault_sync_states(user_id);



-- ============================================================
-- TABLE: vault_audit_events
-- Vault-level audit log (operations on vault items).
-- ============================================================
CREATE TABLE vault_audit_events (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vault_id      UUID NOT NULL,
  item_id       UUID,
  user_id       UUID NOT NULL,
  event_type    VARCHAR(100) NOT NULL,
  ip_address    INET,
  metadata      JSONB DEFAULT '{}',
  occurred_at   TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
) PARTITION BY RANGE (occurred_at);

CREATE INDEX idx_vault_audit_vault_id ON
        vault_audit_events(vault_id);
CREATE INDEX idx_vault_audit_user_id ON
        vault_audit_events(user_id);
CREATE INDEX idx_vault_audit_occurred_at ON vault_audit_events
        USING BRIN (occurred_at);

-- Create quarterly partitions
CREATE TABLE vault_audit_events_2026_q2 PARTITION OF
        vault_audit_events
  FOR VALUES FROM ('2026-04-01') TO ('2026-07-01');
CREATE TABLE vault_audit_events_2026_q3 PARTITION OF
        vault_audit_events
  FOR VALUES FROM ('2026-07-01') TO ('2026-10-01');
CREATE TABLE vault_audit_events_2026_q4 PARTITION OF
        vault_audit_events
  FOR VALUES FROM ('2026-10-01') TO ('2027-01-01');
```

17.3 host_db

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";

-- ============================================================
-- TABLE: hosts
-- SSH host definitions.
-- ============================================================
CREATE TABLE hosts (
  id                          UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
  vault_id                    UUID NOT NULL,
  group_id                    UUID,
  org_id                      UUID NOT NULL,
  created_by                  UUID NOT NULL,
  name                        VARCHAR(255) NOT NULL,
  hostname                    VARCHAR(512) NOT NULL,
  port                        INTEGER NOT NULL DEFAULT 22
                                CHECK (port BETWEEN 1 AND 65535),
  username                    VARCHAR(255),
  auth_method                 VARCHAR(20) NOT NULL DEFAULT 'key'
                                CHECK (auth_method IN (
                                  'key', 'password',
        'certificate',
                                  'interactive', 'agent', 'pgp'
                                )),
  key_id                      UUID,
  encrypted_password          BYTEA,
  certificate_id              UUID,
  os                          VARCHAR(50),
  os_version                  VARCHAR(100),
  arch                        VARCHAR(20),
  description                 TEXT,
  color                       VARCHAR(20),
  icon                        VARCHAR(50),
  tags                        TEXT[] NOT NULL DEFAULT '{}',
  jump_host_id                UUID,
  proxy_command               TEXT,
  connection_timeout_seconds  INTEGER NOT NULL DEFAULT 30,
  keepalive_interval_seconds  INTEGER NOT NULL DEFAULT 60,
  keepalive_count_max         INTEGER NOT NULL DEFAULT 3,
  server_alive_interval       INTEGER NOT NULL DEFAULT 0,
  compression                 BOOLEAN NOT NULL DEFAULT FALSE,
  cipher_suite                TEXT,
```

```
  macs                          TEXT,
  kex_algorithms                TEXT,
  host_key_algorithms           TEXT,
  environment_variables         JSONB NOT NULL DEFAULT '{}',
  startup_snippet_id            UUID,
  status                        VARCHAR(20) NOT NULL DEFAULT
        'active'
                                  CHECK (status IN ('active',
        'inactive', 'unreachable', 'archived')),
  last_connected_at             TIMESTAMP WITH TIME ZONE,
  last_connection_status        VARCHAR(20),
  fingerprint_verified          BOOLEAN NOT NULL DEFAULT FALSE,
  custom_fields                 JSONB NOT NULL DEFAULT '{}',
  sort_order                    INTEGER NOT NULL DEFAULT 0,
  created_at                    TIMESTAMP WITH TIME ZONE NOT NULL
        DEFAULT NOW(),
  updated_at                    TIMESTAMP WITH TIME ZONE NOT NULL
        DEFAULT NOW(),
  deleted_at                    TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_hosts_vault_id ON hosts(vault_id) WHERE
        deleted_at IS NULL;
CREATE INDEX idx_hosts_group_id ON hosts(group_id) WHERE
        deleted_at IS NULL;
CREATE INDEX idx_hosts_org_id ON hosts(org_id) WHERE deleted_at
        IS NULL;
CREATE INDEX idx_hosts_status ON hosts(status) WHERE deleted_at
        IS NULL;
CREATE INDEX idx_hosts_name_trgm ON hosts USING GIN (name
        gin_trgm_ops) WHERE deleted_at IS NULL;
CREATE INDEX idx_hosts_hostname_trgm ON hosts USING GIN (hostname
        gin_trgm_ops) WHERE deleted_at IS NULL;
CREATE INDEX idx_hosts_tags ON hosts USING GIN (tags);
CREATE INDEX idx_hosts_last_connected_at ON
        hosts(last_connected_at DESC NULLS LAST) WHERE deleted_at
        IS NULL;
CREATE INDEX idx_hosts_jump_host_id ON hosts(jump_host_id) WHERE
        jump_host_id IS NOT NULL;


-- ============================================================
-- TABLE: host_groups
-- Hierarchical host grouping.
-- ============================================================
CREATE TABLE host_groups (
  id                            UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
  vault_id                      UUID NOT NULL,
```

```
  org_id                          UUID NOT NULL,
  parent_id                       UUID REFERENCES host_groups(id)
        ON DELETE SET NULL,
  name                            VARCHAR(255) NOT NULL,
  description                     TEXT,
  color                           VARCHAR(20),
  icon                            VARCHAR(50),
  default_key_id                  UUID,
  default_username                VARCHAR(255),
  default_port                    INTEGER CHECK (default_port
        BETWEEN 1 AND 65535),
  default_jump_host_id            UUID,
  default_connection_timeout      INTEGER,
  default_keepalive_interval      INTEGER,
  inherit_from_parent             BOOLEAN NOT NULL DEFAULT TRUE,
  inherit_key                     BOOLEAN NOT NULL DEFAULT TRUE,
  inherit_username                BOOLEAN NOT NULL DEFAULT TRUE,
  inherit_port                    BOOLEAN NOT NULL DEFAULT FALSE,
  inherit_jump_host               BOOLEAN NOT NULL DEFAULT TRUE,
  inherit_environment_variables   BOOLEAN NOT NULL DEFAULT TRUE,
  inherit_startup_snippet         BOOLEAN NOT NULL DEFAULT FALSE,
  sort_order                      INTEGER NOT NULL DEFAULT 0,
  path                            TEXT NOT NULL DEFAULT '',
  depth                           INTEGER NOT NULL DEFAULT 0,
  created_at                      TIMESTAMP WITH TIME ZONE NOT
        NULL DEFAULT NOW(),
  updated_at                      TIMESTAMP WITH TIME ZONE NOT
        NULL DEFAULT NOW(),
  deleted_at                      TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_host_groups_vault_id ON host_groups(vault_id)
        WHERE deleted_at IS NULL;
CREATE INDEX idx_host_groups_parent_id ON host_groups(parent_id)
        WHERE deleted_at IS NULL;
CREATE INDEX idx_host_groups_org_id ON host_groups(org_id) WHERE
        deleted_at IS NULL;
CREATE INDEX idx_host_groups_path ON host_groups(path) WHERE
        deleted_at IS NULL;
CREATE INDEX idx_host_groups_name_trgm ON host_groups USING GIN
        (name gin_trgm_ops) WHERE deleted_at IS NULL;



-- ============================================================
-- TABLE: host_group_members
-- Many-to-many hosts↔groups (a host can be in multiple groups).
-- ============================================================
```

```
CREATE TABLE host_group_members (
  host_id     UUID NOT NULL REFERENCES hosts(id) ON DELETE
        CASCADE,
  group_id    UUID NOT NULL REFERENCES host_groups(id) ON DELETE
        CASCADE,
  added_by    UUID NOT NULL,
  added_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  PRIMARY KEY (host_id, group_id)
);

CREATE INDEX idx_host_group_members_group_id ON
        host_group_members(group_id);



-- ============================================================
-- TABLE: host_labels
-- Flexible key-value label system for hosts.
-- ============================================================
CREATE TABLE host_labels (
  id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  host_id   UUID NOT NULL REFERENCES hosts(id) ON DELETE CASCADE,
  key       VARCHAR(100) NOT NULL,
  value     VARCHAR(500) NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE UNIQUE INDEX idx_host_labels_host_key ON
        host_labels(host_id, key);
CREATE INDEX idx_host_labels_key_value ON host_labels(key, value);



-- ============================================================
-- TABLE: host_known_fingerprints
-- SSH host key fingerprints for TOFU (Trust on First Use).
-- ============================================================
CREATE TABLE host_known_fingerprints (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  host_id         UUID NOT NULL REFERENCES hosts(id) ON DELETE
        CASCADE,
  algorithm       VARCHAR(20) NOT NULL,
  fingerprint     VARCHAR(512) NOT NULL,
  raw_key         TEXT NOT NULL,
  verified        BOOLEAN NOT NULL DEFAULT FALSE,
  verified_by     UUID,
  verified_at     TIMESTAMP WITH TIME ZONE,
  first_seen_at   TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
```

```
  last_seen_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  revoked         BOOLEAN NOT NULL DEFAULT FALSE,
  revoked_at      TIMESTAMP WITH TIME ZONE,
  revoke_reason   VARCHAR(255)
);

CREATE UNIQUE INDEX idx_known_fingerprints_host_algo ON
        host_known_fingerprints(host_id, algorithm)
  WHERE revoked = FALSE;
CREATE INDEX idx_known_fingerprints_fingerprint ON
        host_known_fingerprints(fingerprint);



-- ============================================================
-- TABLE: host_connection_history
-- Log of every SSH connection attempt.
-- ============================================================
CREATE TABLE host_connection_history (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  host_id         UUID NOT NULL,
  user_id         UUID NOT NULL,
  org_id          UUID NOT NULL,
  session_id      UUID,
  client_ip       INET NOT NULL,
  auth_method     VARCHAR(20) NOT NULL,
  key_id          UUID,
  started_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  ended_at        TIMESTAMP WITH TIME ZONE,
  duration_seconds INTEGER,
  bytes_sent      BIGINT NOT NULL DEFAULT 0,
  bytes_received  BIGINT NOT NULL DEFAULT 0,
  exit_code       INTEGER,
  disconnect_reason VARCHAR(255),
  recording_path  TEXT,
  jump_chain      JSONB DEFAULT '[]',
  metadata        JSONB DEFAULT '{}'
) PARTITION BY RANGE (started_at);

CREATE INDEX idx_host_conn_history_host_id ON
        host_connection_history(host_id, started_at DESC);
CREATE INDEX idx_host_conn_history_user_id ON
        host_connection_history(user_id, started_at DESC);
CREATE INDEX idx_host_conn_history_started_at ON
        host_connection_history USING BRIN (started_at);
```

```
CREATE TABLE host_connection_history_2026_q2 PARTITION OF
        host_connection_history
  FOR VALUES FROM ('2026-04-01') TO ('2026-07-01');
CREATE TABLE host_connection_history_2026_q3 PARTITION OF
        host_connection_history
  FOR VALUES FROM ('2026-07-01') TO ('2026-10-01');
CREATE TABLE host_connection_history_2026_q4 PARTITION OF
        host_connection_history
  FOR VALUES FROM ('2026-10-01') TO ('2027-01-01');



-- ============================================================
-- TABLE: jump_host_chains
-- Saved multi-hop jump host configurations.
-- ============================================================
CREATE TABLE jump_host_chains (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vault_id    UUID NOT NULL,
  org_id      UUID NOT NULL,
  name        VARCHAR(255) NOT NULL,
  description TEXT,
  hops        JSONB NOT NULL DEFAULT '[]',
  created_by  UUID NOT NULL,
  created_at  TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_jump_chains_vault_id ON
        jump_host_chains(vault_id);
```

17.4 keychain_db

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";


-- ============================================================
-- TABLE: ssh_keys
-- SSH key metadata (private key stored encrypted).
-- ============================================================
CREATE TABLE ssh_keys (
  id                    UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
  vault_id              UUID NOT NULL,
  org_id                UUID NOT NULL,
  user_id               UUID NOT NULL,
```

```
  name                  VARCHAR(255) NOT NULL,
  type                  VARCHAR(20) NOT NULL DEFAULT 'ssh_key'
                          CHECK (type IN ('ssh_key',
        'certificate', 'pgp', 'gpg')),
  algorithm             VARCHAR(20) NOT NULL
                          CHECK (algorithm IN ('ed25519',
        'ecdsa', 'rsa', 'dsa', 'ecdsa-sk', 'ed25519-sk')),
  bits                  INTEGER,
  comment               VARCHAR(512),
  fingerprint           VARCHAR(512) NOT NULL,
  public_key_openssh    TEXT NOT NULL,
  encrypted_private_key BYTEA,
  private_key_iv        BYTEA,
  has_passphrase        BOOLEAN NOT NULL DEFAULT FALSE,
  passphrase_protected  BOOLEAN NOT NULL DEFAULT FALSE,
  is_agent_forwarding   BOOLEAN NOT NULL DEFAULT FALSE,
  source                VARCHAR(20) NOT NULL DEFAULT 'generated'
                          CHECK (source IN ('generated',
        'imported', 'agent')),
  expires_at            TIMESTAMP WITH TIME ZONE,
  created_at            TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  updated_at            TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  deleted_at            TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_ssh_keys_vault_id ON ssh_keys(vault_id) WHERE
        deleted_at IS NULL;
CREATE INDEX idx_ssh_keys_user_id ON ssh_keys(user_id) WHERE
        deleted_at IS NULL;
CREATE INDEX idx_ssh_keys_fingerprint ON ssh_keys(fingerprint);
CREATE INDEX idx_ssh_keys_name_trgm ON ssh_keys USING GIN (name
        gin_trgm_ops) WHERE deleted_at IS NULL;



-- ============================================================
-- TABLE: key_deployments
-- Record of public key deployments to hosts.
-- ============================================================
CREATE TABLE key_deployments (
  id                    UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
  key_id                UUID NOT NULL REFERENCES ssh_keys(id) ON
        DELETE CASCADE,
  host_id               UUID NOT NULL,
  host_name             VARCHAR(255) NOT NULL,
  target_user           VARCHAR(255) NOT NULL,
```

```
  auth_key_options      JSONB DEFAULT '{}',
  deployed_by           UUID NOT NULL,
  deployed_at           TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  revoked               BOOLEAN NOT NULL DEFAULT FALSE,
  revoked_at            TIMESTAMP WITH TIME ZONE,
  revoked_by            UUID,
  revoke_reason         VARCHAR(255),
  status                VARCHAR(20) NOT NULL DEFAULT 'active'
                          CHECK (status IN ('active', 'revoked',
        'expired', 'error'))
);

CREATE INDEX idx_key_deployments_key_id ON
        key_deployments(key_id);
CREATE INDEX idx_key_deployments_host_id ON
        key_deployments(host_id);



-- ============================================================
-- TABLE: key_usage_log
-- Immutable log of key usage events.
-- ============================================================
CREATE TABLE key_usage_log (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  key_id      UUID NOT NULL,
  host_id     UUID,
  user_id     UUID NOT NULL,
  session_id  UUID,
  event_type  VARCHAR(50) NOT NULL
                CHECK (event_type IN ('auth_success',
        'auth_failure', 'sign', 'deploy', 'revoke')),
  ip_address  INET,
  occurred_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
) PARTITION BY RANGE (occurred_at);

CREATE INDEX idx_key_usage_key_id ON key_usage_log(key_id);
CREATE INDEX idx_key_usage_occurred_at ON key_usage_log
        USING BRIN (occurred_at);

CREATE TABLE key_usage_log_2026_q2 PARTITION OF key_usage_log
  FOR VALUES FROM ('2026-04-01') TO ('2026-07-01');
CREATE TABLE key_usage_log_2026_q3 PARTITION OF key_usage_log
  FOR VALUES FROM ('2026-07-01') TO ('2026-10-01');
CREATE TABLE key_usage_log_2026_q4 PARTITION OF key_usage_log
  FOR VALUES FROM ('2026-10-01') TO ('2027-01-01');
```

```
-- ============================================================
-- TABLE: certificate_store
-- SSH certificates issued or stored for use.
-- ============================================================
CREATE TABLE certificate_store (
  id                    UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
  key_id                UUID REFERENCES ssh_keys(id) ON DELETE
        SET NULL,
  vault_id              UUID NOT NULL,
  user_id               UUID NOT NULL,
  certificate_type      VARCHAR(20) NOT NULL CHECK
        (certificate_type IN ('user', 'host')),
  certificate_openssh   TEXT NOT NULL,
  serial                BIGINT NOT NULL,
  fingerprint           VARCHAR(512) NOT NULL,
  principals            TEXT[] NOT NULL DEFAULT '{}',
  extensions            JSONB DEFAULT '{}',
  critical_options      JSONB DEFAULT '{}',
  valid_after           TIMESTAMP WITH TIME ZONE NOT NULL,
  valid_before          TIMESTAMP WITH TIME ZONE NOT NULL,
  signed_by_ca_id       UUID,
  revoked               BOOLEAN NOT NULL DEFAULT FALSE,
  revoked_at            TIMESTAMP WITH TIME ZONE,
  revoke_reason         VARCHAR(255),
  created_at            TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW()
);

CREATE UNIQUE INDEX idx_cert_store_serial ON
        certificate_store(serial);
CREATE INDEX idx_cert_store_key_id ON certificate_store(key_id);
CREATE INDEX idx_cert_store_user_id ON certificate_store(user_id);
CREATE INDEX idx_cert_store_valid_before ON
        certificate_store(valid_before)
  WHERE revoked = FALSE;
```

17.5 session_db

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- ============================================================
-- TABLE: ssh_sessions
```

```
-- SSH terminal session records.
-- ============================================================
CREATE TABLE ssh_sessions (
  id                    UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
  user_id               UUID NOT NULL,
  host_id               UUID NOT NULL,
  vault_id              UUID NOT NULL,
  org_id                UUID NOT NULL,
  client_ip             INET NOT NULL,
  user_agent            TEXT,
  terminal_cols         SMALLINT NOT NULL DEFAULT 80,
  terminal_rows         SMALLINT NOT NULL DEFAULT 24,
  terminal_type         VARCHAR(50) NOT NULL DEFAULT
        'xterm-256color',
  auth_method           VARCHAR(20),
  key_id                UUID,
  recording_enabled     BOOLEAN NOT NULL DEFAULT FALSE,
  recording_path        TEXT,
  recording_size_bytes  BIGINT DEFAULT 0,
  collab_enabled        BOOLEAN NOT NULL DEFAULT FALSE,
  read_only             BOOLEAN NOT NULL DEFAULT FALSE,
  status                VARCHAR(20) NOT NULL DEFAULT 'connecting'
                          CHECK (status IN (
                            'connecting', 'connected',
        'disconnected',
                            'error', 'terminated'
                          )),
  reason                TEXT,
  ticket_ref            VARCHAR(255),
  startup_snippet_id    UUID,
  jump_chain            JSONB DEFAULT '[]',
  exit_code             INTEGER,
  disconnect_reason     TEXT,
  bytes_sent            BIGINT NOT NULL DEFAULT 0,
  bytes_received        BIGINT NOT NULL DEFAULT 0,
  commands_count        INTEGER NOT NULL DEFAULT 0,
  resize_count          INTEGER NOT NULL DEFAULT 0,
  started_at            TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  connected_at          TIMESTAMP WITH TIME ZONE,
  ended_at              TIMESTAMP WITH TIME ZONE,
  duration_seconds      INTEGER
) PARTITION BY RANGE (started_at);
```

```
CREATE INDEX idx_ssh_sessions_user_id ON ssh_sessions(user_id,
        started_at DESC);
CREATE INDEX idx_ssh_sessions_host_id ON ssh_sessions(host_id,
        started_at DESC);
CREATE INDEX idx_ssh_sessions_org_id ON ssh_sessions(org_id,
        started_at DESC);
CREATE INDEX idx_ssh_sessions_status ON ssh_sessions(status,
        started_at DESC)
  WHERE status IN ('connecting', 'connected');
CREATE INDEX idx_ssh_sessions_started_at ON ssh_sessions USING
        BRIN (started_at);

CREATE TABLE ssh_sessions_2026_q2 PARTITION OF ssh_sessions
  FOR VALUES FROM ('2026-04-01') TO ('2026-07-01');
CREATE TABLE ssh_sessions_2026_q3 PARTITION OF ssh_sessions
  FOR VALUES FROM ('2026-07-01') TO ('2026-10-01');
CREATE TABLE ssh_sessions_2026_q4 PARTITION OF ssh_sessions
  FOR VALUES FROM ('2026-10-01') TO ('2027-01-01');



-- ============================================================
-- TABLE: session_events
-- Per-event recording of terminal I/O (asciinema-compatible).
-- ============================================================
CREATE TABLE session_events (
  id            BIGSERIAL,
  session_id    UUID NOT NULL,
  occurred_at   TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  elapsed_ms    BIGINT NOT NULL,
  direction     CHAR(1) NOT NULL CHECK (direction IN ('i', 'o')),
  data          BYTEA NOT NULL,
  event_type    VARCHAR(20) NOT NULL DEFAULT 'data'
                  CHECK (event_type IN ('data', 'resize',
        'marker', 'metadata'))
) PARTITION BY RANGE (occurred_at);

CREATE INDEX idx_session_events_session_id ON
        session_events(session_id, occurred_at);
CREATE INDEX idx_session_events_occurred_at ON session_events
        USING BRIN (occurred_at);

CREATE TABLE session_events_2026_q2 PARTITION OF session_events
  FOR VALUES FROM ('2026-04-01') TO ('2026-07-01');
CREATE TABLE session_events_2026_q3 PARTITION OF session_events
  FOR VALUES FROM ('2026-07-01') TO ('2026-10-01');
```

```
-- ============================================================
-- TABLE: session_recordings
-- Metadata for session recording files (asciinema v2).
-- ============================================================
CREATE TABLE session_recordings (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id        UUID NOT NULL,
  storage_path      TEXT NOT NULL,
  storage_backend   VARCHAR(20) NOT NULL DEFAULT 's3'
                      CHECK (storage_backend IN ('s3', 'gcs',
        'azure_blob', 'local')),
  file_size_bytes   BIGINT NOT NULL DEFAULT 0,
  duration_seconds  INTEGER,
  format            VARCHAR(20) NOT NULL DEFAULT 'asciicast_v2',
  terminal_cols     SMALLINT NOT NULL,
  terminal_rows     SMALLINT NOT NULL,
  checksum_sha256   VARCHAR(64),
  compressed        BOOLEAN NOT NULL DEFAULT TRUE,
  encryption_key_id UUID,
  processed         BOOLEAN NOT NULL DEFAULT FALSE,
  processed_at      TIMESTAMP WITH TIME ZONE,
  created_at        TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW()
);

CREATE UNIQUE INDEX idx_session_recordings_session_id ON
        session_recordings(session_id);
CREATE INDEX idx_session_recordings_created_at ON
        session_recordings USING BRIN (created_at);



-- ============================================================
-- TABLE: sftp_sessions
-- SFTP session records.
-- ============================================================
CREATE TABLE sftp_sessions (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID NOT NULL,
  host_id           UUID NOT NULL,
  org_id            UUID NOT NULL,
  client_ip         INET NOT NULL,
  cwd               TEXT NOT NULL DEFAULT '/',
  transfer_mode     VARCHAR(10) NOT NULL DEFAULT 'binary'
                      CHECK (transfer_mode IN ('binary',
        'ascii')),
  server_version    VARCHAR(100),
```

```
  status            VARCHAR(20) NOT NULL DEFAULT 'connected'
                      CHECK (status IN ('connected', 'closed',
        'error', 'expired')),
  files_uploaded    INTEGER NOT NULL DEFAULT 0,
  files_downloaded  INTEGER NOT NULL DEFAULT 0,
  bytes_uploaded    BIGINT NOT NULL DEFAULT 0,
  bytes_downloaded  BIGINT NOT NULL DEFAULT 0,
  started_at        TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  ended_at          TIMESTAMP WITH TIME ZONE,
  expires_at        TIMESTAMP WITH TIME ZONE NOT NULL
);

CREATE INDEX idx_sftp_sessions_user_id ON sftp_sessions(user_id);
CREATE INDEX idx_sftp_sessions_host_id ON sftp_sessions(host_id);
CREATE INDEX idx_sftp_sessions_started_at ON sftp_sessions USING
        BRIN (started_at);



-- ============================================================
-- TABLE: sftp_transfers
-- Individual SFTP file transfer records.
-- ============================================================
CREATE TABLE sftp_transfers (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sftp_session_id   UUID NOT NULL REFERENCES sftp_sessions(id) ON
        DELETE CASCADE,
  user_id           UUID NOT NULL,
  host_id           UUID NOT NULL,
  direction         VARCHAR(10) NOT NULL CHECK (direction IN
        ('upload', 'download')),
  local_filename    VARCHAR(1024),
  remote_path       TEXT NOT NULL,
  file_size_bytes   BIGINT NOT NULL DEFAULT 0,
  bytes_transferred BIGINT NOT NULL DEFAULT 0,
  checksum_sha256   VARCHAR(64),
  status            VARCHAR(20) NOT NULL DEFAULT 'completed'
                      CHECK (status IN ('pending', 'in_progress',
        'completed', 'failed', 'cancelled')),
  error_message     TEXT,
  duration_ms       INTEGER,
  transferred_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW()
) PARTITION BY RANGE (transferred_at);

CREATE INDEX idx_sftp_transfers_session_id ON
        sftp_transfers(sftp_session_id);
```

```
CREATE INDEX idx_sftp_transfers_user_id
        ON sftp_transfers(user_id, transferred_at DESC);
CREATE INDEX idx_sftp_transfers_transferred_at ON sftp_transfers
        USING BRIN (transferred_at);

CREATE TABLE sftp_transfers_2026_q2 PARTITION OF sftp_transfers
  FOR VALUES FROM ('2026-04-01') TO ('2026-07-01');
CREATE TABLE sftp_transfers_2026_q3 PARTITION OF sftp_transfers
  FOR VALUES FROM ('2026-07-01') TO ('2026-10-01');



-- ============================================================
-- TABLE: port_forward_rules
-- Port forwarding rule definitions.
-- ============================================================
CREATE TABLE port_forward_rules (
  id                    UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
  user_id               UUID NOT NULL,
  host_id               UUID NOT NULL,
  vault_id              UUID NOT NULL,
  org_id                UUID NOT NULL,
  name                  VARCHAR(255) NOT NULL,
  description           TEXT,
  type                  VARCHAR(20) NOT NULL
                          CHECK (type IN ('local', 'remote',
        'dynamic')),
  local_address         VARCHAR(255) NOT NULL DEFAULT '127.0.0.1',
  local_port            INTEGER NOT NULL CHECK (local_port
        BETWEEN 1 AND 65535),
  remote_address        VARCHAR(255),
  remote_port           INTEGER CHECK (remote_port BETWEEN 1 AND
        65535),
  bind_address          VARCHAR(255),
  auto_start            BOOLEAN NOT NULL DEFAULT FALSE,
  status                VARCHAR(20) NOT NULL DEFAULT 'inactive'
                          CHECK (status IN ('active', 'inactive',
        'error')),
  sort_order            INTEGER NOT NULL DEFAULT 0,
  created_at            TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  updated_at            TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  deleted_at            TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_pf_rules_user_id ON port_forward_rules(user_id)
        WHERE deleted_at IS NULL;
```

```
CREATE INDEX idx_pf_rules_host_id ON port_forward_rules(host_id)
        WHERE deleted_at IS NULL;


-- ============================================================
-- TABLE: port_forward_connections
-- Active and historical port forwarding connections.
-- ============================================================
CREATE TABLE port_forward_connections (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  rule_id         UUID NOT NULL REFERENCES port_forward_rules(id)
        ON DELETE CASCADE,
  user_id         UUID NOT NULL,
  host_id         UUID NOT NULL,
  ssh_session_id  UUID,
  status          VARCHAR(20) NOT NULL DEFAULT 'active'
                    CHECK (status IN ('active', 'closed',
        'error')),
  bytes_sent      BIGINT NOT NULL DEFAULT 0,
  bytes_received  BIGINT NOT NULL DEFAULT 0,
  started_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  ended_at        TIMESTAMP WITH TIME ZONE,
  error_message   TEXT
);

CREATE INDEX idx_pf_connections_rule_id ON
        port_forward_connections(rule_id);
CREATE INDEX idx_pf_connections_started_at ON
        port_forward_connections USING BRIN (started_at);
```

17.6 snippet_db

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";

-- ============================================================
-- TABLE: snippet_categories
-- Hierarchical snippet categorization.
-- ============================================================
CREATE TABLE snippet_categories (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vault_id    UUID NOT NULL,
  org_id      UUID NOT NULL,
  parent_id   UUID REFERENCES snippet_categories(id) ON DELETE
        SET NULL,
```

```
  name        VARCHAR(255) NOT NULL,
  description TEXT,
  color       VARCHAR(20),
  icon        VARCHAR(50),
  sort_order  INTEGER NOT NULL DEFAULT 0,
  created_at  TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_snippet_categories_vault_id ON
        snippet_categories(vault_id);
CREATE INDEX idx_snippet_categories_parent_id ON
        snippet_categories(parent_id);



-- ============================================================
-- TABLE: snippets
-- Command snippets / scripts.
-- ============================================================
CREATE TABLE snippets (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vault_id            UUID NOT NULL,
  org_id              UUID NOT NULL,
  created_by          UUID NOT NULL,
  category_id         UUID REFERENCES snippet_categories(id) ON
        DELETE SET NULL,
  name                VARCHAR(255) NOT NULL,
  description         TEXT,
  content             TEXT NOT NULL,
  language            VARCHAR(50) NOT NULL DEFAULT 'bash',
  interpreter         VARCHAR(255),
  shebang             VARCHAR(255),
  tags                TEXT[] NOT NULL DEFAULT '{}',
  parameters          JSONB NOT NULL DEFAULT '[]',
  shared              BOOLEAN NOT NULL DEFAULT FALSE,
  pinned              BOOLEAN NOT NULL DEFAULT FALSE,
  executions_count    BIGINT NOT NULL DEFAULT 0,
  last_executed_at    TIMESTAMP WITH TIME ZONE,
  fts_vector          TSVECTOR,
  created_at          TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  updated_at          TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  deleted_at          TIMESTAMP WITH TIME ZONE
);
```

```
CREATE INDEX idx_snippets_vault_id ON snippets(vault_id) WHERE
        deleted_at IS NULL;
CREATE INDEX idx_snippets_category_id ON snippets(category_id)
        WHERE deleted_at IS NULL;
CREATE INDEX idx_snippets_tags ON snippets USING GIN (tags) WHERE
        deleted_at IS NULL;
CREATE INDEX idx_snippets_name_trgm ON snippets USING GIN (name
        gin_trgm_ops) WHERE deleted_at IS NULL;
CREATE INDEX idx_snippets_content_trgm ON snippets USING GIN
        (content gin_trgm_ops) WHERE deleted_at IS NULL;
CREATE INDEX idx_snippets_fts ON snippets USING GIN (fts_vector)
        WHERE deleted_at IS NULL;

-- Trigger to maintain FTS vector
CREATE OR REPLACE FUNCTION snippets_fts_update()
RETURNS TRIGGER AS $$
BEGIN
  NEW.fts_vector :=
    setweight(to_tsvector('english', coalesce(NEW.name, '')),
        'A') ||
    setweight(to_tsvector('english', coalesce(NEW.description,
        '')), 'B') ||
    setweight(to_tsvector('english', coalesce(NEW.content, '')),
        'C');
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_snippets_fts
  BEFORE INSERT OR UPDATE ON snippets
  FOR EACH ROW EXECUTE FUNCTION snippets_fts_update();



-- ============================================================
-- TABLE: snippet_executions
-- Log of snippet executions.
-- ============================================================
CREATE TABLE snippet_executions (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  snippet_id        UUID NOT NULL,
  user_id           UUID NOT NULL,
  org_id            UUID NOT NULL,
  execution_mode    VARCHAR(20) NOT NULL DEFAULT 'parallel'
                      CHECK (execution_mode IN ('parallel',
        'sequential')),
  host_count        INTEGER NOT NULL DEFAULT 0,
  parameters        JSONB DEFAULT '{}',
  status            VARCHAR(20) NOT NULL DEFAULT 'pending'
```

```
                    CHECK (status IN ('pending', 'running',
        'completed', 'failed', 'cancelled')),
  completed_count   INTEGER NOT NULL DEFAULT 0,
  failed_count      INTEGER NOT NULL DEFAULT 0,
  timeout_seconds   INTEGER NOT NULL DEFAULT 60,
  started_at        TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  completed_at      TIMESTAMP WITH TIME ZONE
) PARTITION BY RANGE (started_at);

CREATE INDEX idx_snippet_executions_snippet_id ON
        snippet_executions(snippet_id);
CREATE INDEX idx_snippet_executions_user_id ON
        snippet_executions(user_id, started_at DESC);
CREATE INDEX idx_snippet_executions_started_at ON
        snippet_executions USING BRIN (started_at);

CREATE TABLE snippet_executions_2026_q2 PARTITION OF
        snippet_executions
  FOR VALUES FROM ('2026-04-01') TO ('2026-07-01');
CREATE TABLE snippet_executions_2026_q3 PARTITION OF
        snippet_executions
  FOR VALUES FROM ('2026-07-01') TO ('2026-10-01');



-- ============================================================
-- TABLE: snippet_execution_results
-- Per-host results for each execution.
-- ============================================================
CREATE TABLE snippet_execution_results (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  execution_id    UUID NOT NULL,
  host_id         UUID NOT NULL,
  host_name       VARCHAR(255) NOT NULL,
  session_id      UUID,
  status          VARCHAR(20) NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending', 'running',
        'success', 'failure', 'timeout')),
  exit_code       INTEGER,
  stdout          TEXT,
  stderr          TEXT,
  error_message   TEXT,
  started_at      TIMESTAMP WITH TIME ZONE,
  completed_at    TIMESTAMP WITH TIME ZONE,
  duration_ms     INTEGER
);
```

```
CREATE INDEX idx_exec_results_execution_id ON
        snippet_execution_results(execution_id);
```

17.7 workspace_db

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- ============================================================
-- TABLE: workspaces
-- Saved terminal layout configurations.
-- ============================================================
CREATE TABLE workspaces (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         UUID NOT NULL,
  org_id          UUID NOT NULL,
  name            VARCHAR(255) NOT NULL,
  description     TEXT,
  layout          JSONB NOT NULL DEFAULT '{}',
  thumbnail_url   TEXT,
  is_template     BOOLEAN NOT NULL DEFAULT FALSE,
  template_id     UUID,
  pinned          BOOLEAN NOT NULL DEFAULT FALSE,
  auto_connect    BOOLEAN NOT NULL DEFAULT TRUE,
  last_opened_at  TIMESTAMP WITH TIME ZONE,
  open_count      INTEGER NOT NULL DEFAULT 0,
  created_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  updated_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  deleted_at      TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_workspaces_user_id ON workspaces(user_id) WHERE
        deleted_at IS NULL;
CREATE INDEX idx_workspaces_org_id ON workspaces(org_id) WHERE
        deleted_at IS NULL;
CREATE INDEX idx_workspaces_layout ON workspaces USING GIN
        (layout) WHERE deleted_at IS NULL;



-- ============================================================
-- TABLE: workspace_snapshots
-- Point-in-time snapshots of workspace layouts.
-- ============================================================
CREATE TABLE workspace_snapshots (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  workspace_id  UUID NOT NULL REFERENCES workspaces(id) ON DELETE
        CASCADE,
  layout        JSONB NOT NULL,
  snapshot_name VARCHAR(255),
  created_by    UUID NOT NULL,
  created_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_workspace_snapshots_workspace_id ON
        workspace_snapshots(workspace_id, created_at DESC);



-- ============================================================
-- TABLE: workspace_sessions
-- Maps workspace to the sessions opened within it.
-- ============================================================
CREATE TABLE workspace_sessions (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id  UUID NOT NULL REFERENCES workspaces(id) ON DELETE
        CASCADE,
  session_id    UUID NOT NULL,
  pane_id       VARCHAR(50) NOT NULL,
  user_id       UUID NOT NULL,
  created_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_workspace_sessions_workspace_id ON
        workspace_sessions(workspace_id);
CREATE INDEX idx_workspace_sessions_session_id ON
        workspace_sessions(session_id);



-- ============================================================
-- TABLE: workspace_templates
-- Reusable workspace layout templates.
-- ============================================================
CREATE TABLE workspace_templates (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  created_by    UUID NOT NULL,
  org_id        UUID,
  name          VARCHAR(255) NOT NULL,
  description   TEXT,
  category      VARCHAR(100),
  layout        JSONB NOT NULL DEFAULT '{}',
  pane_count    SMALLINT NOT NULL DEFAULT 1,
  preview_url   TEXT,
```

```
  public         BOOLEAN NOT NULL DEFAULT FALSE,
  usage_count    INTEGER NOT NULL DEFAULT 0,
  created_at     TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  updated_at     TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_workspace_templates_org_id ON
        workspace_templates(org_id);
CREATE INDEX idx_workspace_templates_public ON
        workspace_templates(public, usage_count DESC)
  WHERE public = TRUE;
```

17.8 collab_db

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";


-- ============================================================
-- TABLE: collaboration_sessions
-- Collaborative terminal session metadata.
-- ============================================================
CREATE TABLE collaboration_sessions (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ssh_session_id    UUID NOT NULL UNIQUE,
  org_id            UUID NOT NULL,
  owner_id          UUID NOT NULL,
  title             VARCHAR(255),
  max_participants  SMALLINT NOT NULL DEFAULT 10,
  allow_input       BOOLEAN NOT NULL DEFAULT FALSE,
  status            VARCHAR(20) NOT NULL DEFAULT 'active'
                      CHECK (status IN ('active', 'ended')),
  started_at        TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  ended_at          TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_collab_sessions_ssh_session ON
        collaboration_sessions(ssh_session_id);
CREATE INDEX idx_collab_sessions_org_id ON
        collaboration_sessions(org_id, started_at DESC);



-- ============================================================
-- TABLE: collaboration_participants
-- Participants in a collaboration session.
-- ============================================================
```

```
CREATE TABLE collaboration_participants (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  collab_id       UUID NOT NULL REFERENCES
        collaboration_sessions(id) ON DELETE CASCADE,
  user_id         UUID NOT NULL,
  display_name    VARCHAR(100) NOT NULL,
  role            VARCHAR(20) NOT NULL DEFAULT 'viewer'
                    CHECK (role IN ('owner', 'contributor',
        'viewer')),
  cursor_color    VARCHAR(20),
  connected_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  disconnected_at TIMESTAMP WITH TIME ZONE,
  is_active       BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE INDEX idx_collab_participants_collab_id ON
        collaboration_participants(collab_id)
  WHERE is_active = TRUE;
CREATE INDEX idx_collab_participants_user_id ON
        collaboration_participants(user_id);



-- ============================================================
-- TABLE: collaboration_events
-- Event log for collaboration sessions (chat, cursor, control).
-- ============================================================
CREATE TABLE collaboration_events (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  collab_id     UUID NOT NULL,
  user_id       UUID NOT NULL,
  event_type    VARCHAR(50) NOT NULL
                  CHECK (event_type IN ('chat', 'cursor', 'join',
        'leave', 'control_request', 'control_granted')),
  payload       JSONB DEFAULT '{}',
  occurred_at   TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
) PARTITION BY RANGE (occurred_at);

CREATE INDEX idx_collab_events_collab_id ON
        collaboration_events(collab_id, occurred_at);
CREATE INDEX idx_collab_events_occurred_at ON
        collaboration_events USING BRIN (occurred_at);

CREATE TABLE collaboration_events_2026_q2 PARTITION OF
        collaboration_events
  FOR VALUES FROM ('2026-04-01') TO ('2026-07-01');
```

```
CREATE TABLE collaboration_events_2026_q3 PARTITION OF
        collaboration_events
  FOR VALUES FROM ('2026-07-01') TO ('2026-10-01');
```

17.9 org_db

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "citext";

-- ============================================================
-- TABLE: organizations
-- Organizations (tenants).
-- ============================================================
CREATE TABLE organizations (
  id                                UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
  name                              VARCHAR(255) NOT NULL,
  slug                              CITEXT NOT NULL UNIQUE,
  domain                            CITEXT,
  domain_verified                   BOOLEAN NOT NULL DEFAULT FALSE,
  domain_verified_at                TIMESTAMP WITH TIME ZONE,
  domain_verification_token         VARCHAR(255),
  plan                              VARCHAR(50) NOT NULL DEFAULT
        'free'
                                      CHECK (plan IN ('free',
        'pro', 'team', 'enterprise')),
  plan_expires_at                   TIMESTAMP WITH TIME ZONE,
  max_members                       INTEGER NOT NULL DEFAULT 5,
  max_vaults                        INTEGER NOT NULL DEFAULT 3,
  max_hosts                         INTEGER NOT NULL DEFAULT 50,
  max_sessions_concurrent           INTEGER NOT NULL DEFAULT 5,
  enforce_mfa                       BOOLEAN NOT NULL DEFAULT FALSE,
  session_recording_required        BOOLEAN NOT NULL DEFAULT FALSE,
  session_recording_retention_days  INTEGER NOT NULL DEFAULT 90,
  ip_allowlist                      CIDR[] DEFAULT '{}',
  sso_required                      BOOLEAN NOT NULL DEFAULT FALSE,
  audit_log_retention_days          INTEGER NOT NULL DEFAULT 365,
  owner_id                          UUID NOT NULL,
  billing_email                     CITEXT,
  stripe_customer_id                VARCHAR(255),
  status                            VARCHAR(20) NOT NULL DEFAULT
        'active'
                                      CHECK (status IN ('active',
        'suspended', 'trial', 'cancelled')),
  settings                          JSONB NOT NULL DEFAULT '{}',
```

```
  created_at                      TIMESTAMP WITH TIME ZONE NOT
        NULL DEFAULT NOW(),
  updated_at                      TIMESTAMP WITH TIME ZONE NOT
        NULL DEFAULT NOW(),
  deleted_at                      TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_organizations_slug ON organizations(slug) WHERE
        deleted_at IS NULL;
CREATE INDEX idx_organizations_domain ON organizations(domain)
        WHERE deleted_at IS NULL AND domain IS NOT NULL;
CREATE INDEX idx_organizations_owner_id ON
        organizations(owner_id);



-- ============================================================
-- TABLE: org_members
-- Members of organizations.
-- ============================================================
CREATE TABLE org_members (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id        UUID NOT NULL REFERENCES organizations(id) ON
        DELETE CASCADE,
  user_id       UUID NOT NULL,
  -- Canonical 6-role vocabulary (CD-8; single RBAC schema,
        reconciles the prior
  -- owner/admin/member/viewer/billing set). 'billing' is a
        permission, not a
  -- role — granted via `roles` / `role_assignments`, not this
        column.
  role          VARCHAR(20) NOT NULL DEFAULT 'member'
                  CHECK (role IN ('super_admin', 'org_admin',
        'team_admin', 'member', 'auditor', 'api_user')),
  invited_by    UUID,
  joined_at     TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  status        VARCHAR(20) NOT NULL DEFAULT 'active'
                  CHECK (status IN ('active', 'suspended',
        'pending')),
  last_active_at TIMESTAMP WITH TIME ZONE
);

CREATE UNIQUE INDEX idx_org_members_org_user ON
        org_members(org_id, user_id)
  WHERE status != 'suspended';
CREATE INDEX idx_org_members_user_id ON org_members(user_id);
CREATE INDEX idx_org_members_role ON org_members(org_id, role);



-- ============================================================
```

```
-- TABLE: teams
-- Teams within an organization.
-- ============================================================
CREATE TABLE teams (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id        UUID NOT NULL REFERENCES organizations(id) ON
        DELETE CASCADE,
  name          VARCHAR(255) NOT NULL,
  slug          CITEXT NOT NULL,
  description   TEXT,
  settings      JSONB DEFAULT '{}',
  created_by    UUID NOT NULL,
  created_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  updated_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  deleted_at    TIMESTAMP WITH TIME ZONE
);

CREATE UNIQUE INDEX idx_teams_org_slug ON teams(org_id, slug)
        WHERE deleted_at IS NULL;
CREATE INDEX idx_teams_org_id ON teams(org_id) WHERE deleted_at
        IS NULL;



-- ============================================================
-- TABLE: team_members
-- Members of teams.
-- ============================================================
CREATE TABLE team_members (
  team_id     UUID NOT NULL REFERENCES teams(id) ON DELETE
        CASCADE,
  user_id     UUID NOT NULL,
  -- Canonical 6-role vocabulary (CD-8): 'lead' reconciled to
        'team_admin'.
  role        VARCHAR(20) NOT NULL DEFAULT 'member'
                CHECK (role IN ('team_admin', 'member')),
  added_by    UUID NOT NULL,
  added_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  PRIMARY KEY (team_id, user_id)
);

CREATE INDEX idx_team_members_user_id ON team_members(user_id);



-- ============================================================
-- TABLE: roles
-- Custom RBAC roles.
```

```sql
-- ============================================================
CREATE TABLE roles (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id        UUID NOT NULL REFERENCES organizations(id) ON
        DELETE CASCADE,
  name          VARCHAR(100) NOT NULL,
  description   TEXT,
  is_system     BOOLEAN NOT NULL DEFAULT FALSE,
  permissions   JSONB NOT NULL DEFAULT '[]',
  created_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  updated_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE UNIQUE INDEX idx_roles_org_name ON roles(org_id, name);
CREATE INDEX idx_roles_permissions ON roles USING GIN
        (permissions);



-- ============================================================
-- TABLE: role_assignments
-- Assigns roles to users within an org (for custom RBAC).
-- ============================================================
CREATE TABLE role_assignments (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id        UUID NOT NULL,
  user_id       UUID NOT NULL,
  role_id       UUID NOT NULL REFERENCES roles(id) ON DELETE
        CASCADE,
  resource_type VARCHAR(50),
  resource_id   UUID,
  granted_by    UUID NOT NULL,
  granted_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  expires_at    TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_role_assignments_org_user ON
        role_assignments(org_id, user_id);
CREATE INDEX idx_role_assignments_resource ON
        role_assignments(resource_type, resource_id)
  WHERE resource_type IS NOT NULL;



-- ============================================================
-- TABLE: invitations
-- Pending organization invitations.
-- ============================================================
```

```
CREATE TABLE invitations (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id          UUID NOT NULL REFERENCES organizations(id) ON
        DELETE CASCADE,
  email           CITEXT NOT NULL,
  role            VARCHAR(20) NOT NULL DEFAULT 'member',
  team_ids        UUID[] DEFAULT '{}',
  invited_by      UUID NOT NULL,
  invitation_token VARCHAR(255) NOT NULL UNIQUE,
  message         TEXT,
  status          VARCHAR(20) NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending', 'accepted',
        'declined', 'expired', 'cancelled')),
  accepted_at     TIMESTAMP WITH TIME ZONE,
  declined_at     TIMESTAMP WITH TIME ZONE,
  expires_at      TIMESTAMP WITH TIME ZONE NOT NULL,
  created_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_invitations_org_email ON invitations(org_id,
        email)
  WHERE status = 'pending';
CREATE INDEX idx_invitations_token ON
        invitations(invitation_token);
CREATE INDEX idx_invitations_expires_at ON invitations(expires_at)
  WHERE status = 'pending';
```

17.10 audit_db

> **RESOLVED (this increment):** The hash chain below now covers **every PII column** in the row (§17.10.1) and privileged-op audit writes are **fail-closed** (§17.10.1) — a write that cannot be durably committed aborts the privileged action it was recording, rather than silently proceeding unaudited.
>
> **DEFERRED, cross-referenced to `05_security_zero_trust`:** The chain below is still an in-table integrity check only — it has no external/independent anchoring (e.g. S3 Object Lock compliance mode, HSM signature, or off-DB notarization). A DB principal with write access to `audit_db` can rewrite the chain, including the genesis row, undetected. Do not read "Immutable" in the comment below as tamper-proof; it is tamper-*evident* against non-privileged writers only. **External WORM anchoring is specified in `05_security_zero_trust` — this document defines**

> **only the DB-side chain, PII coverage, fail-closed durability, and the crypto-shred erasure lifecycle (§17.10.1); it does not re-specify the WORM anchor mechanism owned by doc 05.**

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- ============================================================
-- TABLE: audit_events
-- Immutable, cryptographically-chained audit log.
-- Partitioned by month for efficient time-range queries.
-- ============================================================
CREATE TABLE audit_events (
  id              UUID NOT NULL DEFAULT gen_random_uuid(),
  seq             BIGSERIAL,
  org_id          UUID NOT NULL,
  event_type      VARCHAR(100) NOT NULL,
  user_id         UUID,
  resource_type   VARCHAR(50),
  resource_id     UUID,
  outcome         VARCHAR(20) NOT NULL DEFAULT 'success'
                    CHECK (outcome IN ('success', 'failure',
      'partial')),
  session_id      UUID,
  source_service  VARCHAR(50) NOT NULL,
  metadata        JSONB NOT NULL DEFAULT '{}',
  -- PII envelope encryption (§17.10.1): ip_address/user_agent/
      resource_name are
  -- stored as pgcrypto ciphertext, decryptable only while
      pii_key_id's DEK is
  -- live. pii_key_id is never NULL for a system-generated event
      (see
  -- audit_pii_keys below) and is the sole handle GDPR erasure
      ever touches.
  ip_address      BYTEA,
  user_agent      BYTEA,
  resource_name   BYTEA,
  pii_key_id      UUID NOT NULL REFERENCES audit_pii_keys(id),
  hash            VARCHAR(128) NOT NULL,
  prev_hash       VARCHAR(128),
  occurred_at     TIMESTAMP WITH TIME ZONE NOT NULL,
  recorded_at     TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  PRIMARY KEY (id, occurred_at)
) PARTITION BY RANGE (occurred_at);

CREATE INDEX idx_audit_events_org_id ON audit_events(org_id,
      occurred_at DESC);
```

```
CREATE INDEX idx_audit_events_user_id ON audit_events(user_id,
        occurred_at DESC)
  WHERE user_id IS NOT NULL;
CREATE INDEX idx_audit_events_event_type ON
        audit_events(event_type, occurred_at DESC);
CREATE INDEX idx_audit_events_resource ON
        audit_events(resource_type, resource_id, occurred_at DESC)
  WHERE resource_type IS NOT NULL;
CREATE INDEX idx_audit_events_occurred_at ON audit_events USING
        BRIN (occurred_at);
CREATE INDEX idx_audit_events_metadata ON audit_events USING GIN
        (metadata);
CREATE INDEX idx_audit_events_pii_key_id ON
        audit_events(pii_key_id);

CREATE TABLE audit_events_2026_01 PARTITION OF audit_events
  FOR VALUES FROM ('2026-01-01') TO ('2026-02-01');
CREATE TABLE audit_events_2026_02 PARTITION OF audit_events
  FOR VALUES FROM ('2026-02-01') TO ('2026-03-01');
CREATE TABLE audit_events_2026_03 PARTITION OF audit_events
  FOR VALUES FROM ('2026-03-01') TO ('2026-04-01');
CREATE TABLE audit_events_2026_04 PARTITION OF audit_events
  FOR VALUES FROM ('2026-04-01') TO ('2026-05-01');
CREATE TABLE audit_events_2026_05 PARTITION OF audit_events
  FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');
CREATE TABLE audit_events_2026_06 PARTITION OF audit_events
  FOR VALUES FROM ('2026-06-01') TO ('2026-07-01');
CREATE TABLE audit_events_2026_07 PARTITION OF audit_events
  FOR VALUES FROM ('2026-07-01') TO ('2026-08-01');
CREATE TABLE audit_events_2026_08 PARTITION OF audit_events
  FOR VALUES FROM ('2026-08-01') TO ('2026-09-01');
CREATE TABLE audit_events_2026_09 PARTITION OF audit_events
  FOR VALUES FROM ('2026-09-01') TO ('2026-10-01');
CREATE TABLE audit_events_2026_10 PARTITION OF audit_events
  FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
CREATE TABLE audit_events_2026_11 PARTITION OF audit_events
  FOR VALUES FROM ('2026-11-01') TO ('2026-12-01');
CREATE TABLE audit_events_2026_12 PARTITION OF audit_events
  FOR VALUES FROM ('2026-12-01') TO ('2027-01-01');


-- ============================================================
-- TABLE: audit_event_hash_chain
-- Tracks the chain head for tamper detection.
-- ============================================================
CREATE TABLE audit_event_hash_chain (
```

```
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id        UUID NOT NULL UNIQUE,
  last_event_id UUID NOT NULL,
  last_hash     VARCHAR(128) NOT NULL,
  chain_length  BIGINT NOT NULL DEFAULT 0,
  updated_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);


-- ============================================================
-- TABLE: audit_exports
-- Records of audit log export jobs.
-- ============================================================
CREATE TABLE audit_exports (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id            UUID NOT NULL,
  requested_by      UUID NOT NULL,
  format            VARCHAR(20) NOT NULL CHECK (format IN
        ('json', 'csv', 'syslog')),
  filter_json       JSONB NOT NULL DEFAULT '{}',
  status            VARCHAR(20) NOT NULL DEFAULT 'pending'
                      CHECK (status IN ('pending', 'processing',
        'completed', 'failed')),
  event_count       BIGINT DEFAULT 0,
  file_size_bytes   BIGINT DEFAULT 0,
  storage_path      TEXT,
  download_token    VARCHAR(255),
  download_expires_at TIMESTAMP WITH TIME ZONE,
  error_message     TEXT,
  created_at        TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  completed_at      TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_audit_exports_org_id ON audit_exports(org_id,
        created_at DESC);
```

17.10.1 Full-PII hash chain, fail-closed durability & crypto-shred GDPR erasure

The problem this solves. `audit_events` is append-only and hash-chained (tamper-*evident*, §17.10 deferred-note) — an ordinary `UPDATE`/`DELETE` to satisfy a GDPR Article 17 erasure request would either break the chain (if the row's bytes change, every subsequent `hash` in the chain that folded in the old bytes stops verifying) or require rewriting the entire downstream chain (defeating the append-only guarantee).

`05_security_zero_trust` §9.2's `audit.AnonymizeUser` is specified there as "PII fields are overwritten with anonymized values while preserving event integrity" — this subsection is the concrete mechanism that makes both halves of that sentence true simultaneously.

Design: crypto-shredding, not in-place anonymization. The three PII columns that can identify a natural person independent of `user_id` — `ip_address`, `user_agent`, `resource_name` (which may contain a hostname, filename, or other operator-supplied string) — are stored as **pgcrypto ciphertext**, encrypted with a per-subject Data Encryption Key (DEK) at write time, **before** the row's hash is computed:

```
-- Must be created BEFORE audit_events in migration order (§19.4)
       since
-- audit_events.pii_key_id is a NOT NULL FK into this table.
CREATE TABLE audit_pii_keys (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id            UUID NOT NULL,
  subject_type      VARCHAR(20) NOT NULL DEFAULT 'user'
                      CHECK (subject_type IN ('user',
       'service_account', 'anonymous')),
  subject_user_id   UUID,
  wrapped_dek       BYTEA,              -- NULL once destroyed
       (crypto-shredded)
  wrap_key_ref      VARCHAR(255) NOT NULL, -- org KEK reference
       (vault key ID, §4)
  algorithm         VARCHAR(20) NOT NULL DEFAULT 'aes-256-gcm',
  created_at        TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
       NOW(),
  destroyed_at      TIMESTAMP WITH TIME ZONE,
  destroy_reason    VARCHAR(50) CHECK (destroy_reason IN (
                      'gdpr_erasure_art17', 'key_rotation',
       'org_offboarded'
                     ))
);

CREATE UNIQUE INDEX idx_audit_pii_keys_subject ON
       audit_pii_keys(org_id, subject_user_id)
  WHERE destroyed_at IS NULL AND subject_user_id IS NOT NULL;
CREATE INDEX idx_audit_pii_keys_org_id ON audit_pii_keys(org_id);
```

Write path (every service's audit-writer library, not application code directly):

1. Resolve (or lazily create) the acting subject's live `audit_pii_keys` row for the current org.

2. Encrypt `ip_address`/`user_agent`/`resource_name` with that row's DEK: `pgp_sym_encrypt(value::text, dek, 'cipher-algo=aes256')` → store as `BYTEA`.
3. Compute `hash` over the **canonical, fixed-order concatenation of every column including the ciphertext bytes**: `sha256(org_id || event_type || user_id || resource_type || resource_id || outcome || ip_address_ciphertext || user_agent_ciphertext || resource_name_ciphertext || session_id || source_service || metadata || pii_key_id || occurred_at || prev_hash)`. Because the hash folds in the *ciphertext bytes*, not the plaintext, the hash is computed exactly once, at write time, and never needs recomputation — including after the subject's key is later destroyed.
4. Append to `audit_event_hash_chain` for the org (existing §17.10 mechanism, unchanged).

Erasure path (`audit.AnonymizeUser`, referenced from `05_security_zero_trust` §9.2):

```
-- The entire GDPR Article 17 "anonymize this user's audit trail"
      operation is
-- ONE update to audit_pii_keys. No audit_events row is ever
      touched: no
-- UPDATE, no DELETE, no rewrite — the append-only invariant and
      every
-- existing hash in the chain remain byte-for-byte valid forever.
UPDATE audit_pii_keys
SET wrapped_dek = NULL,
    destroyed_at = NOW(),
    destroy_reason = 'gdpr_erasure_art17'
WHERE org_id = $1 AND subject_user_id = $2 AND destroyed_at IS
      NULL;
```

Once `wrapped_dek` is NULL, `ip_address`/`user_agent`/`resource_name` ciphertext for every past event tied to that `pii_key_id` is **permanently unrecoverable** — the plaintext cannot be reconstructed by anyone, including a database superuser, because the DEK itself no longer exists anywhere (crypto-shredding). This satisfies the erasure obligation without violating either the append-only rule or the hash chain. The erasure operation itself is recorded as a **new** `audit_events` row (event_type = `gdpr_erasure_completed`, its own fresh `pii_key_id`) so the erasure is auditable without itself carrying erasable PII. `user_id` on historical rows is left as a bare UUID (not PII-encrypted) — once the originating `users` row is hard-deleted per `05`'s erasure flow, an

orphaned UUID with no realistic re-identification path is accepted pseudonymous data under GDPR guidance, consistent with `05`'s existing design.

Fail-closed durability for privileged-op audit writes. Every audit write for a *privileged* operation (role/permission change, vault member removal, key revocation, break-glass access, org setting change — the closed set enumerated in `05_security_zero_trust`'s privileged-action list) runs in the **same database transaction** as the privileged action itself:

```
// Fail-closed: the privileged action and its audit record commit
        atomically,
// or neither does. A privileged action MUST NOT be observable if
        its audit
// record failed to persist — the opposite of "fire and forget"
        logging.
func RevokeVaultMember(ctx context.Context, pool *pgxpool.Pool,
        vaultID, userID uuid.UUID) error {
    tx, err := pool.Begin(ctx)
    if err != nil {
        return err
    }
    defer tx.Rollback(ctx)

    if _, err := tx.Exec(ctx, `DELETE FROM vault_members WHERE
        vault_id = $1 AND user_id = $2`, vaultID, userID); err !=
        nil {
        return err
    }
    if err := writeAuditEvent(ctx, tx, "vault.member_revoked",
        vaultID, userID); err != nil {
        return err // rolls back the DELETE too — the privileged
        action never took effect unaudited
    }
    return tx.Commit(ctx) // synchronous_commit=on for this
        connection (below)
}
```

The `audit_db` connection pool used for privileged-op writes runs with `synchronous_commit = on` (the cluster default; never downgraded to `off`/`local` for this pool even though other, non-privileged, high-volume audit paths may accept `synchronous_commit = local` for throughput) — a commit is not acknowledged to the caller until the WAL record is durably flushed, so a host crash immediately after commit cannot silently lose a privileged-op audit record that the caller believes succeeded.

17.11 pki_db

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- ============================================================
-- TABLE: ca_keys
-- Certificate Authority key configurations.
-- ============================================================
CREATE TABLE ca_keys (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id          UUID NOT NULL,
  ca_type         VARCHAR(20) NOT NULL CHECK (ca_type IN ('user',
        'host')),
  version         INTEGER NOT NULL DEFAULT 1,
  public_key_openssh TEXT NOT NULL,
  fingerprint     VARCHAR(512) NOT NULL,
  algorithm       VARCHAR(20) NOT NULL DEFAULT 'ed25519',
  encrypted_private_key_ref TEXT NOT NULL,
  key_manager     VARCHAR(20) NOT NULL DEFAULT 'vault'
                    CHECK (key_manager IN ('vault', 'aws_kms',
        'gcp_kms', 'hsm')),
  active          BOOLEAN NOT NULL DEFAULT TRUE,
  last_used_at    TIMESTAMP WITH TIME ZONE,
  rotated_at      TIMESTAMP WITH TIME ZONE,
  created_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE UNIQUE INDEX idx_ca_keys_org_type_active
        ON ca_keys(org_id, ca_type)
  WHERE active = TRUE;



-- ============================================================
-- TABLE: certificates
-- Issued SSH certificates.
-- ============================================================
CREATE TABLE certificates (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id          UUID NOT NULL,
  ca_key_id       UUID NOT NULL REFERENCES ca_keys(id),
  cert_type       VARCHAR(10) NOT NULL CHECK (cert_type IN
        ('user', 'host')),
  entity_type     VARCHAR(20) NOT NULL,
  entity_id       UUID NOT NULL,
  serial          BIGINT NOT NULL,
  fingerprint     VARCHAR(512) NOT NULL,
```

```
  certificate_openssh TEXT NOT NULL,
  principals      TEXT[] NOT NULL DEFAULT '{}',
  extensions      JSONB DEFAULT '{}',
  critical_options JSONB DEFAULT '{}',
  valid_after     TIMESTAMP WITH TIME ZONE NOT NULL,
  valid_before    TIMESTAMP WITH TIME ZONE NOT NULL,
  revoked         BOOLEAN NOT NULL DEFAULT FALSE,
  revoked_at      TIMESTAMP WITH TIME ZONE,
  revoke_reason   VARCHAR(255),
  revoked_by      UUID,
  created_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE UNIQUE INDEX idx_certificates_serial ON
        certificates(org_id, serial);
CREATE INDEX idx_certificates_entity ON certificates(entity_type,
        entity_id);
CREATE INDEX idx_certificates_valid_before ON
        certificates(valid_before)
  WHERE revoked = FALSE;
CREATE INDEX idx_certificates_fingerprint ON
        certificates(fingerprint);



-- ============================================================
-- TABLE: certificate_revocations
-- Revocation records with reason codes.
-- ============================================================
CREATE TABLE certificate_revocations (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  certificate_id  UUID NOT NULL REFERENCES certificates(id) ON
        DELETE CASCADE,
  org_id          UUID NOT NULL,
  serial          BIGINT NOT NULL,
  reason          VARCHAR(50) NOT NULL
                    CHECK (reason IN (
                      'unspecified', 'key_compromise',
        'ca_compromise',
                      'affiliation_changed', 'superseded',
        'cessation',
                      'certificate_hold', 'remove_from_crl',
        'privilege_withdrawn',
                      'aa_compromise'
                    )),
  revoked_by      UUID NOT NULL,
  revoked_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);
```

```
CREATE INDEX idx_cert_revocations_org_id ON
        certificate_revocations(org_id);
CREATE INDEX idx_cert_revocations_serial ON
        certificate_revocations(serial);



-- ============================================================
-- TABLE: crl_entries
-- Certificate Revocation List (CRL / KRL) entries for fast
        lookup.
-- ============================================================
CREATE TABLE crl_entries (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id        UUID NOT NULL,
  ca_key_id     UUID NOT NULL REFERENCES ca_keys(id),
  serial        BIGINT NOT NULL,
  revoked_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  reason        VARCHAR(50) NOT NULL
);

CREATE UNIQUE INDEX idx_crl_entries_org_serial ON
        crl_entries(org_id, serial);
CREATE INDEX idx_crl_entries_ca_key_id ON crl_entries(ca_key_id);
```

17.12 Automated Partition Management (pg_partman)

The problem this solves. Every `RANGE`-partitioned table in §17 (`audit_events`, `vault_audit_events`, `host_connection_history`, `key_usage_log`, `ssh_sessions`, `session_events`, `sftp_transfers`, `snippet_executions`, `collaboration_events`) has its partitions **hand-created** with an explicit end date (§17.10 shows `audit_events` created through `2026-12-01` to `2027-01-01`; several others stop at `2026_q3`/`2026_q4`). Once the last hand-created partition's range is exhausted, an `INSERT` for a timestamp beyond it **fails outright** (no default partition is declared) — a silent, entirely foreseeable outage that shows up as "audit logging stopped working" or "SSH sessions can no longer be recorded" on whatever date the last partition's range ends. `pg_partman` (the standard PostgreSQL extension for this exact problem, already assumed available on PostgreSQL 17.2 per RDS/Cloud SQL/self-managed images) automates future-partition creation and past-partition retention so the hand-maintained end date never needs to exist.

Installation + per-table registration (run once per database that owns partitioned tables — `audit_db`, `vault_db`, `host_db`, `keychain_db`, `session_db`, `snippet_db`, `collab_db`):

```
CREATE EXTENSION IF NOT EXISTS pg_partman;
CREATE SCHEMA IF NOT EXISTS partman;
-- (pg_partman's control tables live in the `partman` schema by
        convention)

-- audit_db: audit_events — monthly, matching the existing hand-
        created scheme
SELECT partman.create_parent(
  p_parent_table   => 'public.audit_events',
  p_control        => 'occurred_at',
  p_interval       => 'monthly',
  p_premake        => 4     -- always keep 4 months of future
        partitions pre-created
);
UPDATE partman.part_config
  SET retention           = '7 years',     --
        audit_log_retention_days default is 365d;
                                           -- 7y is the SOC2/PCI-
        typical floor — retention

        -- is a per-org policy read at maintenance time
                                           -- via the trigger
        function below, not a fixed
                                           -- literal, when an
        org's configured retention
                                           -- exceeds this floor
      retention_keep_table = true,         -- detach, never DROP
        — audit data is never

        -- silently destroyed by a maintenance job
      infinite_time_partitions = true
  WHERE parent_table = 'public.audit_events';

-- vault_db: vault_audit_events — quarterly, matching the existing
        scheme
SELECT partman.create_parent('public.vault_audit_events',
        'occurred_at', 'quarterly', p_premake => 4);

-- host_db: host_connection_history — quarterly
SELECT partman.create_parent('public.host_connection_history',
        'started_at', 'quarterly', p_premake => 4);

-- keychain_db: key_usage_log — quarterly
SELECT partman.create_parent('public.key_usage_log',
        'occurred_at', 'quarterly', p_premake => 4);
```

```
-- session_db: ssh_sessions, session_events, sftp_transfers —
       quarterly
SELECT partman.create_parent('public.ssh_sessions', 'started_at',
        'quarterly', p_premake => 4);
SELECT partman.create_parent('public.session_events',
        'occurred_at', 'quarterly', p_premake => 4);
SELECT partman.create_parent('public.sftp_transfers',
        'transferred_at', 'quarterly', p_premake => 4);

-- snippet_db: snippet_executions — quarterly
SELECT partman.create_parent('public.snippet_executions',
        'started_at', 'quarterly', p_premake => 4);

-- collab_db: collaboration_events — quarterly
SELECT partman.create_parent('public.collaboration_events',
        'occurred_at', 'quarterly', p_premake => 4);
```

Scheduled maintenance — `pg_partman` only creates/drops partitions when its maintenance procedure actually runs; it does not run itself. Scheduled hourly via `pg_cron` (co-located in each database):

```
CREATE EXTENSION IF NOT EXISTS pg_cron;
SELECT cron.schedule('partman-maintenance', '0 * * * *', $$CALL
        partman.run_maintenance_proc()$$);
```

An hourly cadence keeps the "next partition doesn't exist yet" window to at most one hour even under a `pg_cron` outage that is caught and paged the same day — compare to a hand-created scheme whose failure window is silent until the multi-month-away end date arrives.

Per-org retention override.
`organizations.audit_log_retention_days` / `.session_recording_retention_days` (§17.9) are the source of truth for how long an individual org's data must be kept; `pg_partman`'s `retention` column above is a **floor** (the shortest time any org's partitions are kept before even being considered for detach), not the enforcement mechanism for per-org retention — a nightly job reads each org's actual configured retention and issues a per-partition, per-org check before any detached partition is finally dropped from storage (never automatic-and-unconditional), so a longer-retention org's data inside an otherwise-eligible-for-drop partition is never destroyed early. `retention_keep_table = true` above additionally means the default action is always **detach** (the partition becomes an ordinary, still-queryable table, just no longer attached to live `INSERT`/read-path routing) rather than `DROP`, giving every retention decision a manual-recovery window before an operator-approved archival step actually deletes the detached table's storage.

Anti-bluff verification. A post-build gate (§11.4.4(b) layer 2) asserts, for every table in this section, that a partition covering `NOW() + p_premake` intervals already exists — i.e. it fails loudly in CI/staging **before** the failure mode is "customers cannot connect because SSH session recording just started throwing insert errors in production."

---

18. Redis Data Structures

Redis 8 is used for ephemeral, high-throughput data: session state, rate limiting, distributed locks, caching, pub/sub, and real-time presence. All keys use a `{service}:{type}:{identifier}` namespace convention to enable Redis Cluster key sharding where appropriate.

Keys are prefixed by service to enable logical separation. TTLs are always set; no key should be persisted indefinitely without explicit review.

18.1 Session Storage

Active SSH Session State

Key: `session:{sessionId}:state`
Type: Hash
TTL: 24 hours (refreshed on activity)
Purpose: Store the current state of an active SSH session for the proxy service.

```
HSET session:sess-550e8400:state
  user_id         "550e8400-e29b-41d4-a716-446655440000"
  host_id         "host-550e8400-0000-0000-0000-aabbccddeeff"
  org_id          "org-550e8400-0000-0000-0000-000000000001"
  status          "connected"
  cols            "220"
  rows            "50"
  terminal_type   "xterm-256color"
  recording       "true"
  collab          "false"
  proxy_node      "proxy-node-2"
  started_at      "1751120400"
  last_activity   "1751125200"
```

```
EXPIRE session:sess-550e8400:state 86400
```

Session Token → Session ID Mapping

Key: `session_token:{tokenHash}:session`
Type: String
TTL: 24 hours
Purpose: Validate WebSocket session tokens quickly without database lookup.

```
SET session_token:sha256abc123:session
"sess-550e8400-0000-0000-0000-aabbccddeeff"
EX 86400
```

User Active Sessions Index

Key: `user:{userId}:active_sessions`
Type: Set
TTL: 24 hours (refreshed on session activity)
Purpose: Track all active session IDs for a user (for `GET /api/v1/sessions` fast path and session limits).

```
SADD user:550e8400:active_sessions "sess-550e8400-aaa"
"sess-550e8400-bbb"
EXPIRE user:550e8400:active_sessions 86400
```

18.2 Authentication Cache

JWT Blocklist

Key: `jwt:blocklist:{jti}`
Type: String (value: reason)
TTL: Set to token's remaining lifetime
Purpose: O(1) lookup for revoked JWT tokens. Backed by `jwt_blocklist` table for persistence.

```
SET jwt:blocklist:jti-550e8400-unique "user_logout"
EXAT 1751120400
```

Auth Rate Limit Counters (Sliding Window)

Key: `rl:auth:{ipAddress}:{windowMinute}`
Type: String (integer counter)
TTL: 2 minutes (covers current + previous window)
Purpose: Rate limit login/register endpoints per IP.

```
INCR rl:auth:192.168.1.10:29185440
EXPIRE rl:auth:192.168.1.10:29185440 120

windowMinute = floor(unix_timestamp / 60)
```

General Rate Limit Counter (User)

Key: `rl:user:{userId}:{windowMinute}`
Type: String (integer counter)
TTL: 2 minutes
Purpose: Rate limit per authenticated user.

```
INCR rl:user:550e8400:29185440
EXPIRE rl:user:550e8400:29185440 120
```

AI Endpoint Rate Limit (Hourly)

Key: `rl:ai:{userId}:{windowHour}`
Type: String (integer counter)
TTL: 2 hours
Purpose: Rate limit AI service usage (100 req/hour per user).

```
INCR rl:ai:550e8400:485757
EXPIRE rl:ai:550e8400:485757 7200

windowHour = floor(unix_timestamp / 3600)
```

MFA Challenge State

Key: `mfa:challenge:{challengeId}`
Type: Hash
TTL: 5 minutes
Purpose: Store MFA challenge context during the login flow.

```
HSET mfa:challenge:mfa-chal-550e8400
  user_id     "550e8400-e29b-41d4-a716-446655440000"
```

```
  method      "totp"
  stage       "pending"
  session_id  "temp-session-id"
  ip          "192.168.1.10"
  attempts    "0"
  created_at  "1751125200"

EXPIRE mfa:challenge:mfa-chal-550e8400 300
```

FIDO2 Authentication Challenge

Key: `fido2:challenge:{challengeId}`
Type: Hash
TTL: 60 seconds
Purpose: Store FIDO2 WebAuthn challenge bytes and expected state.

```
HSET fido2:challenge:fido2-chal-550e8400
  challenge_b64   "bGV0J3MgdGVzdA=="
  user_id         "550e8400-e29b-41d4-a716-446655440000"
  rp_id           "helixterminator.io"
  user_verification "preferred"
  mfa_challenge_id "mfa-chal-550e8400"

EXPIRE fido2:challenge:fido2-chal-550e8400 60
```

TOTP Setup Token

Key: `totp:setup:{setupToken}`
Type: Hash
TTL: 10 minutes
Purpose: Temporarily store TOTP secret during setup (not persisted until verified).

```
HSET totp:setup:setup-token-xyz
  user_id       "550e8400-e29b-41d4-a716-446655440000"
  encrypted_secret "AES256GCM:base64..."
  algorithm     "SHA1"
  digits        "6"
  period        "30"

EXPIRE totp:setup:setup-token-xyz 600
```

18.3 Vault Sync Cursors

Vault Sync Cursor Cache

Key: `vault:{vaultId}:sync:{clientId}:cursor`
Type: String
TTL: 7 days
Purpose: Fast lookup of client sync position without hitting PostgreSQL.

```
SET vault:vault-550e8400:sync:client-abc123:cursor
"eyJzZXEiOjE0Mjh9"
EX 604800
```

Vault Version Counter

Key: `vault:{vaultId}:version`
Type: String (integer)
TTL: None (persistent until vault deleted)
Purpose: Incrementing server version for optimistic concurrency control.

```
INCR vault:vault-550e8400:version
```

Vault Change Notification Stream

Key: `vault:{vaultId}:changes`
Type: Stream
TTL: 48 hours (maxlen 10000)
Purpose: Notify connected WebSocket clients of vault changes for real-time sync.

```
XADD vault:vault-550e8400:changes MAXLEN ~ 10000 *
  item_id   "item-550e8400-aaa"
  operation "upsert"
  version   "6"
  changed_by "550e8400-user"
  timestamp "1751125200"
```

18.4 SSH Connection State

SSH Session Lock

Key: `lock:session:{sessionId}`
Type: String (lock value = node ID)
TTL: 30 seconds (refreshed by lock holder)
Purpose: Distributed mutex to prevent concurrent operations on the same session (e.g., resize while terminating).

```
SET lock:session:sess-550e8400 "proxy-node-2:1751125200" NX EX 30
```

Renewal: `SET lock:session:sess-550e8400 "proxy-node-2:1751125200" XX EX 30`

SSH Proxy Node Registration

Key: `proxy:node:{nodeId}:info`
Type: Hash
TTL: 60 seconds (heartbeat)
Purpose: Register proxy nodes for load balancing and session routing.

```
HSET proxy:node:proxy-node-2:info
  address       "10.0.10.2:8086"
  active_sessions "47"
  max_sessions  "200"
  cpu_percent   "23"
  region        "us-east-1"
  version       "1.2.3"

EXPIRE proxy:node:proxy-node-2:info 60
```

Active Proxy Nodes Index

Key: `proxy:nodes`
Type: Sorted Set (score = last heartbeat timestamp)
TTL: None (managed by heartbeat)
Purpose: Discover available proxy nodes for session routing.

```
ZADD proxy:nodes 1751125200 "proxy-node-2"
ZADD proxy:nodes 1751125195 "proxy-node-1"
```

Remove stale nodes: `ZREMRANGEBYSCORE proxy:nodes -inf (unix_now - 90)`

Port Forward Active Connection

Key: `portfwd:{ruleId}:connection`
Type: Hash
TTL: Duration of connection + 300 seconds
Purpose: Track active port forwarding connection metadata.

```
HSET portfwd:pf-550e8400:connection
  connection_id   "pfconn-550e8400-aaa"
  ssh_session_id  "sess-550e8400-bbb"
  proxy_node      "proxy-node-2"
  local_port      "16379"
  started_at      "1751125200"
  bytes_sent      "0"
  bytes_received  "0"
```

18.5 User Presence

Online User Presence

Key: `presence:org:{orgId}:online`
Type: Sorted Set (score = last seen timestamp)
TTL: None (managed by activity updates)
Purpose: Track which users are currently online for org-level presence indicators.

```
ZADD presence:org:org-550e8400:online 1751125200 "user:550e8400-alice"
ZADD presence:org:org-550e8400:online 1751125190 "user:660e8400-bob"
```

Mark offline: `ZREM presence:org:org-550e8400:online "user:550e8400-alice"`
Get online users: `ZRANGEBYSCORE presence:org:org-550e8400:online (unix_now - 300) +inf`

User Activity Heartbeat

Key: `presence:user:{userId}:heartbeat`
Type: String (value = ISO8601 timestamp)
TTL: 5 minutes
Purpose: Per-user heartbeat; expiry = user went offline.

```
SET presence:user:550e8400:heartbeat "2026-06-28T17:40:00Z"
EX 300
```

18.6 Notification Queues

User Notification Queue

Key: `notifications:user:{userId}:queue`
Type: List (LPUSH producer, BRPOP consumer)
TTL: 7 days
Purpose: Queue notifications for delivery to connected WebSocket clients.

```
LPUSH notifications:user:550e8400:queue
  '{"id":"notif-abc","type":"session_started","data":
{"host":"prod-web-01"},"ts":1751125200}'

EXPIRE notifications:user:550e8400:queue 604800
```

Broadcast Message Queue

Key: `broadcast:session:{sessionId}`
Type: List
TTL: 60 seconds
Purpose: Queue broadcast commands for SSH sessions (for `POST /api/v1/sessions/{sessionId}/broadcast`).

```
LPUSH broadcast:session:sess-550e8400:queue

'{"data":"c3VkbyBzeXN0ZW1jdGwgcmVzdGFydCBuZ2lueAo=","from":"user-
alice"}'
EXPIRE broadcast:session:sess-550e8400:queue 60
```

18.7 Distributed Locks

Idempotency Key Cache

Key: `idempotency:{userId}:{idempotencyKey}`
Type: String (JSON response body)
TTL: 24 hours
Purpose: Cache responses for idempotent requests to prevent duplicate processing.

```
SET idempotency:550e8400:550e8400-idempotency-key
  '{"status":201,"body":{"id":"host-abc","name":"prod-
web-01",...}}'
EX 86400
```

Global Distributed Lock (Generic)

Key: `lock:global:{resourceType}:{resourceId}`
Type: String (lock token)
TTL: Variable (5–60 seconds)
Purpose: Prevent concurrent writes to the same resource across microservices.

```
SET lock:global:vault:vault-550e8400 "lock-token-nonce-xyz" NX EX
30
```

18.8 Caching

User Context Cache

Key: `cache:user:{userId}:context`
Type: Hash
TTL: 60 seconds
Purpose: Cache user org/permissions context for incoming requests (reduces auth_db round-trips).

```
HSET cache:user:550e8400:context
  org_id          "org-550e8400"
  org_role        "admin"
  team_ids        '["team-abc","team-def"]'
  enforce_mfa     "true"
  status          "active"
  cached_at       "1751125200"
```

```
EXPIRE cache:user:550e8400:context 60
```

Vault Member Cache

Key: `cache:vault:{vaultId}:member:{userId}`
Type: Hash
TTL: 30 seconds
Purpose: Cache vault membership for permission checks.

```
HSET cache:vault:vault-550e8400:member:550e8400
  permission  "admin"
  is_owner    "true"
  cached_at   "1751125200"

EXPIRE cache:vault:vault-550e8400:member:550e8400 30
```

Host List Cache (per vault)

Key: `cache:vault:{vaultId}:hosts:page:{cursor}`
Type: String (JSON)
TTL: 10 seconds
Purpose: Cache paginated host list results to reduce DB load on frequently-accessed vaults.

```
SET cache:vault:vault-550e8400:hosts:page:cursor_abc
  '{"data":[...],"pagination":{...}}'
EX 10
```

SSH Host Fingerprint Cache

Key: `cache:host:{hostId}:fingerprint`
Type: Hash
TTL: 1 hour
Purpose: Cache known SSH fingerprints to avoid DB lookups on every connection.

```
HSET cache:host:host-550e8400:fingerprint
  sha256    "SHA256:abc123def456..."
  ed25519   "ssh-ed25519 AAAA..."
  verified  "true"
```

```
EXPIRE cache:host:host-550e8400:fingerprint 3600
```

18.9 SFTP Session State

Key: `sftp:{sftpSessionId}:state`
Type: Hash
TTL: 6 hours
Purpose: Track SFTP session state and CWD.

```
HSET sftp:sftp-550e8400:state
  user_id     "550e8400-user"
  host_id     "host-550e8400"
  cwd         "/var/www/html"
  status      "connected"
  proxy_node  "proxy-node-2"
  started_at  "1751125200"

EXPIRE sftp:sftp-550e8400:state 21600
```

18.10 Collaboration State

Key: `collab:{collabId}:participants`
Type: Hash (field = userId, value = JSON participant state)
TTL: 24 hours
Purpose: Track active collaboration participants for WebSocket routing.

```
HSET collab:collab-550e8400:participants
  "660e8400-bob" '{"name":"Bob
Jones","role":"viewer","color":"#FF6B35","connected_at":1751125200}
  "770e8400-carol" '{"name":"Carol
White","role":"contributor","color":"#4ECDC4","connected_at":175112

EXPIRE collab:collab-550e8400:participants 86400
```

18.11 Snippet Execution State

Key: `snippetexec:{executionId}:state`
Type: Hash
TTL: 2 hours
Purpose: Track multi-host snippet execution progress for real-time status updates.

```
HSET snippetexec:exec-550e8400:state
  snippet_id    "snip-550e8400"
  status        "running"
  total         "20"
  completed     "8"
  failed        "1"
  started_at    "1751125200"

EXPIRE snippetexec:exec-550e8400:state 7200
```

18.12 WebSocket Connection Registry

Key: `ws:user:{userId}:connections`
Type: Set (value = connection ID strings)
TTL: 24 hours
Purpose: Track all WebSocket connections for a user to enable broadcasting.

```
SADD ws:user:550e8400:connections "ws-conn-abc-proxy-1" "ws-conn-
def-proxy-2"
EXPIRE ws:user:550e8400:connections 86400
```

18.13 Persistence Configuration + Cluster Hash-Tags

Redis persistence (AOF/RDB) is configured **per instance/cluster**, not per key — there is no Redis feature to durably persist one key while treating a sibling key as pure ephemeral cache. §18 marks three keyspaces `TTL: None`: `vault:{vaultId}:version` (§18.3), `proxy:nodes` (§18.4), and `presence:org:{orgId}:online` (§18.5). Every other keyspace in §18 carries an explicit TTL and is, by design, safe to lose on a Redis restart (the client/server simply re-derives or re-requests it). This subsection specifies the durability configuration for the cluster these

three load-bearing keyspaces live in, and the cluster hash-tag convention needed for them to work correctly under Redis Cluster in the first place.

Persistence configuration (applies to the Redis 8 cluster/replica-set backing all keyspaces in this document — a single cluster, not a special-purpose one, since AOF's overhead is negligible relative to the throughput headroom already budgeted for session/rate-limit traffic):

```
# redis.conf — durability
appendonly yes
appendfsync everysec
        # ~1s worst-case data loss window on an unclean
                                # shutdown; `always` is not used
        here because it
                                # would add fsync latency to every
        rate-limit

        # INCR and session heartbeat in the same
                                # keyspace, and the three load-
        bearing keys
                                # below are either reconciled from
        PostgreSQL
                                # (vault:version) or self-healing
        from a live

        # heartbeat (proxy:nodes, presence) within one
                                # heartbeat interval regardless
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 64mb

# RDB snapshots as a secondary, fast-full-restore mechanism (not
        the primary
# durability guarantee — AOF is)
save 900 1
save 300 10
save 60 10000
rdb-key-save-delay 0
```

Startup / failover reconciliation (defense in depth beyond AOF). Even with `appendfsync everysec`, an unclean shutdown can lose up to ~1 second of the most recent writes. Because `vault:{vaultId}:version` is a cache of the authoritative `vaults.version` column (§17.2) — not its source of truth — the `vault-service` runs a reconciliation pass on startup and on every Redis failover-promotion event:

```
-- For every vault whose Redis key is missing (or, defensively,
       whose cached
-- value is LOWER than the DB value — never higher, which would
       indicate a
-- Redis-side write the DB never saw and is itself a paged
       incident):
SELECT id, version FROM vaults WHERE deleted_at IS NULL;

-- one SET per vault, only if the Redis value is absent or stale-
low:
SET vault:{<vault-uuid>}:version <db_version>
```

`proxy:nodes` and `presence:org:{orgId}:online` require no reconciliation query — every proxy node and every connected client re-registers via its next heartbeat/activity ping (§18.4, §18.5) within their existing heartbeat interval (60s / 5min respectively), so a lost Redis write here self-heals without any explicit recovery code path; they are included in the persistence configuration above purely to shorten that self-heal window, not because their loss is otherwise unrecoverable.

Cluster hash-tags. Under Redis Cluster, keys are sharded across slots by hashing the key name — a Lua script or `MULTI`/`EXEC` transaction touching two keys that hash to different slots fails with a `CROSSSLOT` error. Several operations in §18 touch multiple keys for the same vault/session atomically (e.g. the vault sync push path reads `vault:{vaultId}:version`, appends to `vault:{vaultId}:changes`, and updates `vault:{vaultId}:sync:{clientId}:cursor` in one Lua script). Every key namespaced by a given `vaultId` or `sessionId` therefore uses the **Redis Cluster hash-tag** syntax — the substring inside `{…}` is the only part of the key Redis Cluster hashes to choose a slot — so all keys sharing that tag always land on the same slot/node regardless of the rest of the key name:

```
vault:{a1b2c3d4-e5f6-47a8-9b0c-1d2e3f4a5b6c}:version
vault:{a1b2c3d4-e5f6-47a8-9b0c-1d2e3f4a5b6c}:changes
vault:{a1b2c3d4-e5f6-47a8-9b0c-1d2e3f4a5b6c}:sync:client-
abc123:cursor
```

The same convention applies to every `session:{sessionId}:*` / `lock:session:{sessionId}` key family (§18.1, §18.4) and every `portfwd:{ruleId}:*` key family (§18.4), so session-resize/terminate and port-forward start/stop — each of which touches more than one key for the same entity — remain atomic under Cluster mode. Single-key operations (rate limiters, blocklist entries, presence) are unaffected and need no hash-tag.

---

19. Database Migrations

HelixTerminator uses `golang-migrate/migrate` for all database schema migrations. Each microservice manages its own migrations in `internal/db/{service}/migrations/`.

19.1 Migration Naming Convention

`{NNNNNN}_{description}.{up|down}.sql`

- `NNNNNN`: Zero-padded 6-digit sequence number
- `description`: Snake_case description of the migration
- `.up.sql`: Forward migration (apply)
- `.down.sql`: Reverse migration (rollback)

Examples:

```
000001_create_users.up.sql
000001_create_users.down.sql
000002_create_sessions.up.sql
000002_create_sessions.down.sql
000003_add_mfa_tables.up.sql
000003_add_mfa_tables.down.sql
```

19.2 Migration Tool Configuration

`cmd/migrate/main.go`:

```go
package main

import (
    "flag"
    "fmt"
    "log"
    "os"

    "github.com/golang-migrate/migrate/v4"
    _ "github.com/golang-migrate/migrate/v4/database/postgres"
    _ "github.com/golang-migrate/migrate/v4/source/file"
)

func main() {
    service := flag.String("service", "", "Service name (auth,
        vault, host, etc.)")
```

```
    action  := flag.String("action", "up", "Action: up, down,
        version, force")
    steps   := flag.Int("steps", 0, "Number of steps (0 = all)")
    flag.Parse()

    dsn := os.Getenv(fmt.Sprintf("DB_%s_URL",
        strings.ToUpper(*service)))
    migrationsPath := fmt.Sprintf("file://internal/db/%s/
        migrations", *service)

    m, err := migrate.New(migrationsPath, dsn)
    if err != nil {
        log.Fatalf("Failed to init migrations: %v", err)
    }
    defer m.Close()

    switch *action {
    case "up":
        if *steps > 0 {
            err = m.Steps(*steps)
        } else {
            err = m.Up()
        }
    case "down":
        if *steps > 0 {
            err = m.Steps(-(*steps))
        } else {
            err = m.Down()
        }
    case "version":
        v, dirty, _ := m.Version()
        fmt.Printf("Version: %d, Dirty: %v\n", v, dirty)
    }

    if err != nil && err != migrate.ErrNoChange {
        log.Fatalf("Migration failed: %v", err)
    }
    log.Println("Migration completed successfully")
}
```

19.3 Zero-Downtime Migration Strategy

Phase 1 — Expand (backward-compatible schema change): Apply the migration while old code is still running. The new column/table must be nullable or have a default value.

Phase 2 — Deploy new application code: Deploy the new application version that reads/writes to both old and new schema.

Phase 3 — Contract (remove old columns/tables): After all instances of the old code are gone, apply the cleanup migration to remove deprecated columns.

This pattern (expand-contract / parallel change) ensures: - No downtime during migrations - Rollback is always possible - Zero-error deployments

Non-destructive migrations (always safe): - `ADD COLUMN ... DEFAULT ...` - `CREATE INDEX CONCURRENTLY` - `CREATE TABLE IF NOT EXISTS` - `ADD CONSTRAINT ... NOT VALID` → then `VALIDATE CONSTRAINT` separately

Risky migrations (require extra caution): - `DROP COLUMN` (expand-contract required) - `ALTER COLUMN ... TYPE` (use new column + backfill + rename) - `ADD NOT NULL` constraint without default (add nullable first, backfill, then add constraint) - `CREATE INDEX` without `CONCURRENTLY` (blocks writes)

19.4 Migration Files

auth_db Migrations

000001_create_extensions.up.sql

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";
CREATE EXTENSION IF NOT EXISTS "citext";
```

000001_create_extensions.down.sql

```
DROP EXTENSION IF EXISTS "citext";
DROP EXTENSION IF EXISTS "pg_trgm";
DROP EXTENSION IF EXISTS "pgcrypto";
```

000002_create_users.up.sql

```sql
CREATE OR REPLACE FUNCTION trigger_set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TABLE users (
  id                          UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
  email                       CITEXT NOT NULL,
  email_verified_at           TIMESTAMP WITH TIME ZONE,
  email_pending               CITEXT,
  email_pending_token         VARCHAR(255),
  email_pending_expires_at    TIMESTAMP WITH TIME ZONE,
  password_hash               VARCHAR(255),
  display_name                VARCHAR(100) NOT NULL,
  avatar_url                  TEXT,
  bio                         TEXT,
  locale                      VARCHAR(20) NOT NULL DEFAULT 'en-US',
  timezone                    VARCHAR(100) NOT NULL DEFAULT 'UTC',
  status                      VARCHAR(20) NOT NULL DEFAULT 'active'
                                CHECK (status IN ('active',
        'suspended', 'pending_deletion', 'deleted')),
  failed_login_attempts       INTEGER NOT NULL DEFAULT 0,
  locked_until                TIMESTAMP WITH TIME ZONE,
  last_login_at               TIMESTAMP WITH TIME ZONE,
  last_login_ip               INET,
  password_changed_at         TIMESTAMP WITH TIME ZONE,
  terms_accepted_at           TIMESTAMP WITH TIME ZONE,
  terms_version               VARCHAR(20),
  deletion_requested_at       TIMESTAMP WITH TIME ZONE,
  deletion_scheduled_at       TIMESTAMP WITH TIME ZONE,
  created_at                  TIMESTAMP WITH TIME ZONE NOT NULL
        DEFAULT NOW(),
  updated_at                  TIMESTAMP WITH TIME ZONE NOT NULL
        DEFAULT NOW(),
  deleted_at                  TIMESTAMP WITH TIME ZONE
);

CREATE UNIQUE INDEX idx_users_email ON users(email) WHERE
        deleted_at IS NULL;
CREATE INDEX idx_users_status ON users(status) WHERE deleted_at
        IS NULL;
CREATE INDEX idx_users_created_at ON users USING BRIN
        (created_at);
```

```
CREATE TRIGGER trg_users_updated_at
  BEFORE UPDATE ON users
  FOR EACH ROW EXECUTE FUNCTION trigger_set_updated_at();
```

000002_create_users.down.sql

```
DROP TRIGGER IF EXISTS trg_users_updated_at ON users;
DROP TABLE IF EXISTS users;
DROP FUNCTION IF EXISTS trigger_set_updated_at();
```

000003_create_user_sessions.up.sql

```
CREATE TABLE user_sessions (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
  device_id         UUID,
  ip_address        INET NOT NULL,
  user_agent        TEXT,
  location_city     VARCHAR(100),
  location_country  VARCHAR(10),
  mfa_verified      BOOLEAN NOT NULL DEFAULT FALSE,
  mfa_method        VARCHAR(20),
  status            VARCHAR(20) NOT NULL DEFAULT 'active'
                      CHECK (status IN ('active', 'expired',
        'revoked')),
  revoked_reason    VARCHAR(100),
  last_active_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  created_at        TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  expires_at        TIMESTAMP WITH TIME ZONE NOT NULL
);

CREATE INDEX idx_user_sessions_user_id ON user_sessions(user_id);
CREATE INDEX idx_user_sessions_status ON user_sessions(status,
        expires_at)
  WHERE status = 'active';
```

000003_create_user_sessions.down.sql

```
DROP TABLE IF EXISTS user_sessions;
```

000004_create_refresh_tokens.up.sql

```sql
CREATE TABLE refresh_tokens (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id      UUID NOT NULL REFERENCES user_sessions(id) ON
        DELETE CASCADE,
  user_id         UUID NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
  token_hash      VARCHAR(255) NOT NULL UNIQUE,
  family          UUID NOT NULL,
  generation      INTEGER NOT NULL DEFAULT 1,
  ip_address      INET,
  user_agent      TEXT,
  used_at         TIMESTAMP WITH TIME ZONE,
  revoked         BOOLEAN NOT NULL DEFAULT FALSE,
  revoked_at      TIMESTAMP WITH TIME ZONE,
  revoked_reason  VARCHAR(100),
  created_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  expires_at      TIMESTAMP WITH TIME ZONE NOT NULL
);

CREATE INDEX idx_refresh_tokens_token_hash ON
        refresh_tokens(token_hash);
CREATE INDEX idx_refresh_tokens_session_id ON
        refresh_tokens(session_id);
CREATE INDEX idx_refresh_tokens_family ON refresh_tokens(family);
```

000004_create_refresh_tokens.down.sql

```sql
DROP TABLE IF EXISTS refresh_tokens;
```

000005_create_device_tokens.up.sql

```sql
CREATE TABLE device_tokens (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id          UUID NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
  name             VARCHAR(255) NOT NULL,
  fingerprint      VARCHAR(512) NOT NULL,
  platform         VARCHAR(255),
  user_agent       TEXT,
  trusted          BOOLEAN NOT NULL DEFAULT TRUE,
  last_seen_at     TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  last_seen_ip     INET,
  revoked          BOOLEAN NOT NULL DEFAULT FALSE,
  revoked_at       TIMESTAMP WITH TIME ZONE,
  created_at       TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);
```

```
CREATE UNIQUE INDEX idx_device_tokens_user_fingerprint
  ON device_tokens(user_id, fingerprint) WHERE revoked = FALSE;
CREATE INDEX idx_device_tokens_user_id ON device_tokens(user_id);
```

000005_create_device_tokens.down.sql

```
DROP TABLE IF EXISTS device_tokens;
```

000006_create_mfa_tables.up.sql

```
CREATE TABLE mfa_totp_credentials (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id          UUID NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
  encrypted_secret TEXT NOT NULL,
  issuer           VARCHAR(100) NOT NULL DEFAULT
        'HelixTerminator',
  algorithm        VARCHAR(20) NOT NULL DEFAULT 'SHA1',
  digits           INTEGER NOT NULL DEFAULT 6,
  period           INTEGER NOT NULL DEFAULT 30,
  enabled          BOOLEAN NOT NULL DEFAULT TRUE,
  last_used_at     TIMESTAMP WITH TIME ZONE,
  last_used_code   VARCHAR(10),
  created_at       TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  updated_at       TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE UNIQUE INDEX idx_mfa_totp_user_enabled ON
        mfa_totp_credentials(user_id)
  WHERE enabled = TRUE;

CREATE TABLE mfa_totp_backup_codes (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id      UUID NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
  code_hash    VARCHAR(255) NOT NULL,
  used         BOOLEAN NOT NULL DEFAULT FALSE,
  used_at      TIMESTAMP WITH TIME ZONE,
  used_ip      INET,
  created_at   TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_backup_codes_user_id ON
        mfa_totp_backup_codes(user_id) WHERE used = FALSE;
```

```
CREATE TABLE mfa_fido2_credentials (
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id               UUID NOT NULL REFERENCES users(id) ON
        DELETE CASCADE,
  credential_id         BYTEA NOT NULL UNIQUE,
  credential_id_b64     TEXT NOT NULL,
  name                  VARCHAR(255) NOT NULL,
  public_key_cbor       BYTEA NOT NULL,
  aaguid                UUID,
  sign_count            BIGINT NOT NULL DEFAULT 0,
  transports            TEXT[] DEFAULT '{}',
  backup_eligible       BOOLEAN NOT NULL DEFAULT FALSE,
  backup_state          BOOLEAN NOT NULL DEFAULT FALSE,
  attestation_type      VARCHAR(50),
  attestation_data      JSONB,
  last_used_at          TIMESTAMP WITH TIME ZONE,
  enabled               BOOLEAN NOT NULL DEFAULT TRUE,
  created_at            TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  updated_at            TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW()
);

CREATE INDEX idx_fido2_user_id ON mfa_fido2_credentials(user_id)
        WHERE enabled = TRUE;
```

000006_create_mfa_tables.down.sql

```
DROP TABLE IF EXISTS mfa_fido2_credentials;
DROP TABLE IF EXISTS mfa_totp_backup_codes;
DROP TABLE IF EXISTS mfa_totp_credentials;
```

000007_create_api_keys.up.sql

```
CREATE TABLE api_keys (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
  org_id            UUID,
  name              VARCHAR(100) NOT NULL,
  description       TEXT,
  key_hash          VARCHAR(255) NOT NULL UNIQUE,
  key_prefix        VARCHAR(20) NOT NULL,
  scopes            TEXT[] NOT NULL DEFAULT '{}',
  allowed_ips       CIDR[] DEFAULT '{}',
  last_used_at      TIMESTAMP WITH TIME ZONE,
```

```
  last_used_ip     INET,
  revoked          BOOLEAN NOT NULL DEFAULT FALSE,
  revoked_at       TIMESTAMP WITH TIME ZONE,
  revoked_reason   VARCHAR(255),
  expires_at       TIMESTAMP WITH TIME ZONE,
  created_at       TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  updated_at       TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_api_keys_user_id ON api_keys(user_id) WHERE
        revoked = FALSE;
CREATE INDEX idx_api_keys_key_hash ON api_keys(key_hash);
```

000007_create_api_keys.down.sql

```
DROP TABLE IF EXISTS api_keys;
```

000008_create_login_history.up.sql

```
CREATE TABLE login_history (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
  event_type        VARCHAR(50) NOT NULL,
  ip_address        INET,
  user_agent        TEXT,
  device_id         UUID,
  session_id        UUID,
  failure_reason    VARCHAR(255),
  metadata          JSONB DEFAULT '{}',
  occurred_at       TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW()
);

CREATE INDEX idx_login_history_user_id ON login_history(user_id);
CREATE INDEX idx_login_history_occurred_at ON login_history USING
        BRIN (occurred_at);
```

000008_create_login_history.down.sql

```
DROP TABLE IF EXISTS login_history;
```

000009_create_sso_tables.up.sql

```
CREATE TABLE sso_providers (
  id                    UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
  org_id                UUID NOT NULL,
  provider              VARCHAR(50) NOT NULL,
  slug                  VARCHAR(100) NOT NULL,
  display_name          VARCHAR(255),
  client_id             VARCHAR(512),
  encrypted_client_secret TEXT,
  discovery_url         TEXT,
  authorization_url     TEXT,
  token_url             TEXT,
  userinfo_url          TEXT,
  jwks_uri              TEXT,
  scopes                TEXT[] DEFAULT ARRAY['openid', 'email',
        'profile'],
  attribute_mapping     JSONB DEFAULT '{}',
  enabled               BOOLEAN NOT NULL DEFAULT TRUE,
  enforce_for_domain    VARCHAR(255),
  created_at            TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  updated_at            TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW()
);

CREATE UNIQUE INDEX idx_sso_providers_org_provider ON
        sso_providers(org_id, provider)
  WHERE enabled = TRUE;


CREATE TABLE sso_identities (
  id                      UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
  user_id                 UUID NOT NULL REFERENCES users(id) ON
        DELETE CASCADE,
  provider_id             UUID NOT NULL REFERENCES
        sso_providers(id) ON DELETE CASCADE,
  subject                 VARCHAR(512) NOT NULL,

        -- Envelope-encrypted at rest — see the canonical column
        comment in §17.1.
  encrypted_access_token  BYTEA,
  encrypted_refresh_token BYTEA,
  token_key_id            UUID,
  token_expires_at        TIMESTAMP WITH TIME ZONE,
  profile_data            JSONB DEFAULT '{}',
  created_at              TIMESTAMP WITH TIME ZONE NOT NULL
        DEFAULT NOW(),
```

```
  updated_at              TIMESTAMP WITH TIME ZONE NOT NULL
        DEFAULT NOW()
);

CREATE UNIQUE INDEX idx_sso_identities_provider_subject ON
        sso_identities(provider_id, subject);
```

000009_create_sso_tables.down.sql

```
DROP TABLE IF EXISTS sso_identities;
DROP TABLE IF EXISTS sso_providers;
```

000010_create_password_history.up.sql

```
CREATE TABLE password_history (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id       UUID NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
  password_hash VARCHAR(255) NOT NULL,
  created_at    TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_password_history_user_id ON
        password_history(user_id);
```

000010_create_password_history.down.sql

```
DROP TABLE IF EXISTS password_history;
```

000011_create_jwt_blocklist.up.sql

```
CREATE TABLE jwt_blocklist (
  jti         VARCHAR(255) PRIMARY KEY,
  user_id     UUID NOT NULL,
  revoked_at  TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  expires_at  TIMESTAMP WITH TIME ZONE NOT NULL,
  reason      VARCHAR(100)
);

CREATE INDEX idx_jwt_blocklist_expires_at ON jwt_blocklist USING
        BRIN (expires_at);
```

000011_create_jwt_blocklist.down.sql

```
DROP TABLE IF EXISTS jwt_blocklist;
```

000012_add_users_org_id.up.sql

```
-- Expand: add nullable column first (zero-downtime)
ALTER TABLE users ADD COLUMN IF NOT EXISTS org_id UUID;
CREATE INDEX idx_users_org_id ON users(org_id) WHERE org_id IS
        NOT NULL;
```

000012_add_users_org_id.down.sql

```
DROP INDEX IF EXISTS idx_users_org_id;
ALTER TABLE users DROP COLUMN IF EXISTS org_id;
```

000013_add_users_org_id_not_null.up.sql

```
-- Contract: after backfill is complete, add NOT NULL constraint
-- Run: UPDATE users SET org_id = (SELECT id FROM organizations
        WHERE owner_id = users.id LIMIT 1) WHERE org_id IS NULL;
ALTER TABLE users ALTER COLUMN org_id SET NOT NULL;
```

000013_add_users_org_id_not_null.down.sql

```
ALTER TABLE users ALTER COLUMN org_id DROP NOT NULL;
```

000014_create_ssh_sessions.up.sql

```
-- (In session_db)
CREATE TABLE ssh_sessions (
  id                    UUID NOT NULL DEFAULT gen_random_uuid(),
  user_id               UUID NOT NULL,
  host_id               UUID NOT NULL,
  vault_id              UUID NOT NULL,
  org_id                UUID NOT NULL,
  client_ip             INET NOT NULL,
  user_agent            TEXT,
  terminal_cols         SMALLINT NOT NULL DEFAULT 80,
  terminal_rows         SMALLINT NOT NULL DEFAULT 24,
  terminal_type         VARCHAR(50) NOT NULL DEFAULT
        'xterm-256color',
  auth_method           VARCHAR(20),
  key_id                UUID,
  recording_enabled     BOOLEAN NOT NULL DEFAULT FALSE,
  recording_path        TEXT,
  recording_size_bytes  BIGINT DEFAULT 0,
  collab_enabled        BOOLEAN NOT NULL DEFAULT FALSE,
  read_only             BOOLEAN NOT NULL DEFAULT FALSE,
  status                VARCHAR(20) NOT NULL DEFAULT 'connecting',
```

```
  reason                TEXT,
  ticket_ref            VARCHAR(255),
  startup_snippet_id    UUID,
  jump_chain            JSONB DEFAULT '[]',
  exit_code             INTEGER,
  disconnect_reason     TEXT,
  bytes_sent            BIGINT NOT NULL DEFAULT 0,
  bytes_received        BIGINT NOT NULL DEFAULT 0,
  commands_count        INTEGER NOT NULL DEFAULT 0,
  resize_count          INTEGER NOT NULL DEFAULT 0,
  started_at            TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  connected_at          TIMESTAMP WITH TIME ZONE,
  ended_at              TIMESTAMP WITH TIME ZONE,
  duration_seconds      INTEGER,
  PRIMARY KEY (id, started_at)
) PARTITION BY RANGE (started_at);

CREATE TABLE ssh_sessions_2026_q2 PARTITION OF ssh_sessions
  FOR VALUES FROM ('2026-04-01') TO ('2026-07-01');
CREATE TABLE ssh_sessions_2026_q3 PARTITION OF ssh_sessions
  FOR VALUES FROM ('2026-07-01') TO ('2026-10-01');
CREATE TABLE ssh_sessions_2026_q4 PARTITION OF ssh_sessions
  FOR VALUES FROM ('2026-10-01') TO ('2027-01-01');

CREATE INDEX idx_ssh_sessions_user_id ON ssh_sessions(user_id,
        started_at DESC);
CREATE INDEX idx_ssh_sessions_host_id ON ssh_sessions(host_id,
        started_at DESC);
CREATE INDEX idx_ssh_sessions_status ON ssh_sessions(status,
        started_at DESC)
  WHERE status IN ('connecting', 'connected');
```

000014_create_ssh_sessions.down.sql

```
DROP TABLE IF EXISTS ssh_sessions CASCADE;
```

000015_create_port_forward_rules.up.sql

```
CREATE TABLE port_forward_rules (
  id                    UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
  user_id               UUID NOT NULL,
  host_id               UUID NOT NULL,
  vault_id              UUID NOT NULL,
  org_id                UUID NOT NULL,
```

```
  name                  VARCHAR(255) NOT NULL,
  description           TEXT,
  type                  VARCHAR(20) NOT NULL CHECK (type IN
        ('local', 'remote', 'dynamic')),
  local_address         VARCHAR(255) NOT NULL DEFAULT '127.0.0.1',
  local_port            INTEGER NOT NULL CHECK (local_port
        BETWEEN 1 AND 65535),
  remote_address        VARCHAR(255),
  remote_port           INTEGER CHECK (remote_port BETWEEN 1 AND
        65535),
  bind_address          VARCHAR(255),
  auto_start            BOOLEAN NOT NULL DEFAULT FALSE,
  status                VARCHAR(20) NOT NULL DEFAULT 'inactive',
  sort_order            INTEGER NOT NULL DEFAULT 0,
  created_at            TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  updated_at            TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  deleted_at            TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_pf_rules_user_id ON port_forward_rules(user_id)
        WHERE deleted_at IS NULL;
CREATE INDEX idx_pf_rules_host_id ON port_forward_rules(host_id)
        WHERE deleted_at IS NULL;
```

000015_create_port_forward_rules.down.sql

```
DROP TABLE IF EXISTS port_forward_rules;
```

000016_create_hosts.up.sql

```
-- (In host_db)
CREATE EXTENSION IF NOT EXISTS "pg_trgm";

CREATE TABLE hosts (
  id                        UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
  vault_id                  UUID NOT NULL,
  group_id                  UUID,
  org_id                    UUID NOT NULL,
  created_by                UUID NOT NULL,
  name                      VARCHAR(255) NOT NULL,
  hostname                  VARCHAR(512) NOT NULL,
  port                      INTEGER NOT NULL DEFAULT 22 CHECK
        (port BETWEEN 1 AND 65535),
  username                  VARCHAR(255),
  auth_method               VARCHAR(20) NOT NULL DEFAULT 'key',
```

```
  key_id                      UUID,
  description                 TEXT,
  color                       VARCHAR(20),
  icon                        VARCHAR(50),
  tags                        TEXT[] NOT NULL DEFAULT '{}',
  jump_host_id                UUID,
  connection_timeout_seconds  INTEGER NOT NULL DEFAULT 30,
  keepalive_interval_seconds  INTEGER NOT NULL DEFAULT 60,
  status                      VARCHAR(20) NOT NULL DEFAULT
        'active',
  last_connected_at           TIMESTAMP WITH TIME ZONE,
  fingerprint_verified        BOOLEAN NOT NULL DEFAULT FALSE,
  environment_variables       JSONB NOT NULL DEFAULT '{}',
  custom_fields               JSONB NOT NULL DEFAULT '{}',
  sort_order                  INTEGER NOT NULL DEFAULT 0,
  created_at                  TIMESTAMP WITH TIME ZONE NOT NULL
        DEFAULT NOW(),
  updated_at                  TIMESTAMP WITH TIME ZONE NOT NULL
        DEFAULT NOW(),
  deleted_at                  TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_hosts_vault_id ON hosts(vault_id) WHERE
        deleted_at IS NULL;
CREATE INDEX idx_hosts_org_id ON hosts(org_id) WHERE deleted_at
        IS NULL;
CREATE INDEX idx_hosts_tags ON hosts USING GIN (tags);
CREATE INDEX idx_hosts_name_trgm ON hosts USING GIN (name
        gin_trgm_ops) WHERE deleted_at IS NULL;
```

000016_create_hosts.down.sql

```
DROP TABLE IF EXISTS hosts;
```

000017_create_vaults.up.sql

```
-- (In vault_db)
CREATE TABLE vaults (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id          UUID NOT NULL,
  owner_id        UUID NOT NULL,
  name            VARCHAR(255) NOT NULL,
  description     TEXT,
  color           VARCHAR(20),
  icon            VARCHAR(50),
  encrypted       BOOLEAN NOT NULL DEFAULT TRUE,
  sync_enabled    BOOLEAN NOT NULL DEFAULT TRUE,
```

```
  item_count      INTEGER NOT NULL DEFAULT 0,
  storage_bytes   BIGINT NOT NULL DEFAULT 0,
  version         BIGINT NOT NULL DEFAULT 1,
  created_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  updated_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  deleted_at      TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_vaults_org_id ON vaults(org_id) WHERE deleted_at
        IS NULL;
CREATE INDEX idx_vaults_owner_id ON vaults(owner_id) WHERE
        deleted_at IS NULL;
```

000017_create_vaults.down.sql

```
DROP TABLE IF EXISTS vaults;
```

000018_create_audit_events.up.sql

```
-- (In audit_db) — audit_pii_keys (§17.10.1) MUST be migrated in
        an earlier
-- numbered file (e.g. 000017a_create_audit_pii_keys.up.sql) since
        pii_key_id
-- below is a NOT NULL FK into it.
CREATE TABLE audit_events (
  id              UUID NOT NULL DEFAULT gen_random_uuid(),
  seq             BIGSERIAL,
  org_id          UUID NOT NULL,
  event_type      VARCHAR(100) NOT NULL,
  user_id         UUID,
  resource_type   VARCHAR(50),
  resource_id     UUID,
  outcome         VARCHAR(20) NOT NULL DEFAULT 'success',
  session_id      UUID,
  source_service  VARCHAR(50) NOT NULL,
  metadata        JSONB NOT NULL DEFAULT '{}',
  -- Envelope-encrypted PII — see the canonical column comment in
        §17.10.1.
  ip_address      BYTEA,
  user_agent      BYTEA,
  resource_name   BYTEA,
  pii_key_id      UUID NOT NULL REFERENCES audit_pii_keys(id),
  hash            VARCHAR(128) NOT NULL,
  prev_hash       VARCHAR(128),
  occurred_at     TIMESTAMP WITH TIME ZONE NOT NULL,
  recorded_at     TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
```

```
  PRIMARY KEY (id, occurred_at)
) PARTITION BY RANGE (occurred_at);

CREATE TABLE audit_events_2026_06 PARTITION OF audit_events
  FOR VALUES FROM ('2026-06-01') TO ('2026-07-01');
CREATE TABLE audit_events_2026_07 PARTITION OF audit_events
  FOR VALUES FROM ('2026-07-01') TO ('2026-08-01');

CREATE INDEX idx_audit_events_org_id ON audit_events(org_id,
        occurred_at DESC);
CREATE INDEX idx_audit_events_occurred_at ON audit_events USING
        BRIN (occurred_at);
CREATE INDEX idx_audit_events_pii_key_id ON
        audit_events(pii_key_id);
```

000018_create_audit_events.down.sql

```
DROP TABLE IF EXISTS audit_events CASCADE;
```

000019_create_organizations.up.sql

```
-- (In org_db)
CREATE EXTENSION IF NOT EXISTS "citext";

CREATE TABLE organizations (
  id                              UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
  name                            VARCHAR(255) NOT NULL,
  slug                            CITEXT NOT NULL UNIQUE,
  domain                          CITEXT,
  domain_verified                 BOOLEAN NOT NULL DEFAULT FALSE,
  plan                            VARCHAR(50) NOT NULL DEFAULT
        'free',
  max_members                     INTEGER NOT NULL DEFAULT 5,
  enforce_mfa                     BOOLEAN NOT NULL DEFAULT FALSE,
  session_recording_required      BOOLEAN NOT NULL DEFAULT FALSE,
  owner_id                        UUID NOT NULL,
  status                          VARCHAR(20) NOT NULL DEFAULT
        'active',
  settings                        JSONB NOT NULL DEFAULT '{}',
  created_at                      TIMESTAMP WITH TIME ZONE NOT
        NULL DEFAULT NOW(),
  updated_at                      TIMESTAMP WITH TIME ZONE NOT
        NULL DEFAULT NOW(),
  deleted_at                      TIMESTAMP WITH TIME ZONE
);
```

```
CREATE INDEX idx_organizations_slug ON organizations(slug) WHERE
        deleted_at IS NULL;
CREATE INDEX idx_organizations_owner_id ON
        organizations(owner_id);
```

000019_create_organizations.down.sql

```
DROP TABLE IF EXISTS organizations;
```

000020_create_snippets.up.sql

```
-- (In snippet_db)
CREATE EXTENSION IF NOT EXISTS "pg_trgm";

CREATE TABLE snippets (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vault_id            UUID NOT NULL,
  org_id              UUID NOT NULL,
  created_by          UUID NOT NULL,
  category_id         UUID,
  name                VARCHAR(255) NOT NULL,
  description         TEXT,
  content             TEXT NOT NULL,
  language            VARCHAR(50) NOT NULL DEFAULT 'bash',
  tags                TEXT[] NOT NULL DEFAULT '{}',
  parameters          JSONB NOT NULL DEFAULT '[]',
  shared              BOOLEAN NOT NULL DEFAULT FALSE,
  pinned              BOOLEAN NOT NULL DEFAULT FALSE,
  executions_count    BIGINT NOT NULL DEFAULT 0,
  last_executed_at    TIMESTAMP WITH TIME ZONE,
  fts_vector          TSVECTOR,
  created_at          TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  updated_at          TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT
        NOW(),
  deleted_at          TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_snippets_vault_id ON snippets(vault_id) WHERE
        deleted_at IS NULL;
CREATE INDEX idx_snippets_fts ON snippets USING GIN (fts_vector)
        WHERE deleted_at IS NULL;
CREATE INDEX idx_snippets_name_trgm ON snippets USING GIN (name
        gin_trgm_ops) WHERE deleted_at IS NULL;

CREATE OR REPLACE FUNCTION snippets_fts_update()
RETURNS TRIGGER AS $$
```

```
BEGIN
  NEW.fts_vector :=
    setweight(to_tsvector('english', coalesce(NEW.name, '')),
        'A') ||
    setweight(to_tsvector('english', coalesce(NEW.description,
        '')), 'B') ||
    setweight(to_tsvector('english', coalesce(NEW.content, '')),
        'C');
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_snippets_fts
  BEFORE INSERT OR UPDATE ON snippets
  FOR EACH ROW EXECUTE FUNCTION snippets_fts_update();
```

000020_create_snippets.down.sql

```
DROP TRIGGER IF EXISTS trg_snippets_fts ON snippets;
DROP FUNCTION IF EXISTS snippets_fts_update();
DROP TABLE IF EXISTS snippets;
```

---

20. Performance Indexes

This section documents the comprehensive indexing strategy for all HelixTerminator databases. Indexes are chosen based on actual query patterns observed in production workloads.

20.1 Index Strategy Overview

Index selection criteria: 1. **Selectivity:** Index columns with high cardinality first. 2. **Query frequency:** Index columns that appear in `WHERE` clauses of frequent queries. 3. **Write overhead:** Every index adds ~10-30% write overhead; only create indexes that pay their cost. 4. **Composite indexes:** Leading column matters — place the most selective or equality-matched column first. 5. **Partial indexes:** Use `WHERE` conditions to reduce index size for sparse columns (e.g., `WHERE deleted_at IS NULL`).

20.2 Composite Indexes for Common Query Patterns

auth_db

```
-- Find all active sessions for a user, ordered by creation time
        (session list page)
CREATE INDEX idx_user_sessions_user_status_created
  ON user_sessions(user_id, status, created_at DESC)
  WHERE status = 'active';

-- Find all non-revoked API keys for a user+org combination
CREATE INDEX idx_api_keys_user_org_active
  ON api_keys(user_id, org_id, created_at DESC)
  WHERE revoked = FALSE;

-- Login history for a user with time range filter (audit page)
CREATE INDEX idx_login_history_user_type_time
  ON login_history(user_id, event_type, occurred_at DESC);

-- SSO identity lookup by provider and subject (login flow)
CREATE INDEX idx_sso_identities_provider_subject_user
  ON sso_identities(provider_id, subject, user_id);
```

host_db

```
-- Host listing with vault + status + sort (main host list)
CREATE INDEX idx_hosts_vault_status_sort
  ON hosts(vault_id, status, sort_order ASC, name ASC)
  WHERE deleted_at IS NULL;

-- Host listing with group filter (group view)
CREATE INDEX idx_hosts_group_sort
  ON hosts(group_id, sort_order ASC)
  WHERE deleted_at IS NULL AND group_id IS NOT NULL;

-- Recent connections across all hosts for a user (dashboard)
CREATE INDEX idx_host_conn_history_user_recent
  ON host_connection_history(user_id, started_at DESC, host_id);

-- Recent connections to a specific host (host detail page)
CREATE INDEX idx_host_conn_history_host_recent
  ON host_connection_history(host_id, started_at DESC, user_id);

-- Hosts by tag (tag filtering)
```

```sql
CREATE INDEX idx_hosts_vault_tags
  ON hosts USING GIN (tags)
  WHERE deleted_at IS NULL;

-- Jump host chain lookup
CREATE INDEX idx_hosts_jump_host
  ON hosts(jump_host_id, vault_id)
  WHERE jump_host_id IS NOT NULL AND deleted_at IS NULL;

-- Host group hierarchy traversal
CREATE INDEX idx_host_groups_parent_vault_sort
  ON host_groups(parent_id, vault_id, sort_order ASC)
  WHERE deleted_at IS NULL;
```

session_db

```sql
-- Active sessions for a host (live monitoring view)
CREATE INDEX idx_ssh_sessions_host_active
  ON ssh_sessions(host_id, status, started_at DESC)
  WHERE status IN ('connecting', 'connected');

-- Session list for an org with time filter (admin view)
CREATE INDEX idx_ssh_sessions_org_time
  ON ssh_sessions(org_id, started_at DESC, user_id);

-- Recording availability check
CREATE INDEX idx_session_recordings_session
  ON session_recordings(session_id, processed)
  WHERE processed = TRUE;

-- SFTP transfers for a session (transfer log)
CREATE INDEX idx_sftp_transfers_session_time
  ON sftp_transfers(sftp_session_id, transferred_at DESC);

-- Port forward rules by host and status (proxy startup)
CREATE INDEX idx_pf_rules_host_status
  ON port_forward_rules(host_id, status, auto_start)
  WHERE deleted_at IS NULL;
```

audit_db

```sql
-- Audit event query: org + time range (most common query pattern)
CREATE INDEX idx_audit_events_org_time
  ON audit_events(org_id, occurred_at DESC);
```

```sql
-- Audit event query: org + event_type + time range
CREATE INDEX idx_audit_events_org_type_time
  ON audit_events(org_id, event_type, occurred_at DESC);

-- Audit event query: org + user + time range
CREATE INDEX idx_audit_events_org_user_time
  ON audit_events(org_id, user_id, occurred_at DESC)
  WHERE user_id IS NOT NULL;

-- Audit event query: resource lookup (investigate specific
       resource)
CREATE INDEX idx_audit_events_resource_time
  ON audit_events(resource_type, resource_id, occurred_at DESC)
  WHERE resource_type IS NOT NULL;

-- Audit event query: outcome filter (failure analysis)
CREATE INDEX idx_audit_events_org_outcome_time
  ON audit_events(org_id, outcome, occurred_at DESC)
  WHERE outcome = 'failure';
```

snippet_db

```sql
-- Snippet list: vault + category (main list)
CREATE INDEX idx_snippets_vault_category_sort
  ON snippets(vault_id, category_id, name ASC)
  WHERE deleted_at IS NULL;

-- Snippet list: most recently used
CREATE INDEX idx_snippets_vault_last_executed
  ON snippets(vault_id, last_executed_at DESC NULLS LAST)
  WHERE deleted_at IS NULL;

-- Execution results: lookup by execution ID + status
CREATE INDEX idx_exec_results_exec_status
  ON snippet_execution_results(execution_id, status);
```

org_db

```sql
-- Member list: org + role + status
CREATE INDEX idx_org_members_org_role_status
  ON org_members(org_id, role, status, joined_at DESC);

-- Member lookup: find all orgs for a user
```

```
CREATE INDEX idx_org_members_user_status
  ON org_members(user_id, status);

-- Invitation lookup: pending invitations for an org
CREATE INDEX idx_invitations_org_status_expires
  ON invitations(org_id, status, expires_at DESC)
  WHERE status = 'pending';

-- Team membership: all teams for a user
CREATE INDEX idx_team_members_user_team
  ON team_members(user_id, team_id);
```

20.3 Partial Indexes for Soft-Delete Patterns

Partial indexes with `WHERE deleted_at IS NULL` are applied to every table with soft deletion. This dramatically reduces index size because deleted records (typically 5–15% of total) are excluded.

```
-- Soft-delete partial indexes (applied to all soft-delete tables)
-- These are already included in the schema definitions above.
-- The pattern is consistently: CREATE INDEX ... WHERE deleted_at
       IS NULL

-- Additional covering partial indexes for read performance:

-- Hosts: active hosts with key columns for list rendering
CREATE INDEX idx_hosts_active_list_covering
  ON hosts(vault_id, sort_order, name, hostname, status,
       last_connected_at)
  WHERE deleted_at IS NULL AND status != 'archived';

-- API keys: non-expired, non-revoked keys for lookup
CREATE INDEX idx_api_keys_active
  ON api_keys(key_hash, user_id, scopes)
  WHERE revoked = FALSE AND (expires_at IS NULL OR expires_at >
       NOW());
```

20.4 GIN Indexes for JSONB Columns

```
-- Hosts: environment_variables (for searching hosts by env var
       key)
CREATE INDEX idx_hosts_env_vars_gin
  ON hosts USING GIN (environment_variables)
  WHERE deleted_at IS NULL;
```

```
-- Hosts: custom_fields (for custom attribute search)
CREATE INDEX idx_hosts_custom_fields_gin
  ON hosts USING GIN (custom_fields)
  WHERE deleted_at IS NULL;

-- Audit events: metadata search (IP address, resource metadata)
CREATE INDEX idx_audit_events_metadata_gin
  ON audit_events USING GIN (metadata);

-- Snippets: parameters (search snippets with specific parameter
        names)
CREATE INDEX idx_snippets_parameters_gin
  ON snippets USING GIN (parameters)
  WHERE deleted_at IS NULL;

-- Organizations: settings (feature flags, custom configuration)
CREATE INDEX idx_organizations_settings_gin
  ON organizations USING GIN (settings);

-- Workspace layout (find workspaces by host ID embedded in layout
        JSON)
CREATE INDEX idx_workspaces_layout_gin
  ON workspaces USING GIN (layout)
  WHERE deleted_at IS NULL;

-- SSH sessions: jump_chain (find sessions through a specific jump
        host)
CREATE INDEX idx_ssh_sessions_jump_chain_gin
  ON ssh_sessions USING GIN (jump_chain);

-- PKI certificates: extensions
CREATE INDEX idx_certificates_extensions_gin
  ON certificates USING GIN (extensions);

-- Roles: permissions array
CREATE INDEX idx_roles_permissions_gin
  ON roles USING GIN (permissions);
```

20.5 BRIN Indexes for Time-Series Data

BRIN (Block Range INdex) indexes are used for append-only, time-ordered tables where rows are inserted in roughly chronological order. They are tiny (a few KB for millions of rows) but very effective for range queries.

```
-- BRIN indexes are appropriate for naturally time-ordered tables:

-- auth_db
CREATE INDEX idx_users_created_at_brin ON users USING BRIN
        (created_at);
CREATE INDEX idx_login_history_occurred_at_brin ON login_history
        USING BRIN (occurred_at);
CREATE INDEX idx_refresh_tokens_expires_at_brin ON refresh_tokens
        USING BRIN (expires_at);

-- host_db
CREATE INDEX idx_host_conn_history_started_at_brin
  ON host_connection_history USING BRIN (started_at);

-- session_db
CREATE INDEX idx_ssh_sessions_started_at_brin ON ssh_sessions
        USING BRIN (started_at);
CREATE INDEX idx_session_events_occurred_at_brin ON
        session_events USING BRIN (occurred_at);
CREATE INDEX idx_sftp_transfers_transferred_at_brin ON
        sftp_transfers USING BRIN (transferred_at);

-- audit_db
CREATE INDEX idx_audit_events_occurred_at_brin ON audit_events
        USING BRIN (occurred_at);
CREATE INDEX idx_audit_events_recorded_at_brin ON audit_events
        USING BRIN (recorded_at);

-- keychain_db
CREATE INDEX idx_key_usage_occurred_at_brin ON key_usage_log
        USING BRIN (occurred_at);

-- vault_db
CREATE INDEX idx_vault_audit_occurred_at_brin ON
        vault_audit_events USING BRIN (occurred_at);
```

BRIN index parameters: - Default `pages_per_range = 128` (1 MB ranges) - For high-insertion-rate tables: `WITH (pages_per_range = 64)` - For long-term archives: `WITH (pages_per_range = 256)`

20.6 pg_trgm Indexes for Full-Text Search

Trigram indexes enable efficient `LIKE '%pattern%'` and similarity search. They back the `?q=` search parameter on host, snippet, and key endpoints.

```
-- host_db: trigram search on name and hostname
CREATE INDEX idx_hosts_name_trgm ON hosts USING GIN (name
        gin_trgm_ops)
```

```
  WHERE deleted_at IS NULL;
CREATE INDEX idx_hosts_hostname_trgm ON hosts USING GIN (hostname
        gin_trgm_ops)
  WHERE deleted_at IS NULL;
CREATE INDEX idx_hosts_description_trgm ON hosts USING GIN
        (description gin_trgm_ops)
  WHERE deleted_at IS NULL AND description IS NOT NULL;
CREATE INDEX idx_host_groups_name_trgm ON host_groups USING GIN
        (name gin_trgm_ops)
  WHERE deleted_at IS NULL;

-- keychain_db: trigram search on key names
CREATE INDEX idx_ssh_keys_name_trgm ON ssh_keys USING GIN (name
        gin_trgm_ops)
  WHERE deleted_at IS NULL;

-- snippet_db: trigram for content search
CREATE INDEX idx_snippets_name_trgm ON snippets USING GIN (name
        gin_trgm_ops)
  WHERE deleted_at IS NULL;
CREATE INDEX idx_snippets_content_trgm ON snippets USING GIN
        (content gin_trgm_ops)
  WHERE deleted_at IS NULL;

-- vault_db: vault name search
CREATE INDEX idx_vaults_name_trgm ON vaults USING GIN (name
        gin_trgm_ops)
  WHERE deleted_at IS NULL;

-- org_db: organization and team name search
CREATE INDEX idx_organizations_name_trgm ON organizations USING
        GIN (name gin_trgm_ops)
  WHERE deleted_at IS NULL;
CREATE INDEX idx_teams_name_trgm ON teams USING GIN (name
        gin_trgm_ops)
  WHERE deleted_at IS NULL;

-- Similarity threshold configuration:
-- SET pg_trgm.similarity_threshold = 0.3;
-- Used in queries like: WHERE name % 'search_term' (similarity
        operator)
-- Or: WHERE name ILIKE '%search_term%' (uses GIN trigram
        automatically)
```

20.7 Expression and Covering Indexes

```
-- Lowercase email lookup (case-insensitive search without CITEXT)
-- (Not needed if using CITEXT, but useful for non-CITEXT columns)
```

```
CREATE INDEX idx_users_email_lower ON users(lower(email::TEXT))
  WHERE deleted_at IS NULL;

-- Expired sessions cleanup (maintenance queries)
CREATE INDEX idx_user_sessions_expired
  ON user_sessions(expires_at)
  WHERE status = 'active' AND expires_at < NOW();

-- API key by prefix (for user display — no full hash needed)
CREATE INDEX idx_api_keys_prefix ON api_keys(key_prefix, user_id)
  WHERE revoked = FALSE;

-- Vault items by type (counting items per type for vault stats)
CREATE INDEX idx_vault_items_vault_type
  ON vault_items(vault_id, item_type)
  WHERE is_deleted = FALSE;

-- Covering index for session list rendering (avoids heap fetch
       for common columns)
CREATE INDEX idx_ssh_sessions_list_covering
  ON ssh_sessions(user_id, started_at DESC)
  INCLUDE (host_id, status, recording_enabled, ticket_ref,
       ended_at)
  WHERE started_at > NOW() - INTERVAL '90 days';

-- PKI certificate validity check
CREATE INDEX idx_certificates_validity
  ON certificates(valid_before, valid_after, revoked)
  WHERE revoked = FALSE AND valid_before > NOW();
```

20.8 Index Maintenance

`pg_stat_user_indexes` is monitored weekly. Indexes with zero `idx_scan` for 30 days are candidates for removal after review.

`REINDEX CONCURRENTLY` is run monthly for: - Tables with high UPDATE rates (sessions, hosts, vault_items) - After large bulk operations

`VACUUM ANALYZE` is configured via `autovacuum` with aggressive settings for: - `audit_events` partitions: `autovacuum_vacuum_scale_factor = 0.01` - `session_events` partitions: `autovacuum_vacuum_scale_factor = 0.005` - `host_connection_history` partitions: `autovacuum_analyze_scale_factor = 0.01`

Partition maintenance script (run monthly via cron):

```sql
-- Create next quarter's partition in advance
CREATE TABLE IF NOT EXISTS ssh_sessions_2027_q1 PARTITION OF
        ssh_sessions
  FOR VALUES FROM ('2027-01-01') TO ('2027-04-01');

-- Archive old partitions (detach after retention period)
ALTER TABLE host_connection_history DETACH PARTITION
        host_connection_history_2025_q1;
-- Move to cold storage or drop

-- Update statistics on large tables after bulk operations
ANALYZE VERBOSE ssh_sessions;
ANALYZE VERBOSE audit_events;
ANALYZE VERBOSE host_connection_history;
```

End of HelixTerminator API & Database Specification

Document Information:

Attribute	Value
Document Version	1.0.0
Project	HelixTerminator
Module	`helixterminator.io/core`
Backend	Go 1.25, Gin Gonic
Database	PostgreSQL 17.2, Redis 8, SQLite (dev)
Prepared By	Engineering Team
Classification	Internal — Engineering Confidential
Last Updated	2026-06-28